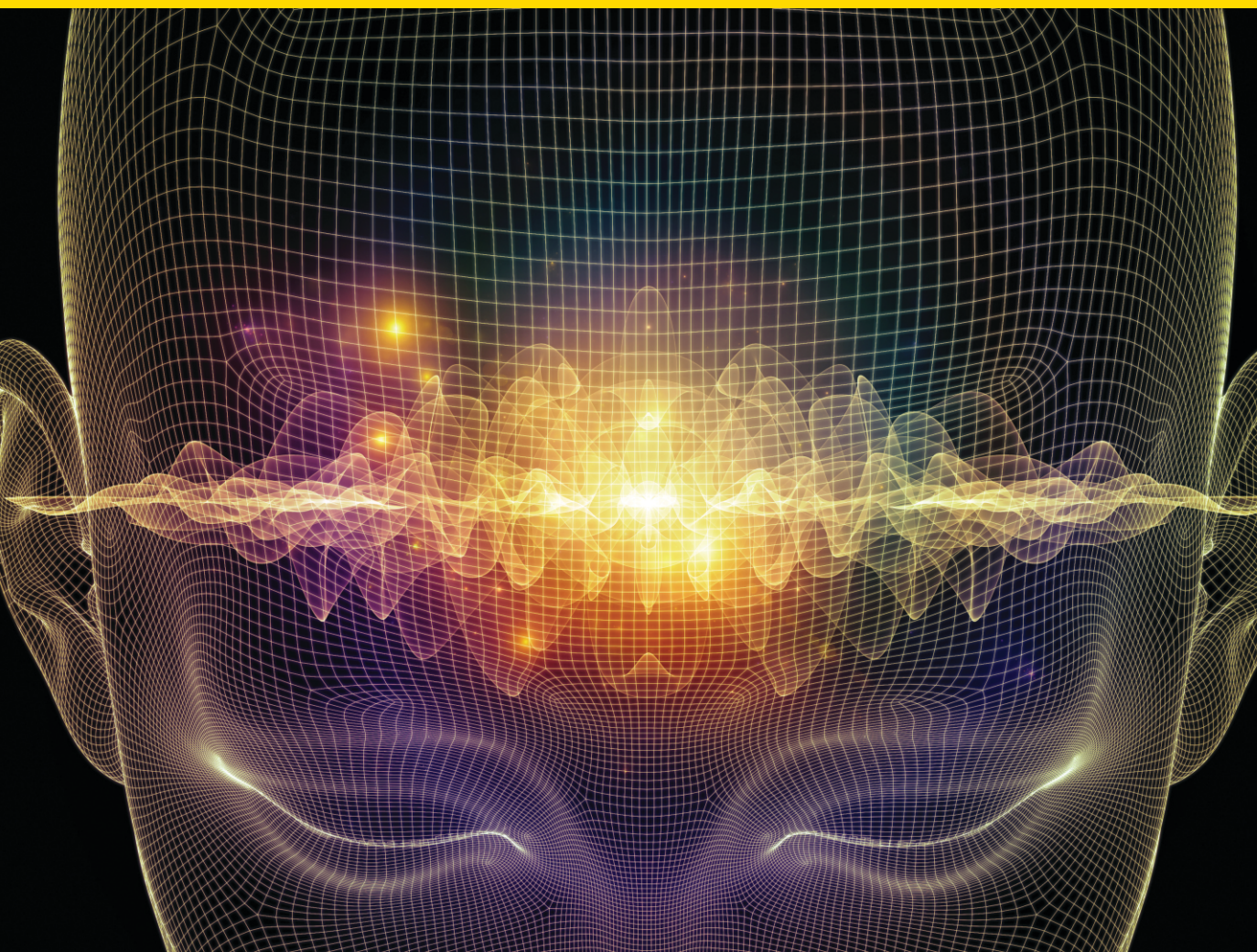




DEEP LEARNING USING PYTHON

Dr. S Lovelyn Rose | Dr. L Ashok Kumar | Dr. D Karthika Renuka

WILEY

DEEP LEARNING USING PYTHON

DEEP LEARNING USING PYTHON

Dr. S Lovelyn Rose

Associate Professor

Department of Computer Science & Engineering

PSG College of Technology, Peelamedu

Coimbatore

Dr. L Ashok Kumar

Professor and Associate HoD

Department of Electrical & Electronics Engineering

PSG College of Technology, Peelamedu

Coimbatore

Dr. D Karthika Renuka

Associate Professor

Department of Information Technology

PSG College of Technology, Peelamedu

Coimbatore

WILEY

DEEP LEARNING USING PYTHON

Copyright © 2019 by Wiley India Pvt. Ltd., 4436/7, Ansari Road, Daryaganj, New Delhi-110002.

Cover Image: © Shutterstock

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or scanning without the written permission of the publisher.

Limits of Liability: While the publisher and the author have used their best efforts in preparing this book, Wiley and the author make no representation or warranties with respect to the accuracy or completeness of the contents of this book, and specifically disclaim any implied warranties of merchantability or fitness for any particular purpose. There are no warranties which extend beyond the descriptions contained in this paragraph. No warranty may be created or extended by sales representatives or written sales materials. The accuracy and completeness of the information provided herein and the opinions stated herein are not guaranteed or warranted to produce any particular results, and the advice and strategies contained herein may not be suitable for every individual. Neither Wiley India nor the author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

Disclaimer: The contents of this book have been checked for accuracy. Since deviations cannot be precluded entirely, Wiley or its author cannot guarantee full agreement. As the book is intended for educational purpose, Wiley or its author shall not be responsible for any errors, omissions or damages arising out of the use of the information contained in the book. This publication is designed to provide accurate and authoritative information with regard to the subject matter covered. It is sold on the understanding that the Publisher is not engaged in rendering professional services.

Trademarks: All brand names and product names used in this book are trademarks, registered trademarks, or trade names of their respective holders. All logos are standard trademarks of the companies. Wiley is not associated with any product or vendor mentioned in this book.

Other Wiley Editorial Offices:

John Wiley & Sons, Inc. 111 River Street, Hoboken, NJ 07030, USA

Wiley-VCH Verlag GmbH, Pappelallee 3, D-69469 Weinheim, Germany

John Wiley & Sons Australia Ltd, 42 McDougall Street, Milton, Queensland 4064, Australia

John Wiley & Sons (Asia) Pte Ltd, 1 Fusionopolis Walk #07-01 Solaris, South Tower, Singapore 138628

John Wiley & Sons Canada Ltd, 22 Worcester Road, Etobicoke, Ontario, Canada, M9W 1L1

First Edition: 2019

ISBN: 978-81-265-7991-4

ISBN: 978-81-265-8856-5 (ebk)

www.wileyindia.com

Printed at:

Dedication

To my father, Prof. P. Selwin Cecil Rose, and mother, Ms. P. Jayanthi Rose, who laid the foundation for all my successes.

– *Dr. S. Lovelyn Rose*

To my wife, Y. Uma Maheswari, and daughter, A. K. Sangamithra, for their constant support and love.

– *Dr. L. Ashok Kumar*

To my mother-in-law, Mrs. Padmavathi Rajalakshmanan, who always blesses me in all my endeavors.

– *Dr. D. Karthika Renuka*

Preface

Deep learning, which was a science proposed in the 1980s, has become a technology now. Deep learning has become a choice for any Artificial Intelligence (AI) problem because of its ability to learn well. It can be used in any application, ranging from simple applications such as image classification to complex real-time applications such as autonomous driving. Many Silicon Valley tech giants such as Google, Microsoft, Amazon and Facebook have been backing up deep learning since they have found extensive usage of neural networks in their products and solutions. Data analytics has a lot to do with these improvements.

Learning the concepts of deep learning is not enough to handle real-time cases. *Deep Learning Using Python* aims at giving a hands-on experience on common deep learning applications using popular frameworks of the most suitable language, Python. There is no need for a reader to be a professional programmer. All the codes are simple and the exhaustive comments will help readers to understand of the available code. The readers will be able to solve AI problems using neural networks and deep learning architectures.

The book has been divided into seven chapters. Chapter 1 elaborately deals with the fundamentals of deep learning, to enable any reader to understand the deep learning architectures elaborated in subsequent chapters. Chapter 2 deals with Convolutional Neural Networks (CNNs), which have proven to be very effective in the area of computer vision. Chapter 3 deals with Recurrent Neural Networks (RNNs) and its variants. The various types of autoencoders, which are a type of Artificial Neural Network used to learn efficient data encoding, are presented in Chapter 4. To learn the probability distribution over the set of inputs, Restricted Boltzmann Machine (RBM) is discussed in Chapter 5. Chapter 6 presents popular open source frameworks in Python for deep learning applications. Chapter 7 describes how to utilize the knowledge that you have gained from previous chapters in real-time applications.

For the reader to test his understanding of the chapters, review questions are given at the end of every chapter. To improve the thinking process and understanding, assignment problems of the apply-and-analyze type are also provided for every chapter.

INSTRUCTOR RESOURCES

The following resources are available for instructors on request. To register, log onto https://www.wileyindia.com/Instructor_Manuals/Register/login.php

1. Chapter-wise Solution Manuals
2. Chapter-wise PowerPoint Presentations (PPTs)

Dr. S. Lovelyn Rose
Dr. L. Ashok Kumar
Dr. D. Karthika Renuka

Acknowledgements

The authors are thankful to Shri L. Gopalakrishnan, Managing Trustee, PSG Institutions and Dr. R. Rudramoorthy, Principal, PSG College of Technology, Coimbatore, for their wholehearted cooperation and constant encouragement in this successful endeavor. The authors wish to acknowledge the role of DST-ICPS in sowing the seeds for a joint venture among the authors in the area of deep learning.

Dr. S. Lovelyn Rose wishes to thank her children D. Dalyn Joanna and D. Danyl Jonathan in being patient and giving her all the love, time and space to finish her work. She wishes to acknowledge her husband A. David for his support in the successful completion of this book.

Dr. L. Ashok Kumar would like to take this opportunity to acknowledge the people who helped him in completing this book. He is thankful to his wife, Y. Uma Maheswari, for her constant support during writing. He is also grateful to his daughter, A. K. Sangamithra, for her support; it helped him a lot in completing this work.

Dr. D. Karthika Renuka would like to express her gratitude to all the people who made this book possible. Special thanks to her research scholars and students Mr. G. Pooventhiran, Mr. S. Adarsh, and Mr. A. Sandeep for their efforts in implementing the programs. She would like to express her gratefulness to all her well-wishers and friends. Her special thanks to her brother, Dr. L. Ashok Kumar, for his incredible support and guidance. Her gratitude to her parents, Mr. N. Dhanaraj and Ms. D. Anuradha; her brother, Mr. Andy Dhanaraj; and sister, Dr. D. Krithika Renuka, for their constant support. Her heartfelt thanks to her husband, Mr. R. Sathish Kumar, and her dear daughter, Ms. P. S. Preethi, for their unconditional love which made her capable of achieving all her goals.

The authors thank Meenakshi, Yoofisaca and team from Wiley for their relentless efforts in bringing out this book.

The authors are thankful to the almighty God for His immeasurable blessing upon their lives.

About the Authors

Dr. S. Lovelyn Rose completed her PhD with Anna University, Chennai, under the faculty of Information and Communication Engineering. Her major interest is mathematics and she did her undergraduation in Mathematics from Nirmala College for Women and MCA from Government Arts College, Coimbatore. She acquired M.E in Computer Science and Engineering from PSG College of Technology and her professional career of 16 years has been with PSG College of Technology. She has authored a book *Data Structures*, which has been very well received. She has several publications in national and international journals and conferences. She offers consultancy to companies on machine learning and computer vision. Her subject expertise and interests include data structures and algorithms, algorithm design techniques, machine learning and deep learning. She has been a keynote speaker and resource person for various national and international conferences and workshops in her area of expertise.

Dr. L. Ashok Kumar was a postdoctoral research fellow from San Diego State University, California. He is a recipient of the BHAVAN fellowship from the Indo-US Science and Technology Forum and SYST Fellowship from DST, Govt. of India. His current research focuses on integration of renewable energy systems in the smart grid and wearable electronics. He has 3 years of industrial experience and 19 years of academic and research experience. He has published 187 technical papers in international and national journals and presented 197 papers in national and international conferences. He has completed 27 Government of India-funded projects, and currently 5 projects are in progress. His PhD work on wearable electronics earned him a National Award from ISTE, and he has received 24 awards on the national level. Dr. Kumar has seven patents to his credit. He has guided 98 graduate and postgraduate projects. He holds many prestigious positions in various national forums. He has visited many countries for institute industry collaboration and also as a keynote speaker. He has been an invited speaker in 178 programs. Moreover, he has organized 79 events, including conferences, workshops and seminars. He completed his graduation in Electrical and Electronics Engineering from University of Madras and his postgraduation from PSG College of Technology, India, and master's in Business Administration from IGNOU, New Delhi. After completion of his graduate degree, he joined as project engineer for Serval Paper Boards Ltd., Coimbatore (now ITC Unit, Kovai). Presently, he is working as a Professor and Associate HoD in the Department of EEE, PSG College of Technology. He is currently doing research work in wearable electronics, smart grid, solar PV, and wind energy systems. He is also a Certified Chartered Engineer and BSI Certified ISO 500001 2008 Lead Auditor. He has authored many books in his areas of interest, including *Computational Intelligence Paradigms for Optimization Problems Using MATLAB®/SIMULINK®*; *Electronics in Textiles and Clothing: Design, Products and Applications*; *Automation In Textile Machinery: Instrumentation And Control System Design Principles*; and *Computational Paradigm Techniques for Enhancing Electric Power Quality*, which were published by CRC Press; *Solar PV and Wind Energy Conversion Systems—An Introduction to Theory, Modeling with MATLAB/SIMULINK*, and *the Role of Soft Computing Techniques—Green Energy and Technology*, published by

Springer; and *Power Electronics with MATLAB*, published by Cambridge University Press. He has also authored a monograph on *Smart Textiles*; monograph on *Information Technology for Textiles*; and monograph on *Instrumentation & Textile Control Engineering*.

Dr. D. Karthika Renuka is working as Associate Professor in Department of Information Technology, PSG College of Technology. Her professional career of 15 years has been with PSG College of Technology. Her area of specialization includes Data Mining, Evolutionary Algorithms, Machine Learning and Deep Learning. She has organized an International Conference, a National Conference and several workshops. She is a reviewer for *Computer and Electrical Engineering*, Elsevier, Wiley book chapters, Springer Book Chapters "*Knowledge Computing and its Applications*". Her ongoing projects include UGC minor research project titled "Multiple Classifier for E-mail Spam Classification in a Distributed Environment for reducing Network Traffic and improving the Network Performance", AICTE – MODEROB project titled "Data Analysis with High Performance Analytics", DST-ICPS project titled "Design and Implementation of E-Learning System using Deep Learning Based on Audio-Video Speech Recognition for Hearing Impaired in Native Language". She is currently guiding 6 research scholars for their Ph.D under Anna University. She has published several papers in reputed national and international journals and conferences.

Contents

Dedication	v
Preface	vii
Acknowledgements	ix
About the Authors	xi

1. Fundamentals of Neural Networks	1
1.1 Introduction	1
1.2 Types of Machine Learning	1
1.2.1 Classification Problem	2
1.2.2 The Regression Problem	3
1.2.3 Overfitting and Underfitting	5
1.2.4 Bias and Variance	6
1.3 Overview of Artificial Neural Networks	7
1.3.1 Biological Neuron	7
1.3.2 Types of Artificial Neural Networks	8
1.3.3 Optimization Techniques	19
1.3.4 Vanishing Gradient Problem	34
1.3.5 Exploding Gradient Problem	34
1.3.6 Weight Initialization	35
1.3.7 What is Deep Learning?	36
Summary	37
Review Questions	37
Assignment Problems	37
References	38

2. Convolutional Neural Network	41
2.1 Introduction	41
2.2 Components of CNN Architecture	42
2.2.1 Convolution Layer	44
2.2.2 Pooling or Downsampling Layer	48
2.2.3 Flattening Layer	51
2.2.4 Fully Connected Layer	51

2.3	Rectified Linear Unit (ReLU) Layer	52
2.3.1	Leaky ReLU and Randomized ReLU	52
2.4	Exponential Linear Unit (ELU, or SELU)	52
2.4.1	Maxout	53
2.5	Unique Properties of CNN	53
2.5.1	Weight Sharing	53
2.5.2	Translation Invariance	54
2.6	Architectures of CNN	54
2.6.1	LeNet	54
2.6.2	AlexNet	56
2.6.3	ZFNet	56
2.6.4	GoogLeNet	56
2.6.5	VGGNet	56
2.6.6	ResNet	61
2.6.7	DenseNet	61
2.7	Applications of CNN	61
2.7.1	Object Detection	61
2.7.2	Face Recognition	61
2.7.3	Scene Labeling	65
2.7.4	Optical Character Recognition (OCR)	66
2.7.5	Handwritten Digit Recognition	66
	Summary	66
	Review Questions	67
	Assignment Problems	67
	References	67

3. Recurrent Neural Network: Basic Concepts

69

3.1	Introduction	69
3.1.1	RNN versus CNN	70
3.1.2	Feedforward Neural Network versus RNN	70
3.2	Simple Recurrent Neural Network	71
3.2.1	Training an RNN	73
3.2.2	Backpropagation Through Time (BPTT) Illustration	74
3.2.3	RNN Topology	75
3.2.4	Challenges with Vanishing Gradients	77
3.2.5	Bidirectional and Stateful RNNs	79
3.2.6	Long Short-Term Memory (LSTM)	80

3.3	LSTM Implementation	85
3.4	Gated Recurrent Unit (GRU)	85
3.5	Deep Recurrent Neural Network	86
	Summary	88
	Review Questions	88
	Assignment Problems	88
	References	89

4. Autoencoder 91

4.1	Introduction	91
4.2	Features of Autoencoder	92
4.3	Types of Autoencoder	93
4.3.1	Vanilla Autoencoder	93
4.3.2	Multilayer Autoencoder	95
4.3.3	Stacked Autoencoder	95
4.3.4	Deep Autoencoder	97
4.3.5	Denoising Autoencoder	98
4.3.6	Convolutional Autoencoder	99
4.3.7	Regularization in Autoencoder (Regularized Autoencoder)	100
	Summary	106
	Review Questions	107
	Assignment Problems	107
	References	107

5. Restricted Boltzmann Machine 109

5.1	Boltzmann Machine	109
5.1.1	Relation to Hopfield Networks	110
5.2	RBM Architecture	110
5.2.1	Energy-Based Model	110
5.2.2	Gibbs Distribution	111
5.2.3	Gibbs Sampler	111
5.2.4	Contrastive Divergence	112
5.3	Example	113
5.4	Types of RBM	114
	Summary	116
	Review Questions	116

Assignment Problems	116
References	116

6. Open-Source Frameworks for Deep Learning 117

6.1 Python – An Introduction	117
6.1.1 Characteristics of Python	117
6.1.2 Features of Python	118
6.2 Environmental Setup	118
6.3 Deep Learning with Python	119
6.4 Scientific Python (SciPy)	119
6.4.1 Numerical Python (NumPy)	120
6.4.2 Matplotlib	120
6.4.3 Pandas	120
6.5 Frameworks	120
6.5.1 TensorFlow	121
6.5.2 Keras	127
6.5.3 PyTorch	133
6.6 Hardware Support for Deep Learning	136
6.6.1 Central Processing Unit	137
6.6.2 Graphics Processing Unit	137
6.6.3 VPU (Vision Processing Unit)	138
6.6.4 Neural Compute Stick (NCS)	139
6.6.5 Tensor Processing Unit	140
Summary	140
Review Questions	141
Assignment Problems	141
References	141

7. Applications of Deep Learning 143

7.1 Introduction	143
7.2 Image Classification Using CNN	143
7.2.1 Problem Description	143
7.2.2 Language and Framework	144
7.2.3 Dataset	144

7.2.4	Code Snippets	144
7.2.5	Complete Code	148
7.2.6	Result	150
7.3	Visual Speech Recognition Using 3D-CNN	151
7.3.1	Problem Description	151
7.3.2	Language and Framework	151
7.3.3	Dataset	151
7.3.4	Code Snippets	152
7.3.5	Complete Code	154
7.3.6	Result	160
7.4	Stock Market Prediction Using Recurrent Neural Network	161
7.4.1	Problem Description	161
7.4.2	Language and Framework	162
7.4.3	Dataset	162
7.4.4	Code Snippets	163
7.4.5	Complete Code	165
7.4.6	Result	169
7.5	Next-Word Prediction Using RNN-LSTM	169
7.5.1	Problem Description	169
7.5.2	Language and Framework	169
7.5.3	Dataset	170
7.5.4	Code Snippets	170
7.5.5	Complete Code	174
7.5.6	Result	177
7.6	Tamil Handwritten Character Optical Recognition Using CRNN	178
7.6.1	Problem Description	178
7.6.2	Language and Framework	178
7.6.3	Dataset	178
7.6.4	Code Snippets	178
7.6.5	Complete Code	180
7.6.6	Result	184
7.7	Future Scope	184
	Summary	184
	Review Questions	184
	Assignment Problems	185
	References	185

Fundamentals of Neural Networks

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand the basic machine learning tasks namely, classification and regression.
- Comprehend the structure of an artificial neural network.
- Identify the basic building blocks of an artificial neural network.

1.1 | INTRODUCTION

Deep learning is an area that has garnered a lot of attention in recent past. It is a subdivision of machine learning. Machine learning is a type of algorithm in which the user gives the input and expected output, and expects the machine to learn (find) the algorithm (model) to produce the desired (expected) output. This is in stark contrast to programming wherein we give the input and the algorithm produces the output. Machine learning tasks take a set of data that is termed as dataset. The machine learning algorithm finds the patterns and inferences that can be made on the dataset.

1.2 | TYPES OF MACHINE LEARNING

Machine learning can be classified into three major learning areas:

1. **Supervised Learning:** This happens when a model learns from data that has samples of data with both the input and the expected output. The expected output may be a string. For example, in the iris dataset, when the sepal length, sepal width, petal length and petal width are given as input, the model is expected to learn if the sample is Iris setosa, Iris versicolour, or Iris virginica from the training data. The model is learnt using the input and output attributes. This type of problem in which the expected output is a category is termed as classification. On the other hand, if the expected output is continuous data, the problem is referred to as a regression problem. For example, given the previous year's sales, predicting the sales for the next year is a regression problem as sales is continuous data.

2. **Unsupervised Learning:** This happens when the model is expected to discover the patterns on its own. There is no guidance as to what it is expected to learn. For example, consider a dataset with different types of cars. An unsupervised learning algorithm can group the cars based on either the color, size, or cost. Unsupervised learning algorithms simply group or cluster similar items.
3. **Reinforcement Learning:** This is like unsupervised learning as there is no available class label. But based on the solution provided by the model, positive or negative feedback is given. This feedback is used by the model to make a better decision.

The book covers the basics of neural networks, the fundamentals required for a good understanding of deep learning and some common supervised and unsupervised deep learning architectures. This chapter covers the two most common types of problems solved using machine learning (classification and regression), types of neural networks, activation functions, optimization techniques and weight initialization methods.

1.2.1 | Classification Problem

Classification is one of the commonly performed machine learning tasks. Given a set of features as input, classification attempts to assign a predefined label (class) to the sample. To understand better, let us consider a simple structured dataset of ' n ' rows and ' m ' columns. The ' n ' rows represent the data tuples or samples of input data and the ' m ' columns represent the features of the data. If X is a data row, then $\{x_1, x_2, x_3, \dots, x_m\}$ represents the values for the various features of the data.

1.2.1.1 Iris Dataset

The iris flower dataset is a famous dataset showing the morphological variations in three species of the iris flower. This is a simple dataset that has four attributes that help classify the species of the iris flower. There are three species of iris, namely, Iris setosa, Iris virginica and Iris versicolor. The dataset has 50 samples for each species of iris with a total of 150 data tuples. The attributes are:

1. Sepal length in cm
2. Sepal width in cm
3. Petal length in cm
4. Petal width in cm
5. Class: Iris setosa, Iris versicolour, Iris virginica

The following snapshot shows the dataset of iris:

```
Sample 1: 5.1, 3.5, 1.4, 0.2, Iris-setosa
Sample 2: 4.9, 3.0, 1.4, 0.2, Iris-setosa
Sample 3: 4.7, 3.2, 1.3, 0.2, Iris-setosa
Sample 4: 7.0, 3.2, 4.7, 1.4, Iris-versicolor
Sample 5: 6.4, 3.2, 4.5, 1.5, Iris-versicolor
Sample 6: 6.9, 3.1, 4.9, 1.5, Iris-versicolor
Sample 7: 6.3, 3.3, 6.0, 2.5, Iris-virginica
Sample 8: 5.8, 2.7, 5.1, 1.9, Iris-virginica
Sample 9: 7.1, 3.0, 5.9, 2.1, Iris-virginica
```

If X is Sample 1, then $\{x_1, x_2, x_3, x_4, x_5\} = \{5.1, 3.5, 1.4, 0.2, \text{Iris-setosa}\}$

In the case of the iris dataset, given the sepal, petal length and width in cm of an iris flower that we have newly spotted, the task of classification finds whether it belongs to the class Iris setosa, Iris virginica or Iris versicolor. To perform this classification, the classification algorithms make use of the information already present in the 150 samples in the iris dataset. With the help of the 150 samples, the classification task comes up with an equation or a set of rules or an algorithm to identify the class of the iris flower. This is termed as the classification model. The 150 samples used to create the classification model collectively form the training dataset.

1.2.1.2 Generalization

The classification model is based on 150 samples. How well can it perform on a new, unseen sample is termed as a generalization. The classification model created is good when it can generalize well to new data. Generalization is important because the training data we have used is only a sample which may or may not completely represent the population. The iris dataset is a tried and tested benchmarked dataset mostly used for explanation and understanding purpose. But, when we collect datasets for our application in real time, it will be incomplete and noisy and may not represent the entire population well. The next section deals with regression problems. The simplest type of regression, namely, linear regression is used for explanation.

1.2.2 | The Regression Problem

Given a set of data points, regression aims to find the equation that best fits all the data points. For simplicity and for easy understanding let us take a simple type of regression termed linear regression. Linear regression assumes that when we plot the data points, the points exhibit a linear relationship to one another. For example, consider the “House price” dataset given in Table 1.1. The table gives the purchasing cost of a house based on the built-up area.

TABLE 1.1 House price dataset

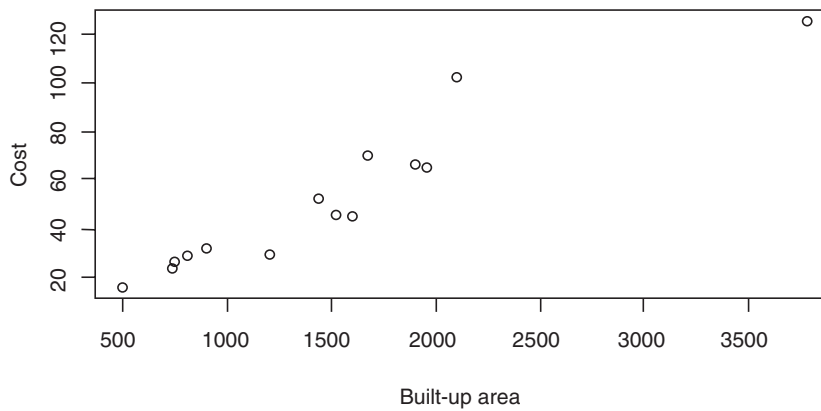
Built-up Area (sq. ft)	Price (in lakhs)
498	16.1
737	23.85
746	27
807	29.22
900	32
1200	30
1440	52.56
1526	45.5
1600	45

(Continued)

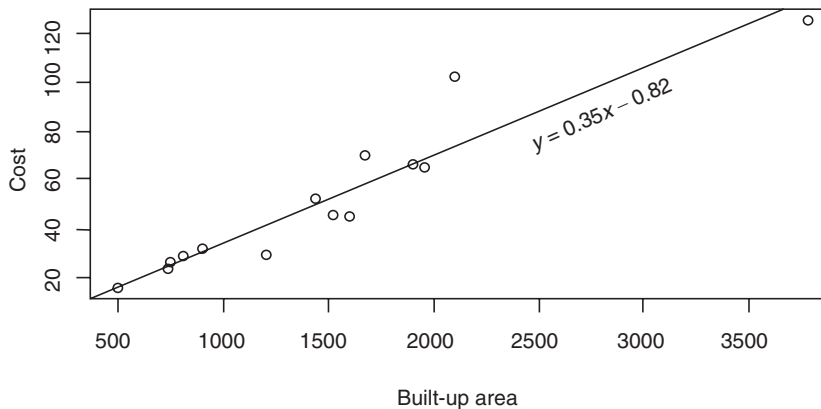
TABLE 1.1 (Continued)

Built-up Area (sq. ft)	Price (in lakhs)
1673	70
1900	67
1950	65
2100	102
3772	125

The data points can be plotted using a scatter plot as shown in Fig. 1.1.

**FIGURE 1.1** Scatterplot for house price dataset.

The graph shows that as the built-up area increases, the price also increases. That is, a linear relationship exists between the data. A simple straight-line equation $y = wx + b$ can be used to represent the data points. The data points and the line that best fits the data points are given in Fig. 1.2.

**FIGURE 1.2** Regression line.

This plotted line, called the regression line, best fits the data. We get the regression model when we find the values of w and b for the 'House Price' dataset. Here $w = 0.35$ and $b = -0.82$.

1.2.3 | Overfitting and Underfitting

If the model fits the training dataset perfectly as in Fig. 1.3, it might result in a condition termed 'overfitting' and it might not be able to classify a new unknown sample. This condition is shown in Fig. 1.3.

If the model does not fit (is unable to model) the training set correctly and is not able to generalize to a new sample, it results in a condition termed 'underfitting'. Ideally, we would like to select a model that neither overfits nor underfits. Figure 1.4 illustrates underfitting. The optimal fit is given in Fig. 1.5.

The symbols $\oplus$ and $\otimes$ refer to two different classes. The condition of underfitting, overfitting and optimal fit can be represented as in Fig. 1.6.

overfit--memorizes training set data

underfit--performs badly in training set

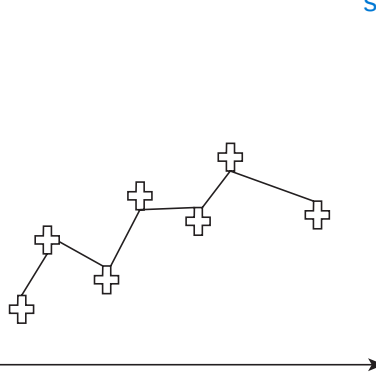


FIGURE 1.3 Overfitting.

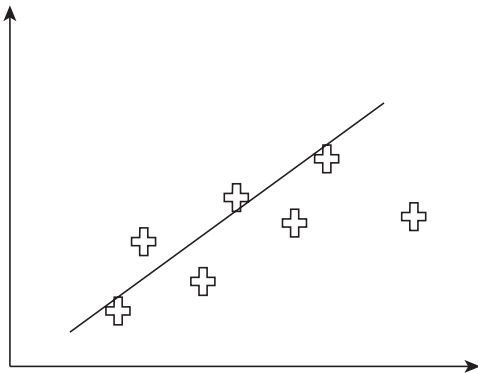


FIGURE 1.4 Underfitting.

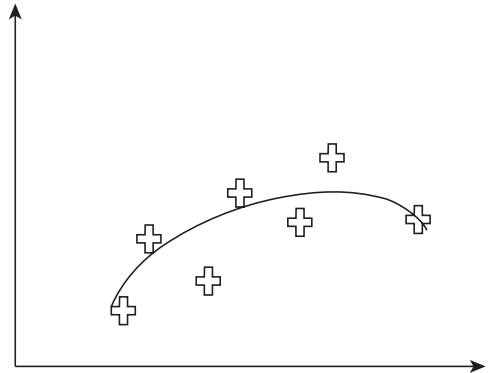


FIGURE 1.5 Optimal fit.

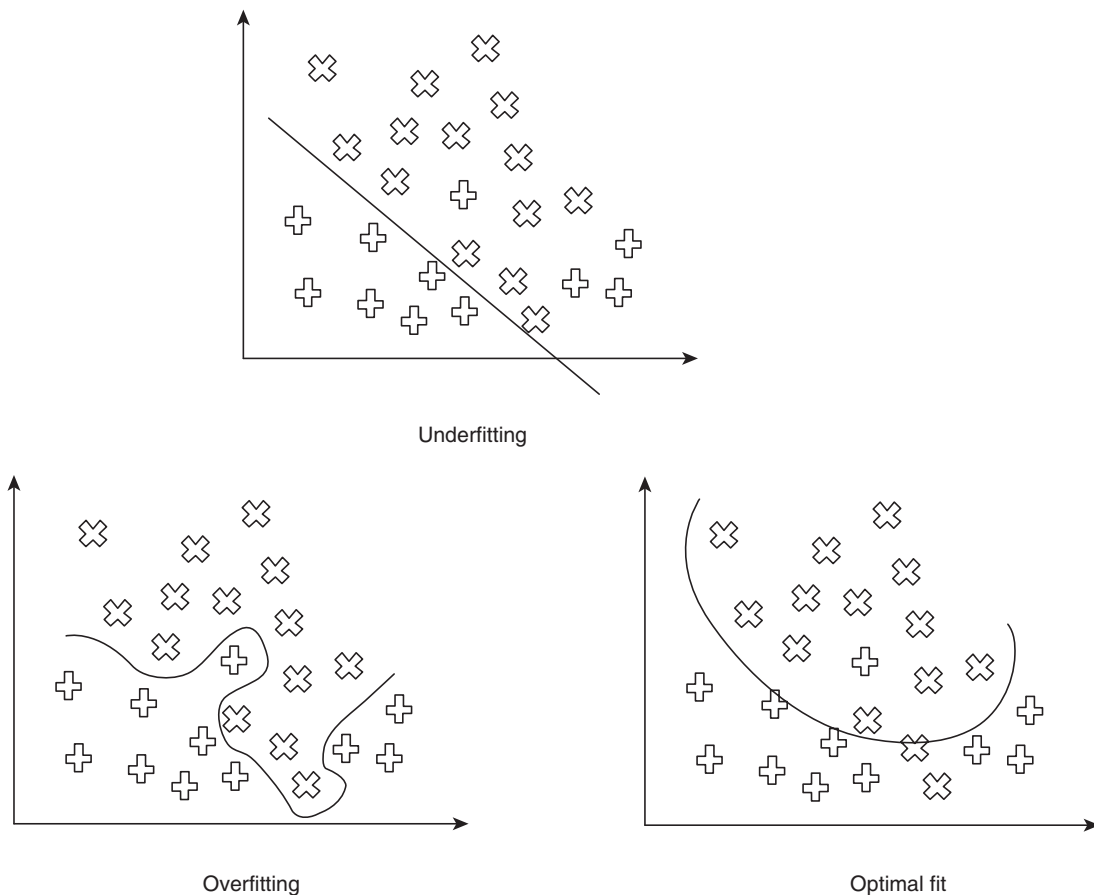


FIGURE 1.6 Binary classification problem.

1.2.4 | Bias and Variance

Bias and variance are **two types of errors** that can occur in classification and regression models. High bias occurs when the model is unable to learn the underlying pattern in the data. This is the result of underfitting the data. In certain cases, the model learns the training data very well and gives almost zero error when we classify on the data present in the training data. But classification is about classifying previously unseen data. These models that give zero error on training data may not be able to classify unseen data well if the unseen data is not similar to data present in the training data. This results in high variance. That is, the model changes drastically with the training set provided. This is the result of overfitting the data. **A good model tries to maintain low bias and low variance.**

Now that we have seen the types of problems we are trying to solve and the issues in creating a model, let us understand deep learning.

high bias, low variance ---> underfit

low bias, high variance --> overfit

1.3 | OVERVIEW OF ARTIFICIAL NEURAL NETWORKS

Deep learning is a technique that uses a special type of machine learning method termed as *artificial neural network* (ANN). Artificial neural network can be used to solve both regression and classification problems. Since the basis for deep learning are the artificial neural networks, the remainder of this chapter introduces the basics in artificial neural networks. We have concluded this section with the definition of deep learning and the reason for its current upsurge. Before learning about artificial neural networks, let us see the structure of a biological neuron.

1.3.1 | Biological Neuron

The biological neural network refers to a group of biological nerve cells that are connected to one another. A typical biological neural network is the brain. The brain is composed of a number of neurons that are interlinked to form a huge network. This network is used to transmit information between any two points. The brain uses different routes to transmit different types of information between the two points. A typical biological neuron looks as shown in Fig. 1.7.

The outline of the structure and working of the biological neuron required for the understanding of artificial neural networks is as follows. At the simplest level, a biological neuron has a cell body with a nucleus inside. This cell body is a round polygon which processes signals. When message is passed from one neuron to another, electrical signals are produced. These electrical signals produced in one neuron are passed to the next neuron using a long branch-like structure called axon. To receive signals from other neurons, each neuron has many dendrites attached to the cell body. Between the axon terminal of one neuron and the dendrite of the next neuron, is a small gap called synapses. When a nerve signal reaches the end of the neuron, it triggers the release of neurotransmitters which carry the signal across the synapse to the next neuron. This allows signals to pass from one neuron to the next.

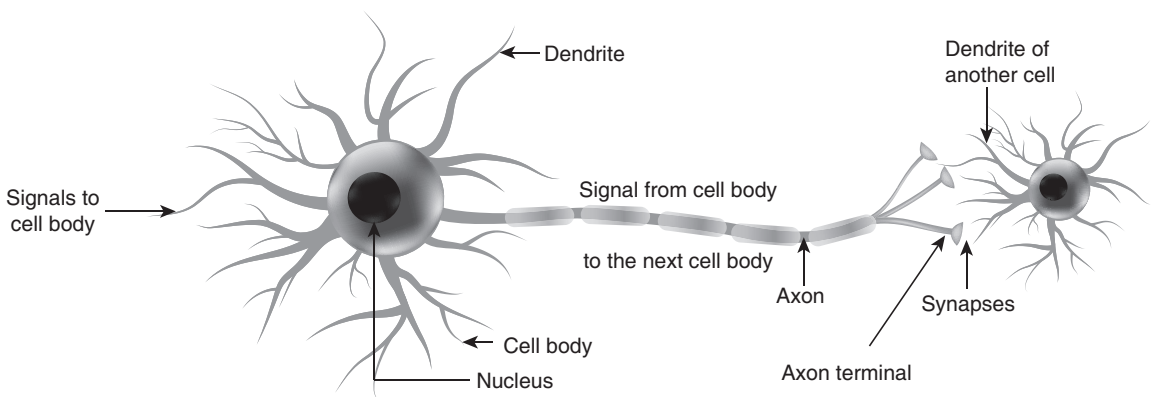


FIGURE 1.7 Schematic diagram of biological neuron.

1.3.2 | Types of Artificial Neural Networks

An artificial neural network is a mathematical model of biological neurons. Like biological neurons, it has many connecting neurons (nodes). A look at the various improvements made in artificial neural networks will help us get a better understanding.

1.3.2.1 The McCulloch–Pitts Network

The simplest network is the McCulloch–Pitts model of the network. It was introduced by Warren McCulloch and Walter Pitts in 1943. The input $\{x_1, x_2, x_3, \dots, x_n\}$ is given to a neuron and it produces one output y . The output is binary. The neuron should either fire (1) or not fire (0) based on the input. The processing unit is found in the output layer. To combine the input and give to the processing unit, the input is summed. Multiple inputs given are combined using summation as follows:

$$\text{Net input, } I = \sum_{i=1}^n x_i$$

The processing unit now takes a decision based on the summation value. The net input (I) is given as input to a threshold function like the binary step function. The decision is based on the outcome of the step function. Whether the neuron fires or not depends on this function; this function is called the activation function. The activation function takes the net input as input and produces the required output.

$$\text{Activation Function} = f(I) = \begin{cases} 1, & \text{if } I \geq T \\ 0, & \text{if } I < T \end{cases} - \text{binary step function}$$

This is called a binary step function because the output is either a 0 or 1 based on the threshold (T) that is set. This is shown in Fig. 1.8. The threshold is set at 0.5. For all net input values < 0.5 , the output is 0. The output is 1 for all values ≥ 0.5 . The network model is as given in Fig. 1.9.

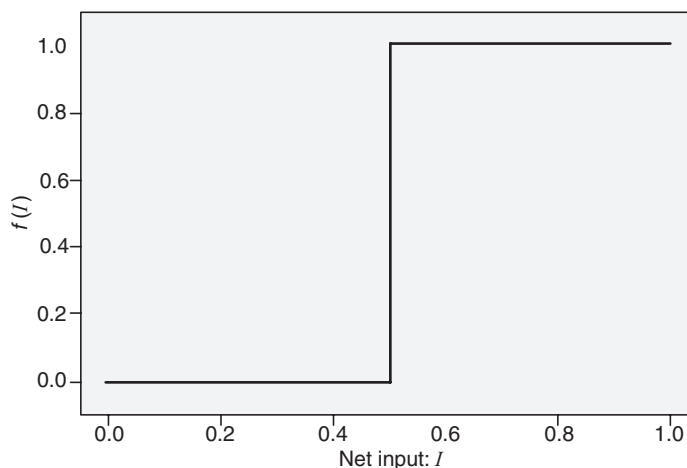


FIGURE 1.8 Binary step function.

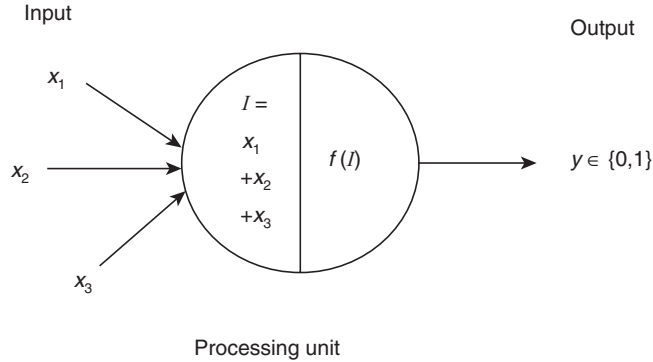


FIGURE 1.9 McCulloch–Pitts unit.

The simulation model shows that the output of Σ is given as input to the activation function. The output of the activation model is the output of the network. The McCulloch–Pitts model of a neuron is a simple model with the capability of computing. The disadvantage with the model is that it only generates a binary output.

Modelling a Simple AND Function

An AND function is one of the simplest logical functions that follows the following truth table for two features: x_1 and x_2 .

x_1	x_2	$x_1 \text{ AND } x_2$
0	0	0
0	1	0
1	0	0
1	1	1

Let us design a neural network to model this function. The network should trigger a ‘1’ if both the inputs are ‘1’, else it should trigger a ‘0’. There are two inputs x_1 and x_2 and one output y . A simple network would look as in Fig. 1.10.

Now let us design the network. The net input given by $\sum_{i=1}^2 x_i$ is

Net input (1, 1): $1 + 1 = 2$

Net input (1, 0): $1 + 0 = 1$

Net input (0, 1): $0 + 1 = 1$

Net input (0, 0): $0 + 0 = 0$

When the value of the net input exceeds ‘1’, the network should output ‘1’. This is given in Fig. 1.11.

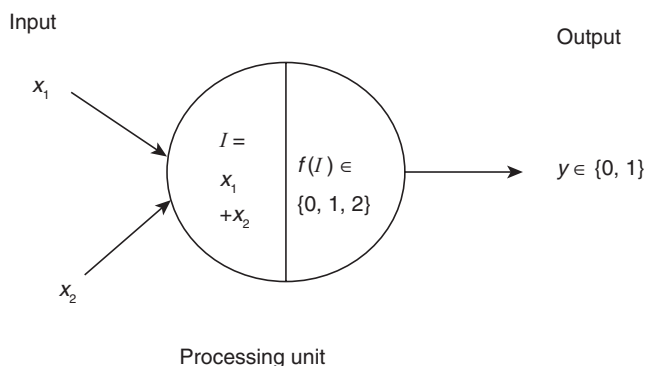


FIGURE 1.10 McCulloch–Pitts network for the AND function.

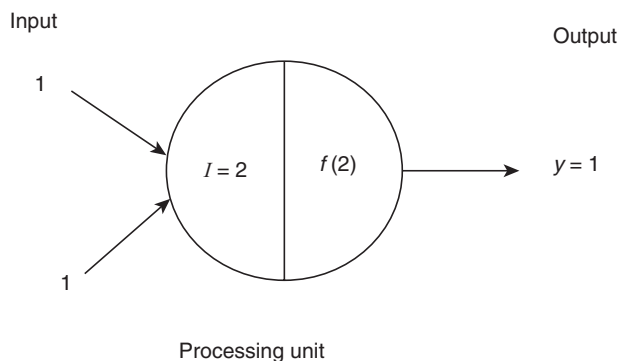


FIGURE 1.11 McCulloch–Pitts network for the AND function when both inputs are 1.

In the remaining three cases, the value of the net input is either 0 or 1 and the network should output ‘0’. This helps us to fix the threshold as $T = 2$ when a simple step function is used as an activation function.

Output: 1 if $T \geq 2$

Output: 0 if $T < 2$

Remember we designed the network. We manually set threshold by checking the various cases. This was possible because of the simplicity of the AND function. What if the number of input variables increases? In that case, how shall we design the network? Is it possible to fix the threshold as we did? These questions, if answered, will help us get a better understanding of the fundamentals of neural networks.

1.3.2.2 Perceptron

The improvement to the McCulloch–Pitts network is the perceptron. It is a linear classifier that performs binary classification. This has a single layer, that is, one input layer and one output layer. The single layer referred to is the output layer. The input to a perceptron is vectored with real values. The output is binary 0 or 1. The major improvement over the McCulloch–Pitts network is the

possibility to introduce weights in the networks and learn the weights. The structure of a perceptron is given in Fig. 1.12.

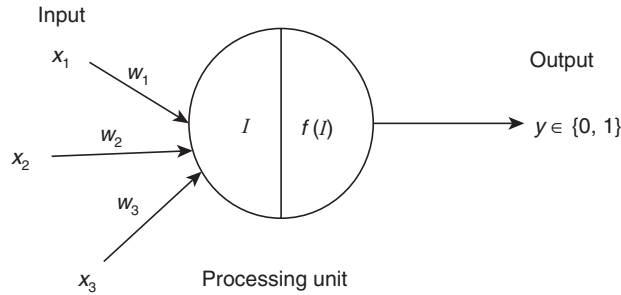


FIGURE 1.12 Perceptron.

Weights $\{w_1, w_2, w_3, \dots, w_n\}$ are associated with each input. The neuron should either fire (1) or not fire (0) based on the input and the weights. The net input is given by

$$\text{Net input, } I = \sum_{i=1}^m w_i * x_i$$

Note the inclusion of weights that is not present in the McCulloch–Pitts network. Here m is the size of the vector and w_i are real-valued weights. This can be written as a

$$\text{Dot product of two vectors as } w \cdot x \quad \text{or} \quad \text{Product of two matrices as } w^T \cdot x$$

As given in McCulloch–Pitts network

$$\text{If } \sum_{i=1}^m x_i * w_i \geq T, \quad \text{it belongs to class 1}$$

and

$$\text{If } \sum_{i=1}^m x_i * w_i < T, \quad \text{it belongs to class 0}$$

can be written as

$$\text{If } \sum_{i=1}^m x_i * w_i - T \geq 0, \quad \text{it belongs to class 1}$$

and

$$\text{If } \sum_{i=1}^m x_i * w_i - T < 0, \quad \text{it belongs to class 0}$$

or $w \cdot x - T = 0$ is the line separating the two classes and can be written as $w \cdot x + b = 0$ where $b = -T$ and is called the bias.

This is diagrammatically shown in Fig. 1.13. In this figure, x is the input vector $(x_1, x_2, x_3, \dots, x_n)$, w is the weight vector $(w_1, w_2, w_3, \dots, w_n)$ and b is the bias $(b_1, b_2, b_3, \dots, b_m)$, where n is the number of input nodes and m is the number of output nodes. **Varying the weights (w) and bias (b) results in different lines or classifier models. Perceptron finds the values for w and b and outputs the line** (classifier model) that separates the two classes. The major difference between the McCulloch–Pitts network and the perceptron is that perceptron learns the weights (w) and the threshold (b) unlike the McCulloch–Pitts network that decides the threshold. In the output neuron, the inputs are linearly combined as $\sum_{i=1}^m x_i * w_i$ and passed through a step function to produce the output '0' or '1'. The equation $wx + b$ is the equation of the straight line

$$y = wx + b$$

To understand how the values of w and b affect the final output in ANN, a good understanding of the equation of a straight line is required.

Equation of a Straight Line

Let us consider a binary classification problem as given in Fig. 1.14.

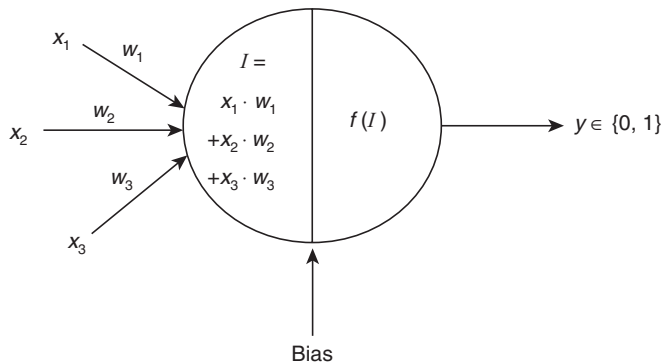


FIGURE 1.13 Working of perceptron.

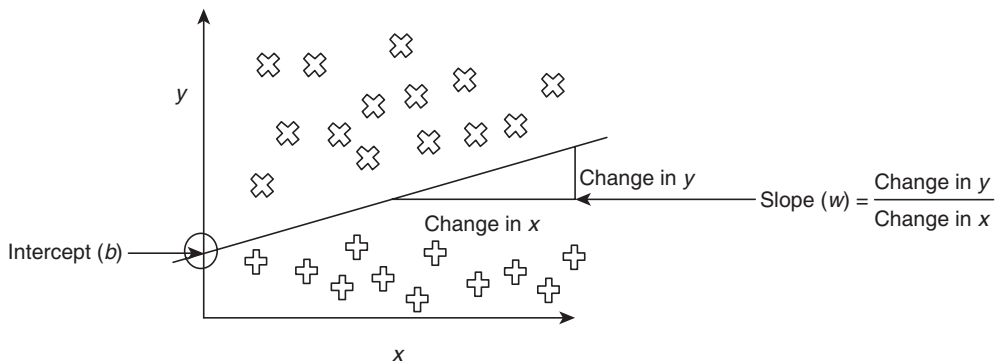


FIGURE 1.14 Binary classification and separating plane.

The line that separates the two classes is the separating line. In a two-dimensional space, the separator is a line. In a three-dimensional space, the separator is a plane. For dimensions above three, the separator can be called using a generalized term, namely, the hyperplane. Note that the separating line is always one dimension less than the space in which the data points exist. Note that the orientation and position of the line are determined by the slope and y-intercept. Figure 1.15 shows the effect of changing the weights (w) and bias (b).

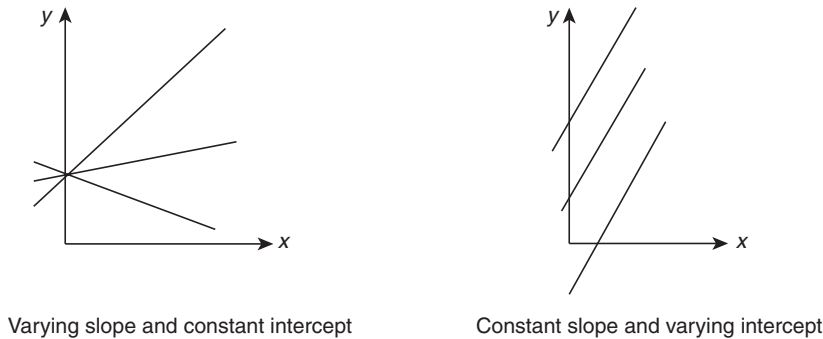


FIGURE 1.15 Effect of changing slope and intercept.

Remember that if $b = 0$, the line would always pass through the origin. Also, if $b = 1$, the line would always pass through $y = 1$. Varying the y-intercept varies the position of the line and varying the slope varies the orientation of the line. That is, different values of the weight and bias produce different lines. We say we train an ANN when we try to find the line that best separates the two classes by varying the values of the weight and bias. The ANN model is the value of the parameters (w, b) after training.

Learning Algorithm

In the context of artificial neural networks, a learning algorithm is an algorithm that helps to find the values of the weights and bias. The simplest way to train an ANN is to perform the following steps:

1. Initialize the weights and bias randomly.
2. Combine the input, weight and bias as

$$I = \sum_{i=1}^m x_i * w_i + b$$

3. Feed I to the neuron in the output layer.
4. Pass I to the activation function $f(I)$ to introduce non-linearity.
5. Get the output of the output neuron as y .
6. Check the difference between the actual output O and the predicted output y .

7. If $(y - O) \neq 0$ then
 - (a) Adjust (increase or decrease) the weights and bias accordingly.
 - (b) Go to step (2).
8. Else
 - (a) Stop the process as the weights and bias have been correctly identified.

This process is termed as training the ANN. At the end of the process, the model would have learnt the weights and bias. The simplest learning algorithm finds the values of weight and bias by trial and error. But recent advancements solve the problem of finding the weights and bias using optimization techniques covered in the later part of this chapter. The time taken to train an ANN largely depends on the optimization algorithm used.

1.3.2.3 The Sigmoid Neuron

In the previous section, we saw the simplest activation function, namely, the step function. Step functions are useful when the problem is a binary classification problem requiring an output of 1 or 0. If the problem requires an output of real numbers between (0, 1) and (-1, 1), the best option is to use a sigmoid activation function. The sigmoid function produces an S-shaped curve commonly referred to as the sigmoid curve. The two commonly used sigmoid functions in ANN are the logistic function and the hyperbolic tangent.

Logistic Function

The logistic function is given by

$$f(x) = \frac{1}{1 + e^{-x}}$$

It squashes the input to a value in the range (0, 1). Functions that either only increase or only decrease as the input increases are monotonic functions. Logistic functions exhibit this property and are monotonic. Being monotonic is desirable in activation functions.

First Derivative of the Logistic Function

The logistic function can be rewritten as

$$f(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

Let us derive the first derivative of the logistic function to give a better understanding of the later topics. Note that the first derivative is also the gradient of the logistic function. The logistic function can be derived as follows:

$$\begin{aligned}
\frac{df(x)}{dx} &= \frac{e^x(1-e^x) - e^x \cdot e^x}{(1+e^x)^2} = \frac{e^x}{(1+e^x)^2} = \frac{e^x + e^x \cdot e^x - e^x \cdot e^x}{(1+e^x)^2} \\
&= \left(\frac{e^x}{(1+e^x)} \right) \left(\frac{1+e^x - e^x}{(1+e^x)} \right) = \left(\frac{e^x}{(1+e^x)} \right) \left(\frac{1+e^x}{(1+e^x)} - \frac{e^x}{(1+e^x)} \right) \\
&= f(x)[1 - f(x)]
\end{aligned}$$

The proof also gives the first derivative with respect to x . The sigmoid function is differentiable at every point in the domain and is hence differentiable. The function is continuous since arbitrary small changes in the input results in arbitrary small changes in the output. A major drawback of the sigmoid function is that it is too flat when the value of the function is near 0 or 1.

Hyperbolic Tangent

The hyperbolic tangent is given by

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

It squashes the input in the range $(-1, 1)$. It is a scaled version of the logistic function and has all the properties of the logistic function. In most cases the hyperbolic tangent is preferred over the logistic function because it has stronger gradients. Also, based on the range of the output required we can choose between the logistic function and hyperbolic tangent.

1.3.2.4 Linear Separable Problem

Perceptrons solve only linearly separable problems. Consider a binary classification problem, that is, there are only two possible class labels. For example, the AND and OR functions can be viewed as a binary classification problem with class labels 0 and 1. This can be viewed graphically as in Fig. 1.16. Every point to the right of the line belongs to the '1' class. A single line was enough to classify the 2 classes. These are called linearly separable problems. Classifiers that solve only linear separable problems are linear classifiers.

Consider the XOR function given by the following truth table:

x	y	$x \text{ XOR } y$
1	1	0
1	0	1
0	1	1
0	0	0

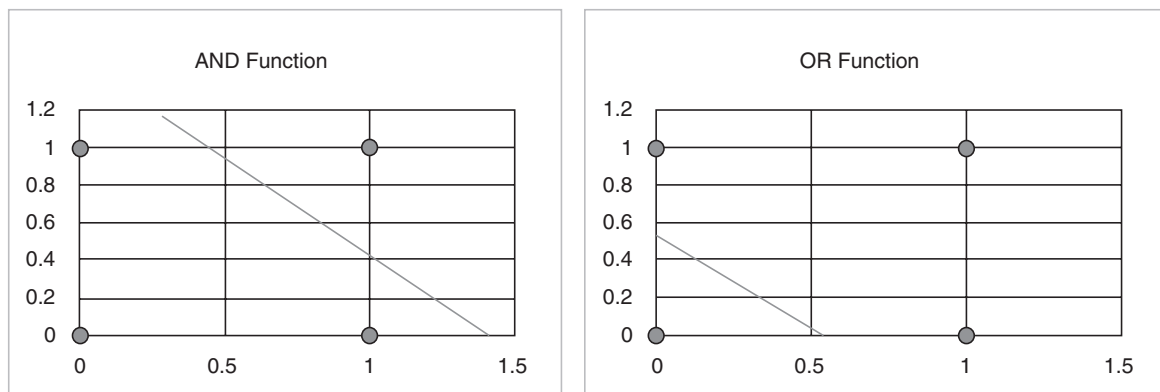


FIGURE 1.16 Linear separable problems.

As seen in Fig. 1.17 it is not possible to differentiate the '0' and '1' class using a single line. A minimum of two lines is essential to differentiate the classes as shown in Fig. 1.17. These are termed as non-linearly separable problems and the classifiers that solve these problems are non-linear classifiers.

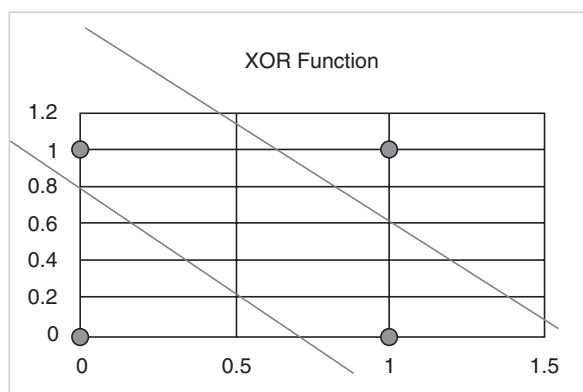


FIGURE 1.17 XOR function.

The data points shown in Fig. 1.17 are non-linearly separable while the data points in Fig. 1.16 are linearly separable.

Perceptron to Solve Linearly Separable Problems

In this section, we show why it is possible to find a separating line for the AND and OR functions using perceptrons, while it is impossible to solve the XOR function using perceptrons. Consider the AND function.

x_1	x_2	$x_1 \text{ AND } x_2$	Equation of Separator	Inequalities
0	0	0	$\sum_{i=1}^2 x_i * w_i + w_0 < 0$	$0 \cdot w_1 + 0 \cdot w_2 + w_0 < 0$ $w_0 < 0$
0	1	0	$\sum_{i=1}^2 x_i * w_i + w_0 < 0$	$0 \cdot w_1 + 1 \cdot w_2 + w_0 < 0$ $w_2 < -w_0$
1	0	0	$\sum_{i=1}^2 x_i * w_i + w_0 < 0$	$1 \cdot w_1 + 0 \cdot w_2 + w_0 < 0$ $w_1 < -w_0$
1	1	1	$\sum_{i=1}^2 x_i * w_i + w_0 \geq 0$	$1 \cdot w_1 + 1 \cdot w_2 + w_0 \geq 0$ $w_1 + w_2 \geq -w_0$

It is possible to find a solution for this problem. For example, $w_0 = -0.5$, $w_1 = 1.5$, $w_2 = 1$ is one of the many possible solutions to the inequalities. This shows that perceptrons can be used to find the separating line for the AND problem. Solution can be given to the OR problem in a similar manner.

x_1	x_2	$x_1 \text{ OR } x_2$	Equation of Separator	Inequalities
0	0	0	$\sum_{i=1}^2 x_i * w_i + w_0 < 0$	$0 \cdot w_1 + 0 \cdot w_2 + w_0 < 0$ $w_0 < 0$
0	1	1	$\sum_{i=1}^2 x_i * w_i + w_0 \geq 0$	$0 \cdot w_1 + 1 \cdot w_2 + w_0 \geq 0$ $w_2 \geq -w_0$
1	0	1	$\sum_{i=1}^2 x_i * w_i + w_0 \geq 0$	$1 \cdot w_1 + 0 \cdot w_2 + w_0 \geq 0$ $w_1 \geq -w_0$
1	1	1	$\sum_{i=1}^2 x_i * w_i + w_0 \geq 0$	$1 \cdot w_1 + 1 \cdot w_2 + w_0 \geq 0$ $w_1 + w_2 \geq -w_0$

One possible solution to this problem is again $w_0 = -0.5$, $w_1 = 1.5$, $w_2 = 1$. Perceptrons can be used to give the separating line for the OR problem as well. But now let us try to find a solution for the XOR problem.

x_1	x_2	$x_1 \text{ XOR } x_2$	Equation of Separator	Inequalities
0	0	0	$\sum_{i=1}^2 x_i * w_i + w_0 < 0$	$0 \cdot w_1 + 0 \cdot w_2 + w_0 < 0$ $w_0 < 0$
0	1	1	$\sum_{i=1}^2 x_i * w_i + w_0 \geq 0$	$0 \cdot w_1 + 1 \cdot w_2 + w_0 \geq 0$ $w_2 \geq -w_0$
1	0	1	$\sum_{i=1}^2 x_i * w_i + w_0 \geq 0$	$1 \cdot w_1 + 0 \cdot w_2 + w_0 \geq 0$ $w_1 \geq -w_0$
1	1	0	$\sum_{i=1}^2 x_i * w_i + w_0 < 0$	$1 \cdot w_1 + 1 \cdot w_2 + w_0 < 0$ $w_1 + w_2 < -w_0$

When we try to solve the inequalities, we get from the second and third inequalities

$$w_1 + w_2 \geq -w_0$$

This contradicts the fourth inequality $w_1 + w_2 < -w_0$.

A perceptron that solves both the above-mentioned inequalities is impossible to design. We require a different neural network architecture to solve non-linearly separable problems. Multilayer perceptron introduced next can be used to solve both linearly separable and non-linearly separable problems.

1.3.2.5 Multilayer Perceptron (MLP)

Consider a multilayered directed graph. Multilayered graphs have layers of nodes with the flow of data from the first layer to the second layer, from the second layer to the third layer and so on. This type of network with information flowing from one end to the other in one direction through the intermediate layers is called a *feedforward network*. The first and the last layers are the input and output layers, respectively, and the intermediate layers are the hidden layers. A network with multiple hidden layers is a multilayered perceptron. The nodes in the hidden layers and in the output layer are processing units with non-linear activation functions. The multiple hidden layers combined with non-linear activation function helps it to solve non-linearly separable problems. The difference between linear and non-linear activation functions is as follows:

Linear functions

1. Are represented using a straight line.
2. Are a polynomial of degree less than or equal to 1.
3. The slope is constant throughout the function.

Non-linear functions

1. Are not represented using a straight line, but rather using curves and other structures.
2. Are a polynomial of degree greater than 1.
3. The slope is always changing.

A multilayered perceptron is shown in Fig. 1.18. The network is a four-layered network with three hidden layers and an output layer. The hidden layers can have any number of nodes in each layer. But every node in one layer is connected to every node in the next layer. In Fig. 1.18, there are seven nodes in the first hidden layer, four nodes in the second hidden layer, five nodes in the third hidden layer and three nodes in the output layer. Every edge shown in the figure has a weight attached to it. These are the parameters of the MLP that are to be learnt. The more the number of hidden layers and hidden nodes, the more parameters are to be learnt. The number of weights and bias learnt between each layer is given in Table 1.2.

TABLE 1.2 Weights and bias

Associated Layer	Number of Weights	Bias
Input layer to hidden layer 1	$4 \times 7 = 28$	7
Hidden layer 1 to hidden layer 2	$7 \times 4 = 28$	4
Hidden layer 2 to hidden layer 3	$4 \times 5 = 20$	5
Hidden layer 3 to output layer	$5 \times 3 = 15$	3

This results in a total of $101 + 19 = 120$ parameters. The complexity of the neural network depends on the number of hidden nodes. The best way known so far to find the right number of hidden nodes is to fine tune the model trying various combinations.

The number of nodes on the input layer is determined by the number of features. For the iris dataset, the sepal and petal length and width are the four features of an iris flower. This results in four input nodes. The number of output nodes depends on the number of output. In the iris dataset, the possible outputs are Iris setosa, Iris versicolour and Iris virginica. So the number of output nodes for the iris dataset is three.

Multilayer perceptrons have now introduced a new problem. When we had a single layer perceptron, the weights and bias between the hidden and output layers were updated using the difference between the actual and the predicted outputs in the output layer. In multilayer perceptrons there is no basis to know the expected output in a hidden layer. The learning algorithm used for single-layer perceptrons needs to be modified and applied for multilayer perceptrons. The learning algorithm used in multilayer perceptrons is the backpropagation algorithm. To have a better understanding of the working of an ANN, we shall see the optimization techniques used.

1.3.3 | Optimization Techniques

The difference between the actual and the predicted output shows the amount of error in the model. As this error is minimized, the model becomes better. Calculating the difference between the actual output (O) and the predicted output (y) is a major step in the learning algorithm.

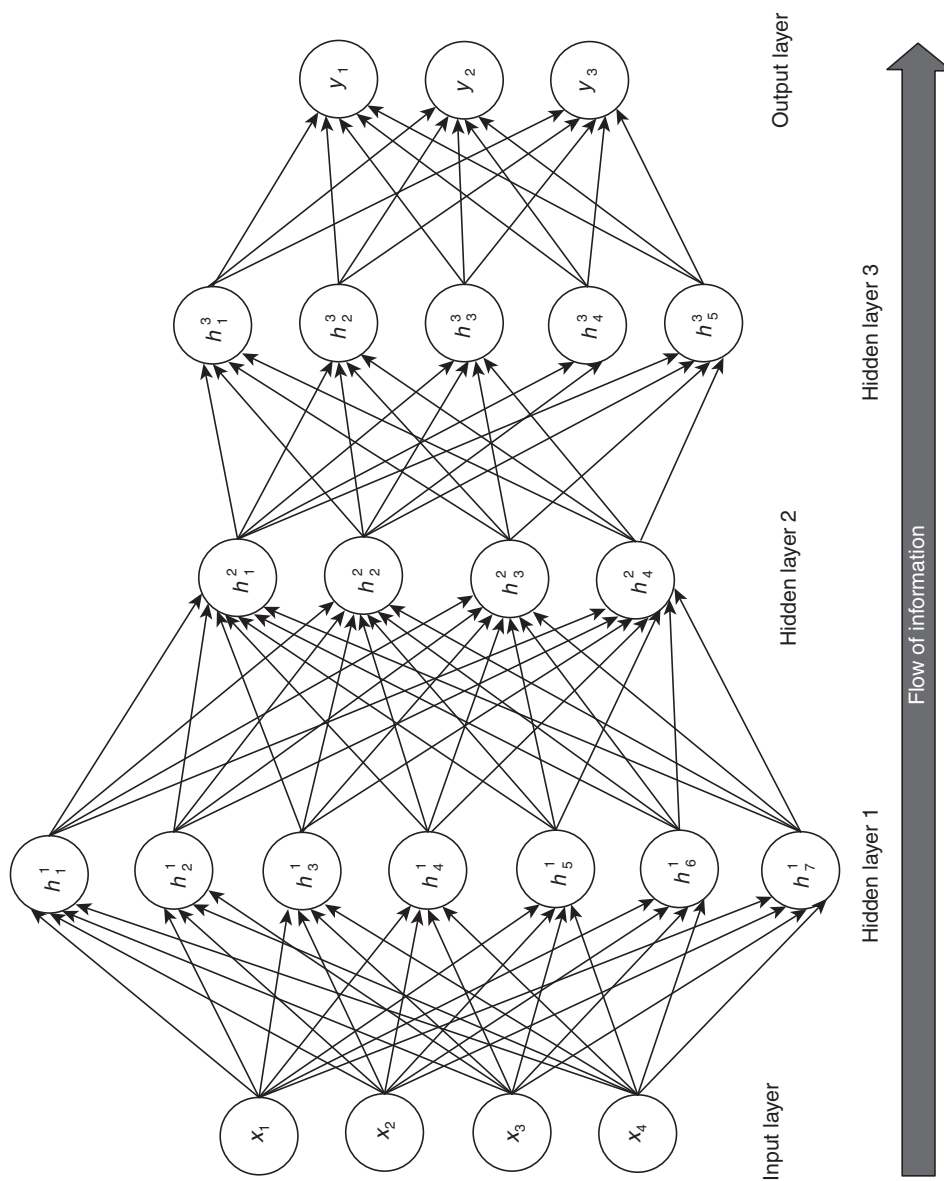


FIGURE 1.18 Multilayer perceptron.

1.3.3.1 Error Functions

The error function, also termed the loss function, is used to represent the difference between the actual and predicted outputs. Some of the common loss functions are the least absolute deviations, least square error and the cross-entropy loss function. Let us denote the actual output as O and the predicted output as y .

1. **Least Absolute Deviations (LAD):** LAD (Least Absolute Deviations) is also termed the L1-norm loss function. It is given by the formula

$$\text{LAD} = \sum_{i=1}^n |O_i - y_i|$$

where n is the number of samples.

2. **Least Square Error (LSE):** LSE is also termed the L2-norm loss function. It is given by the formula

$$\text{LSE} = \sum_{i=1}^n (O_i - y_i)^2$$

3. **Cross-Entropy Loss:** If the output of a classification model is a probability value between 0 and 1, the best loss function to use is the cross-entropy loss (CEL) function, given by

$$\text{CEL} = -\sum_{i=1}^c O_i \log(y_i)$$

where c is the number of classes.

The error function is a function of the weights and bias because the alteration of ' y ' and ' x ' is beyond our control and depends only on the dataset. This can be represented as $L(w, b)$. A model with an error of 0 learns the training data perfectly. The problem at hand is how to change the values of w and b such that $L(w, b)$ moves towards 0, that is, find those ' w ', ' b ' that minimize $L(w, b)$. For this purpose, we use optimization techniques. The following section deals elaborately with the optimization techniques commonly used in ANN and deep learning.

1.3.3.2 Gradient Descent

This is the simplest optimization algorithm used to minimize the error function; it was invented by Cauchy in 1847. For an explanation and easy understanding of gradient descent let us consider a function with a single parameter, denoted by $L(w)$. The problem at hand is to find that ' w ' which minimizes $L(w)$. The relationship between w and $L(w)$ is shown in the graph in Fig. 1.19.

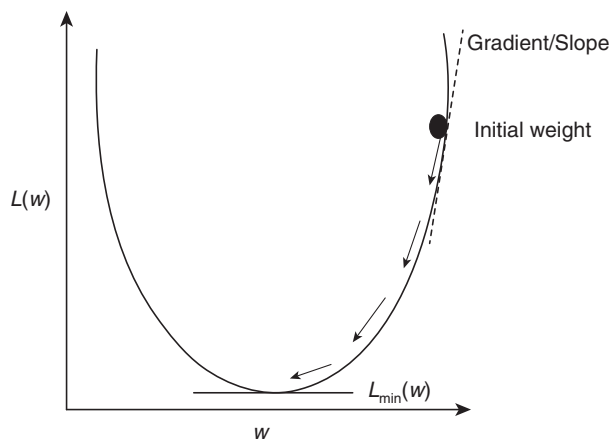


FIGURE 1.19 Minimization of error function.

We start with an initial random weight w . The solution to this problem can be viewed as sliding a ball down a hill. When pushed from the initial position, the ball comes to rest when it reaches the valley. This is analogous to the working of the gradient descent algorithm. The weight should be changed in such a way that the loss function decreases in the next iteration. The weight is continuously changed till the loss function reaches the global minima. In a two-dimensional plane, the term gradient refers to the ‘slope’ of the function. This is shown in the graph in Fig. 1.19.

$$\text{Slope of } J(w) = \frac{\text{Change of } J(w)}{\text{Change of } w} = \nabla w$$

The slope gives how much movement is required in the direction of ‘ w ’. In vector calculus, the gradient or slope is the direction in which a given function increases the maximum. The negative of gradient gives the direction in which the function decreases the most. The idea behind gradient descent is to keep moving in that direction opposite to the slope till the function decreases no more. When the function decreases no more, the minimum has been reached. When this is extended to three dimensions, the error function looks as shown in Fig. 1.20.

Since L has two variables, namely, w and b , the direction to move is given by $(\partial L/\partial w, \partial L/\partial b)$. These are partial derivatives (i.e., keep other variables constant). This gives the direction of movement along the w and b axes.

Epoch

The number of times the weight is updated as it moves towards the global minima need not be till the difference between the predicted and the actual output is zero. Most of the times, the user may decide on the number of updations to reduce the computational complexity. This number is termed as epoch.

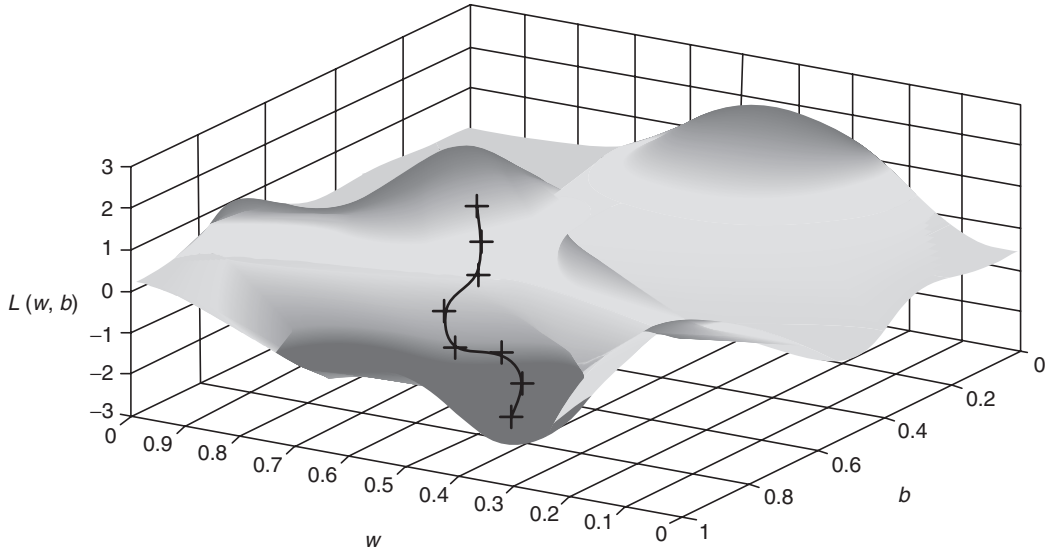


FIGURE 1.20 Error function in three dimensions.

Weight Updation

The weight is updated as per the following algorithm.

Algorithm Weight_Update_GD(j, old_weight)

```
{
    for (specified number of epochs) OR (until minima of error function is reached)
        for each weight j
            new_weight = old_weight -  $\nabla w$ 
            new_bias = old_bias -  $\nabla b$ 
}
```

where $\nabla w = \frac{\partial L(w, b)}{\partial w}$ is the gradient of the cost function with respect to w and $\nabla b = \frac{\partial L(w, b)}{\partial b}$ is the gradient of the cost function with respect to b .

Learning Rate

The speed with which the descent is made in a direction opposite to the slope is given by the learning rate. If we take small steps at every move, the time taken to reach the minima increases. But if the steps are too large, the algorithm may miss the minima and jump to the other side. The algorithm moves back to find the minima. If it again takes a large step, the algorithm would be bouncing back and forth without touching the minima. Usually a value like 0.01 is chosen for the learning rate and the model is tuned to find the appropriate learning rate. The learning rate is represented using η . The weight update algorithm is modified as follows.

Algorithm Weight_Update_GD(j, old_weight)

```
{
    for (specified number of epochs) OR (until minima of error function is reached)
        for each weight j
            new_weight = old_weight -  $\eta \nabla w$ 
            new_bias = old_bias -  $\eta \nabla b$ 
}
```

1.3.3.3 Stochastic Gradient Descent

When computing the gradient descent, we compute the gradient after passing the entire training set once as a batch. Then the direction is updated. That is, for every computation involving the entire training set, one step is taken. This slows the updation of weights and is a huge problem when the size of the training set is very large. This, in turn, increases the time taken to find the minima of the cost function. But consider the case when we update the weights for every training sample. This is termed as stochastic gradient descent. This was developed by Herbert Robbins and Sutton Monro in the year 1951 and was published in an article titled “A *Stochastic Approximation Method*”. Stochastic gradient descent is termed so because it does not take the samples in the order given, but rather shuffles and takes the training samples randomly one by one.

Weight Updation

The weight is updated as per the following algorithm.

Algorithm Weight_Update_SGD(I, j, old_weight)

```
{
    for (specified number of epochs) OR (until minima of error function is reached)
        for each training sample i
            for each weight j
                new_weight = old_weight -  $\eta \nabla w$ 
                new_bias = old_bias -  $\eta \nabla b$ 
}
```

1.3.3.4 Mini-Batch Gradient Descent

The gradient descent algorithm mentioned above can also be called the batch gradient descent because it views the entire training set as a single batch. Mini-batch gradient descent is in between batch and stochastic gradient descent. It takes small batches of training samples before updating the weights. It utilizes the best of gradient descent and stochastic gradient descent and is most commonly used in deep learning. The size of the mini-batch determines the efficiency of the methods. It should not be too large to behave like batch gradient descent nor too small to degenerate to stochastic gradient descent. The training samples are shuffled before splitting into batches. GPU and

CPU hardware perform better on data that are powers of two. To enable easy implementation on the underlying computer architecture, batch sizes are powers of two (32, 64, 128, 256, ...). Dominic Masters and Carlo Luschi suggested batch sizes of 32 in their paper titled “*Revisiting Small Batch Training for Deep Neural Networks*” in the year 2018.

Weight Update

The weight is updated as per the following algorithm.

Algorithm Weight_Update_BGD(I, j, old_weight)

```
{
    for (specified number of epochs) OR (until minima of error function is reached)
        for a batch of training samples  $b_i$ 
            for each weight  $j$ 
                 $new\_weight = old\_weight - \eta \nabla w$ 
                 $new\_bias = old\_bias - \eta \nabla b$ 
}
```

1.3.3.5 Backpropagation Algorithm

It is termed the backpropagation algorithm because though the information flows in the forward direction, the error is propagated in the backward direction. Though it was originally proposed in the 1970s, it gained popularity after a paper published by Rumelhart *et al.* in 1986. He was the first person to apply the concept of backpropagation to multilayer networks. Backpropagation algorithm is the heart of the multilayer perceptron and is explained using an example. Consider the neural network in Fig. 1.21.

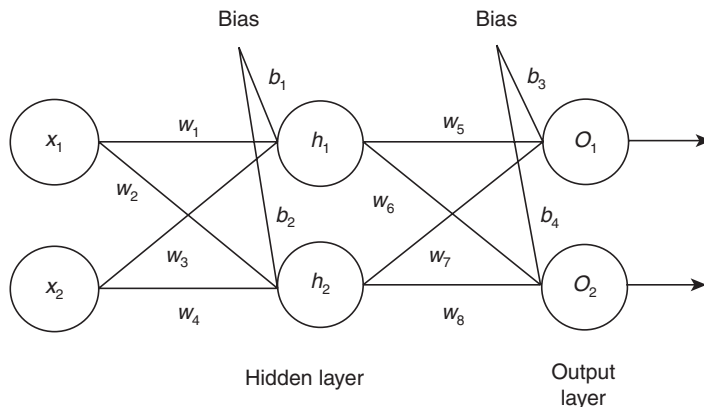


FIGURE 1.21 Two-layered artificial neural network.

This is a two-layered ANN with 2 nodes in the hidden layer and 2 nodes in the output layer. Let us use the logistic function as the activation function. The weights and bias are randomly initialized as given in the table.

w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	b_1	b_2	b_3	b_4
-0.1	0.2	0.6	-0.3	0.4	0.6	0.5	-0.2	-0.1	0.3	0.4	0.2

The input and output vectors are given in the following table.

x_1	x_2	O_1	O_2
0	1	0	1

Calculation for Hidden Layers

Net input to hidden node $h_1(i_1) = x_1w_1 + x_2w_3 + b_1 = 0 * (-0.1) + 1 * (0.6) + (-0.1) = 0.5$

Output of activation function at $h_1(a_1) = \frac{1}{1+e^{-i_1}} = \frac{1}{1+e^{-0.5}} = 0.62$

Net input to hidden node $h_2(i_2) = x_1w_2 + x_2w_4 + b_2 = 0 * (0.2) + 1 * (-0.3) + (0.3) = 0$

Output of activation function at $h_2(a_2) = \frac{1}{1+e^{-i_2}} = \frac{1}{1+e^{-0}} = 0.5$

Calculation for Output Layers

Net input to output node $O_1(i_3) = a_1w_5 + a_2w_7 + b_3 = (0.62) * (0.4) + (0.5) * (0.5) + (0.4) = 0.9$

Output of activation function at $O_1(y_1) = \frac{1}{1+e^{-i_3}} = \frac{1}{1+e^{-0.9}} = 0.71$

Net input to output node $O_2(i_4) = a_1w_6 + a_2w_8 + b_3 = (0.62) * (0.6) + (0.5) * (-0.2) + (0.2) = 0.41$

Output of activation function at $O_2(y_2) = \frac{1}{1+e^{-i_4}} = \frac{1}{1+e^{-0.41}} = 0.6$

Calculation of Error

To find the difference between the actual and predicted outputs, we use the squared error function $[L(w, b)]$. $L(w, b)$ shows that the error function is a function of the weights and bias.

$$L(w, b) = \sum_{i=1}^n \frac{1}{2} (O_i - y_i)^2$$

where c is the number of classes. For our problem,

$$L(w, b) = (0.5) * (0 - 0.71)^2 + (0.5) * (1 - 0.6)^2 = 0.17$$

Backpropagation of Error

The parameters of the error function are the weights and bias. The goal in parameter updation is that the weights and bias must be changed in such a way that the error function approaches 0. That is, the actual and target values should be as close as possible and preferably the same. So what should be the value of w_5 if $L(w, b)$ should tend to 0. This can be written as, how does $L(w, b)$ change when w_5 changes and is given by the partial derivative $\partial L(w, b) / \partial w_5$. Applying the chain rule, we get

$$\frac{\partial L(w, b)}{\partial w_5} = \frac{\partial L(w, b)}{\partial y_1} * \frac{\partial y_1}{\partial i_3} * \frac{\partial i_3}{\partial w_5} \quad (1.1)$$

$$\begin{aligned} \frac{\partial L(w, b)}{\partial y_1} &= \frac{\partial \left(\frac{1}{2}(O_1 - y_1)^2 - \frac{1}{2}(O_2 - y_2)^2 \right)}{\partial y_1} \\ &= 2 * \frac{1}{2}(O_1 - y_1)^{2-1}(-1) + 0 \\ &= -(O_1 - y_1) = -(0 - 0.71) = 0.71 \end{aligned} \quad (1.2)$$

As seen, the reason for using 1/2 in the error function is to keep the derivative simple.

$$y_1 = \frac{1}{1 + e^{-i_3}}$$

We have already derived under Section 1.3.2.2 on sigmoid functions that

$$\frac{df(x)}{dx} = f(x) [1 - f(x)]$$

Therefore,

$$\frac{\partial y_1}{\partial i_3} = y_1(1 - y_1) = 0.71 * (1 - 0.71) = 0.2 \quad (1.3)$$

Also, $i_3 = a_1 w_5 + a_2 w_7 + b_3$. So

$$\frac{\partial i_3}{\partial w_5} = 1 * a_1 * w_5^{(1-1)} + 0 + 0 = a_1 = 0.62 \quad (1.4)$$

Substituting it in Eq. (1.1), we get

$$\frac{\partial L(w, b)}{\partial w_5} = 0.71 * 0.2 * 0.62 = 0.09$$

Substituting Eqs. (1.2), (1.3) and (1.4) in Eq. (1.1) gives

$$\begin{aligned}\frac{\partial L(w, b)}{\partial w_5} &= \frac{\partial L(w, b)}{\partial y_1} * \frac{\partial y_1}{\partial i_3} * \frac{\partial i_3}{\partial w_5} \\ \frac{\partial L(w, b)}{\partial w_5} &= -(O_1 - y_1) * y_1(1 - y_1) * a_1\end{aligned}$$

Similarly, we can find the weights for w_6 , w_7 and w_8 as follows:

$$\begin{aligned}\frac{\partial L(w, b)}{\partial w_6} &= -(O_2 - y_2) * y_2(1 - y_2) * a_1 = -0.06 \\ \frac{\partial L(w, b)}{\partial w_7} &= -(O_1 - y_1) * y_1(1 - y_1) * a_2 = 0.07 \\ \frac{\partial L(w, b)}{\partial w_8} &= -(O_2 - y_2) * y_2 * a_2 = -0.05\end{aligned}$$

For generalization we write the error at output node j due to the processing unit as

$$Err_j = -(O_j - y_j) * y_j(1 - y_j)$$

The gradient at output node j due to the processing unit and weight w_k from hidden unit h is

$$\frac{\partial L(w, b)}{\partial w_k} = \nabla w_k = Err_j * a_h$$

where a_h is the output of the activation function at the hidden node. Similarly,

$$\begin{aligned}\frac{\partial L(w, b)}{\partial b_3} &= -(O_1 - y_1) * y_1(1 - y_1) = 0.15 \\ \frac{\partial L(w, b)}{\partial b_4} &= -(O_2 - y_2) * y_2(1 - y_2) = -0.10\end{aligned}$$

The gradient at output node j due to the processing unit and bias b_k is

$$\frac{\partial L(w, b)}{\partial b_k} = \nabla b_k = \text{Err}_j$$

Let $\eta = 0.1$. Then w_5 is updated using gradient descent as follows:

$$\text{new_weight} = \text{old_weight} - \eta \nabla w$$

$$w_k = w_k - \eta \nabla w_k$$

$$w_5 = w_5 - \eta \frac{\partial L(w, b)}{\partial w_5} = 0.4 - (0.1 * 0.09) = 0.39$$

The bias is updated using gradient descent as follows:

$$\text{new_bias} = \text{old_bias} - \eta \nabla b$$

$$b_k = b_k - \eta \nabla b_k$$

$$b_3 = b_3 - \eta \nabla b_3 = 0.4 - 0.1 * 0.15 = 0.39$$

The updated weights are

w_5	w_6	w_7	w_8	b_3	b_4
0.39	0.61	0.49	-0.2	0.39	0.21

The weights shown above are updated only after the weights in the hidden layer are calculated.

Updating Weights at Hidden Layer

We move backwards and update the weights to the hidden nodes. This updation is slightly more complex because the hidden nodes affect the output nodes. Let us use the following notations:

1. w_{jk} is the weight of the nodes between layers j and k .
2. w_{kl} is the weight of the nodes between layers k and l .
3. a_h is the output of the node h at hidden layer k .

Note that layers j and k can be both hidden nodes or an input layer (j) and a hidden layer (k) as in our example. The nodes in layer k affect the output of the nodes in the next layer denoted as l . The error at unit h in the hidden layer k is given as

$$\text{Err}_h = (a_h)(1 - a_h) \sum_l \text{Err}_l w_{kl}$$

The weights and bias in the hidden layers are updated using the formulas

$$w_{jk} = w_{jk} - \eta \text{Err}_h a_h$$

$$b_j = b_j - \eta \text{Err}_h$$

This is applicable for any number of hidden layers. Substituting the values in the example problem, we get

$$\begin{aligned} \text{Err}_{h1} &= (a_1)(1 - a_1) (\text{Err}_1 \cdot w_5 + \text{Err}_1 \cdot w_6) \\ &= (0.62)(1 - 0.62)(0.15 * 0.4 + 0.15 * 0.6) = 0.03 \\ \text{Err}_{h2} &= (a_2)(1 - a_2) (\text{Err}_2 \cdot w_7 + \text{Err}_2 \cdot w_8) \\ &= 0.5 * (1 - 0.5) * [-0.1 * 0.5 + (-0.1) * (-0.2)] = -0.01 \end{aligned}$$

The above mentioned updates are continued till the error function in the output-layer approaches 0. The values of w_1 and b_1 are calculated as follows:

$$w_1 = w_1 - \eta \cdot \text{Err}_{h1} \cdot a_1 = -0.1 - 0.1 * 0.03 * 0.62 = -0.1$$

$$b_1 = b_1 - \eta \cdot \text{Err}_{h1} = -0.1 - 0.1 * 0.03 = -0.1$$

The remaining weights are also updated in the following manner.

w_1	w_2	w_3	w_4	b_1	b_2
-0.1	0.2	0.6	-0.3	-0.1	0.3

After the calculation of all the weights, all the weights are updated. This is said to be an epoch. The above-mentioned updates are continued till the error function in the output layer approaches 0.

Algorithm Weight_Update_Backpropagation(i,j,old_weight)

{
 for (specified number of epochs) OR (until minima of the error function is reached)
 for each weight j between the hidden layer and the output layer

$$\text{Err}_j = -(O_j - y_j) * y_j(1 - y_j)$$

$$\nabla w = \text{Err}_j * a_h$$

$$\nabla b = \text{Err}_j$$

End for

for each weight jk in the hidden layer

$$\text{Err}_h = (a_h)(1 - a_h) \sum_k \text{Err}_k w_{jk}$$

$$\nabla w = \text{Err}_h \cdot a_j$$

$$\nabla b = \text{Err}_h$$

End for

$$\text{new_weight} = \text{old_weight} - \eta \nabla w$$

$$\text{new_bias} = \text{old_bias} - \eta \nabla b$$

}

Some of the other learning algorithms are backpropagation through time explained in Chapter 3 under RNN and contrastive divergence explained under RBM in Chapter 5. Other recent variations are described in the following subsections.

1.3.3.6 Momentum-Based Gradient Descent

While performing gradient descent it is sometimes possible that we keep moving along the same direction. In that case, the algorithm can be speeded by travelling faster in the direction repeatedly moved. This is the intuition behind momentum-based gradient descent by Polyak (1964).

Weight Update

While updating the weight and bias, a fraction γ of the previous update is added while making the current updation. The update rule for the weights is given by

$$v_t = \gamma v_{t-1} + \eta \nabla w$$

$$\text{new_weight} = \text{old_weight} - v_t$$

Similarly, the update rule for the bias is given by

$$v_t = \gamma v_{t-1} + \eta \nabla b$$

$$\text{new_bias} = \text{old_bias} - v_t$$

1.3.3.7 Nesterov Accelerated Gradient (NAG)

When the algorithm performs gradient descent, it is possible that sometimes the minima is missed, and the algorithm oscillates back and forth as seen in Section 1.3.3.2 which includes learning rates. Nesterov accelerated gradient by Sutskever *et al.* (2013) is a mechanism to look ahead and decide if the pace should be slowed so that the algorithm reaches the minima and does not jump to the other side.

Weight Update

The update rule for NAG is given by

$$w_{\text{future}} = \text{old_weight} - \gamma \cdot v_{t-1}$$

$$v_t = \gamma \cdot v_{t-1} + \eta \nabla w_{\text{future}}$$

$$\text{new_weight} = \text{old_weight} - v_t$$

Similarly, the update rule for the bias is given by

$$b_{\text{future}} = \text{old_bias} - \gamma \cdot v_{t-1}$$

$$v_t = \gamma \cdot v_{t-1} + \eta \nabla b_{\text{future}}$$

$$\text{new_bias} = \text{old_bias} - v_t$$

1.3.3.8 AdaGrad

AdaGrad stands for adaptive gradient algorithm and was developed by Duchi *et al.* in 2011. Stochastic gradient descent uses same learning rate in all the iterations and in all the dimensions and is a tunable hyperparameter. Methods that take different steps across different dimensions are useful. Gradient descent with adaptive learning rate is a step in this direction. The learning rate is increased in proportion to the number of updates. That is, the learning rate of each parameter is updated based on the history of gradients.

Weight Update

The update rule for AdaGrad is given by

$$v_t = v_{t-1} + (\nabla w)^2$$

$$\text{new_weight} = \text{old_weight} - \frac{\eta}{\sqrt{v_t} + \epsilon} * (\nabla w)$$

Similarly, the update rule for the bias is given by

$$v_t = v_{t-1} + (\nabla b)^2$$

$$\text{new_bias} = \text{old_bias} - \frac{\eta}{\sqrt{v_t} + \epsilon} * (\nabla b)$$

Here ϵ is a very small value used for numerical stability. As seen in the formulas, v_t is a summation of the gradient. As time progresses the learning rate may converge to 0 because the denominator in $\eta/(\sqrt{v_t} + \epsilon)$ is positive.

1.3.3.9 RMSProp

RMSProp is an adaptive learning algorithm by Hinton *et al.* (2012) that uses the exponentially decaying average than the sum of gradients. RMS stands for Root Mean Square.

Weight Updation

The update rule for RMSProp is given by

$$E[g^2]_t = \beta \cdot E[g^2]_{t-1} + (1 - \beta) (\nabla w)^2$$

$$\text{new_weight} = \text{old_weight} - \frac{\eta}{\sqrt{E[g^2]_t}} * (\nabla w)$$

where g_t is the gradient at time ' t ', $E[g^2]_t$ is the moving average of squared gradients for each weight at time ' t ', $E[g^2]_{t-1}$ is the moving average of squared gradients for each weight at time ' $t-1$ ', β is the moving average parameter.

The best default value for β is 0.9. Similarly, the update rule for the bias is given by

$$E[g^2]_t = \beta \cdot E[g^2]_{t-1} + (1 - \beta) (\nabla b)^2$$

$$\text{new_bias} = \text{old_bias} - \frac{\eta}{\sqrt{E[g^2]_t}} * (\nabla b)$$

1.3.3.10 AdaDelta

AdaDelta, developed by Zeiler (2012), has lot of similarity to RMSProp and momentum-based gradient descent. Like RMSProp it accumulates the gradients. Later it also accumulates the updates like in momentum based gradient descent.

Weight Updation

The update rule for AdaDelta is given by

$$E[g^2]_t = \beta \cdot E[g^2]_{t-1} + (1 - \beta) g_t^2$$

$$E[\Delta x^2]_t = \beta \cdot E[\Delta x^2]_{t-1} + (1 - \beta) \Delta x_t^2$$

$$\Delta x_t = -\frac{\sqrt{E[g^2]_{t-1} + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

$$\text{new_weight} = \text{old_weight} + \Delta x_t$$

where Δx^2 is the squared parameter update

1.3.3.11 Adam

Adam stands for Adaptive Moment Estimation and was developed by Kingma *et al.* in 2015. It is a combination of AdaDelta, RMSProp and momentum.

Weight Updation

The update rule for Adam is given by the following steps:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) g_t^2$$

where m_t is the estimate of the first moment and is the decaying average of the gradient, v_t is the estimate of the second moment and is the decaying average of the square of the gradient.

These values are initialized to 0. To ensure that the value does not get biased towards 0, the bias-corrected first and second moment estimates are computed as follows:

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The new weight now becomes

$$\text{new_weight} = \text{old_weight} - \eta \cdot \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

The best values for the parameters are $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$.

1.3.4 | Vanishing Gradient Problem

Certain times as the weights pass through the network, the gradient keeps on decreasing as it moves from one layer to the other. When the number of layers in the architecture is very large, the gradient becomes so small that it cannot be used. This is called the vanishing gradient problem. This problem arises because of the nature of the gradient-based method and the activation function.

1.3.5 | Exploding Gradient Problem

The converse of vanishing gradient problem is the exploding gradient problem wherein the weights are updated by very large values during training. The result is that the model will be unable to learn from the training data. This problem can occur easily in RNNs and is explained in detail in the Chapter 3. During training, the model loss may go to NaN.

1.3.5.1 Gradient Clipping

The simplest solution to the exploding gradient problem is using gradient clipping. A threshold value is set and if the gradient of the loss function exceeds the threshold, the gradient is clipped or set to the value of the threshold.

1.3.5.2 Weight Regularization

Weight regularization is useful for solving the problem of exploding gradient and to avoid overfitting in machine learning problems. The regularization term is added to the loss function to encourage smaller weights. There are two types of regularization namely, L1 regularization and L2 regularization.

L1 Regularization

In L1 regularization, the sum of the absolute weights is added to the loss function $[L(w, b)]$, that is,

$$L(w, b) + \lambda \sum_{i=1}^n |w_i|$$

where λ is the regularization parameter ($0 < \lambda < 1$).

L2 Regularization

In L2 regularization, the sum of the square of the weights is added to the loss function $[L(w, b)]$, that is

$$L(w, b) + \lambda \sum_{i=1}^n (w_i)^2$$

It is also termed as ‘weight decay’ or ‘shrinkage’.

1.3.6 | Weight Initialization

Weight updation is a major factor in improving the ANN. The right weight helps in the loss function to converge after a reasonable amount of time. Weights are usually initialized randomly. When the weights are too large or too small (close to zero), the gradients will get closer to zero resulting in the vanishing gradient problem.

1.3.6.1 Zero Initialization

A bad choice for initializing weights would be to initialize all weights to 0. When the weights are initialized to zero, the weights in the subsequent layer would be updated the same way and remain the same throughout the training.

1.3.6.2 Uniform Initialization

When the weights are uniform across, then the output of the activation function of every neuron in the hidden and output layers will be the same. The gradient computed will be the same. The parameter update depends on the gradient and that also remains uniform. Breaking the symmetry

is essential for proper weight updation. This problem is called the symmetry breaking problem and arises when all the weights are initialized in a uniform way to the same value. Random initialization is required to break this symmetry.

More principled ways of initializing weights are being researched and the two most common techniques are explained as follows:

1. **Xavier:** Xavier initialization as given by Xavier Glorot and Yoshua Bengio (2010) assigns the weight from a Gaussian distribution. For this purpose, we consider a normal distribution with mean 0 and finite variance. The intention behind the method is to not change the variance as it passes from one layer to another. This avoids the vanishing gradient problem and the exploding gradient problem. Xavier initialization initializes the weights from a Gaussian distribution with mean 0 and variance $1/N$. In the Caffe implementation, N is the number of input neurons.

$$\text{Var}(w_i) = \frac{1}{N_{\text{in}}}$$

But Glorot and Bengio (2009) in their initial paper used the average of the number of input neurons and output neurons to calculate N .

$$\text{Var}(w_i) = \frac{2}{N_{\text{in}} + N_{\text{out}}}$$

2. **He:** The Xavier initialization does not work well with ReLu neurons. So, a factor of 2 is included in the paper by He, Rang, Zhen and Sun (2015). Thus, variance is given by

$$\text{Var}(w_i) = \frac{2}{N_{\text{in}}}$$

In a ReLu, the output is zero for half the input. To ensure a constant variance, the variance needs to be doubled as given by He *et al.*

1.3.7 | What is Deep Learning?

The world is moving towards Artificial Intelligence (AI) and to widen and break the boundaries of AI, deep learning is a powerful tool. Deep learning has found its application in processing image, video, text and sound. Its strength was evident when it was used to analyze the surrounding in driverless cars, recognize speech in Alexa, recognize and detect objects in images in the ImageNet challenge, perform sentiment analysis and so on.

So, what is deep learning. Deep learning is a type of machine learning in which large volumes of training data and artificial neural networks with multiple layers are used to achieve the machine learning tasks. We use the terminology ‘Deep’ because the number of processing layers is huge. Based on the many ways the layers are connected, we have many types of deep learning networks.

The functioning of deep learning is slightly different from traditional machine learning. To show how deep learning differs from traditional machine learning let us differentiate classifying a horse with and without using deep learning. A horse has Crest, Hoof, Muscular legs, Muzzle and Lush tail. In traditional learning we explicitly state these features to classify a horse using a phase called “feature extraction”. But in deep learning, feature extraction and classification are intertwined in the same phase. The deep learning architecture is expected to learn the features by itself.

To perform this deep learning requires a lot of processing power and training data. The current scenario given below has made this easy. Getting parallel, fast, cheap processing power is no longer difficult. From PC CPU which works in 1–3 Gflops/sec average, we now have GPUs that work in 100 Gflops/sec average. The second necessity of the deep learning architecture is huge training data. With the advent of big data, Internet of things and cloud computing, we now have huge volumes of data to use.

SUMMARY

This chapter introduces the fundamental concepts in neural networks since it is the basic building block of deep learning architectures. The optimization techniques and the problems posed by various activation functions have been dealt with in detail in this chapter. Readers must have a good understanding of the concepts and terminologies used this chapter for further reading. The remaining chapters introduce the various types of deep learning architectures and software frameworks on which deep learning can be implemented.

REVIEW QUESTIONS

1. Differentiate the various machine learning tasks.
2. Why is it not possible to use perceptrons to solve non-linearly separable problems? Prove with an example.
3. What is the exploding gradient problem? Mention some solutions to solve this problem.
4. How does the initialization of weights affect the performance of the ANN?
5. In optimization techniques, what is the significance of the learning rate?

ASSIGNMENT PROBLEMS

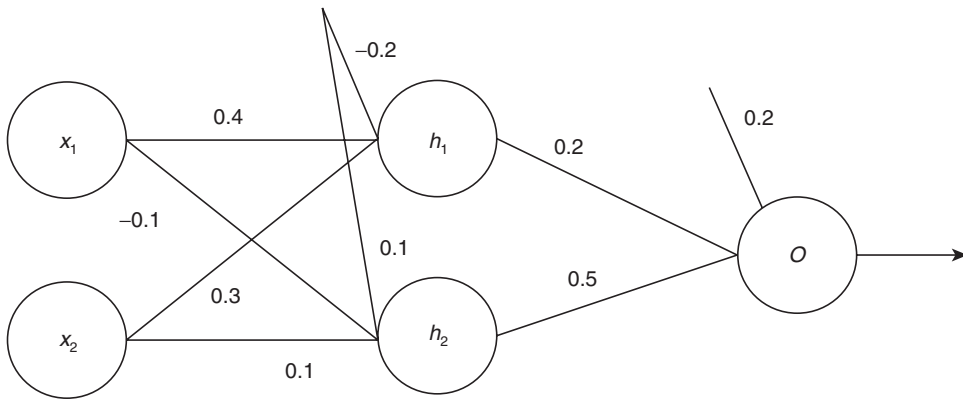
1. Consider the XNOR operation. Assuming that it is a classification problem with the output being the two classes ‘0’ and ‘1’, check if it is a linearly separable problem.
2. The Harley–Davidson Iron 883 has the following normalized features:
(Displacement, Mileage, Kerb Weight, IsRedAvailable) = (8.83, 0.20, 2.56, 1)
Now consider a person who wants to decide whether to buy a Harley–Davidson Iron 883. He assigns the following weights to each of these inputs: $W = [0.9, 0.4, 0.7, 1]$. Further, suppose that $\theta = 8$. Based on the above information, do you think he will buy the bike based on a simple McCulloch–Pitts neuron.

3. Consider 4 data points with 3 dimensions that are divided into two classes, Class 1 and Class 2.

Sample	Data Point (x)	Class
S1	[1, 0, 1]	1
S2	[0, 1, 1]	0
S3	[1, 1, 0]	1
S4	[0, 0, 1]	0

Let the initial weight vector be $[-0.5, 1, 0.2]$. Apply the simple trial-and-error based perceptron learning algorithm to the sample data in the same order. Find the values of the weight vector after the algorithm converges. If required, repeat accessing the samples in the same order till the algorithm converges. (Hint: If $x \in 1$, perform $w + x$, else perform $w - x$. Continue this for every x and redo till the algorithm converges.)

4. Consider the following multilayer feedforward network:



Here $(x_1, x_2) = (1, 0)$ and $O = (0.3)$. Using the backpropagation algorithm, show how the weights and bias are updated after one epoch.

5. Derive the gradient of the hyperbolic tangent.

REFERENCES

1. Diederik, Kingma; Ba, Jimmy. 2014. *Adam: A Method for Stochastic Optimization*. [arXiv:1412.6980v9 [cs.LG] 30 Jan 2017]
2. Duchi, John, Hazan, Elad, and Singer, Yoram. 2011. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*. 12: 2121–2159
3. Glorot, Xavier and Yoshua Bengio. 2010. Understanding the difficulty of training deep feed-forward neural networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, PMLR. 9: 249–256

4. McCulloch, Warren and Walter Pitts. 1943. A Logical Calculus of Ideas Immanent in Nervous Activity. *Bulletin of Mathematical Biophysics*. 5 (4): 115–133
5. Polyak, Boris T. and Juditsky, Anatoli B. 1992. Acceleration of stochastic approximation by averaging. *SIAM J. Control Optim.* 30 (4): 838–855
6. Robbins, Herbert and Siegmund, David O. 1971. A convergence theorem for non-negative almost supermartingales and some applications. In *Optimizing Methods in Statistics* (Rustagi, Jagdish S). Academic Press
7. Rumelhart, David E., Hinton, Geoffrey E., and Williams, Ronald J. 1986. Learning representations by back-propagating errors. *Nature*. 323 (6088): 533–536
8. Sutskever, Ilya, Martens, James, Dahl, George, and Hinton, Geoffrey E. 2013. On the importance of initialization and momentum in deep learning (Sanjoy Dasgupta and David Mcallester, ed). In *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*. 28. Atlanta, GA. pp. 1139–1147.
9. Tieleman, Tijmen and Hinton, Geoffrey. 2012. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning*, 4: 26–31.
10. Zeiler, Matthew D. 2012. ADADELTA: An Adaptive Learning Rate Method. [arXiv:1212.5701v1 [cs.LG] 22 Dec 2012]

Convolutional Neural Network

CHAPTER 2

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand the basics of convolutional neural network and various components in its architecture.
- Analyze the effect of different activation functions of a CNN unit.
- Understand CNN properties, architectural variants and their applications.

2.1 | INTRODUCTION

A Convolutional Neural Network (CNN), or ConvNet, is a class of deep, feed-forward artificial neural networks, most commonly applied to analyze visual imagery. It uses comparatively little preprocessing compared to other image classification algorithms. Convolutional networks were inspired by biological processes as the connectivity pattern between neurons looks like the organization of an animal visual cortex. Individual cortical neurons respond to stimuli only in a restricted region of the visual field known as the *receptive field*. The receptive fields of different neurons partially overlap such that they cover the entire visual field.

CNNs, like neural networks, are made up of neurons with learnable weights and biases. Each neuron receives numerous inputs, takes a weighted sum over them, passes it through an activation function, and responds with an output. CNN showed the most efficacies in image and video recognition, recommender systems, image classification, medical image analysis, and natural language processing. Thus, CNNs assure promising applications in deep learning systems. Computer vision through CNNs has several applications such as self-driving cars and robotics.

The main operation in a CNN system is mathematical convolution. Convolution is a simple mathematical operation; it means to roll together multiple data or functions that represent data. Convolution is a measure of the overlapping of two functions.

Like other neural network architectures, a CNN also has an input layer, number of hidden layers, and an output layer, some of which are convolutional layers using a mathematical model to pass on the results to successive layers.

The first layer distinguishes basic attributes like lines and curves. The next layer takes in more complex mathematical features. At higher levels, the brain recognizes that the image considered is an object with the configuration of edges and colors. Similarly, a CNN processes an image using weight matrix (also called filters or features) that detect specific attributes of the image such as

diagonal edges, vertical edges, curves, etc. As the image progresses through each layer, the filters recognize more complex attributes as well. In the case of a CNN, the convolution is performed on the input data with the use of a filter or kernel to then produce a feature map.

We execute a convolution by sliding the filter over the input. At every location, a matrix multiplication is performed and the result is summed onto the feature map. The input image is given as a matrix or as a tensor to the CNN. Since the input image cannot be fed directly into a CNN system, suitable processing has to be done on the image. We first extract the features from an image. These features are then provided as input to the CNN system. On further downsampling, a list of kernels or filters are applied by convoluting such a filter with the image.

Figure 2.1 shows the convolved image after applying filters such as edge detection, smoothing, Gaussian blur with 15×15 kernel, Median blur with 15×15 kernel, sharpening using blur, and Laplacian filter.

2.2 | COMPONENTS OF CNN ARCHITECTURE

A CNN is shaped as a hierarchical structure for fast feature extraction and classification. The main objective is to extract the input image volume and convert it into an output volume that holds class scores. A differential function is used for image processing. CNN consists of a stack of convolutional and subsampling layers, followed by a series of fully connected layers. Figure 2.2 represents the structure of a CNN.

The various layers composing a CNN are as follows:

1. **Convolutional Layer:** This layer is used for feature extraction, obtaining original features from the volume map, extracting data, and creating feature maps.
2. **Pooling or Downsampling Layer:** This layer reduces the number of weights and controls overfitting.
3. **Flattening Layer:** This layer prepares the CNN output to be fed to a fully connected neural network. It should be noted that CNN is not fully connected like a traditional neural network.
4. **Fully Connected Layer:** These are the layers at the top of CNN hierarchy. As mentioned above, neurons in the other layers in a CNN are not *fully connected*, that is, each neuron is connected with all activations in the previous layers. The fully connected layers are responsible to sum up all the detections made in previous layers. They detect global characteristics of the input using the features detected in the lower layers.

A basic CNN is usually made of these four layers, but there is no restriction on the total number of layers. The main concern when dealing with CNN is to find the right kernel or right features needed. Convolution techniques are very efficient in finding the features of images if the right kernel is used. The usual method takes in every pixel of the image as a feature and thus also as an input node. The result from each convolution is placed in the next hidden layer node. A typical network consists of four-layered convolution network followed by a regular neural network which is provided into a logistic processor.

Operation	Convolved Image	Operation	Convolved Image
Original image: Lenna		Median blur with 15×15 kernel	
Edge detection		Sharpening using blur	
Smoothing using 5×5 kernel		Laplacian filter	
Gaussian blur with 15×15 kernel			

FIGURE 2.1 Convolved image after applying filters.

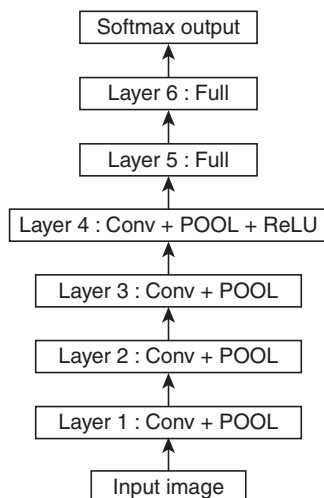


FIGURE 2.2 Structure of a CNN.

2.2.1 | Convolution Layer

The convolution layer is the basic layer that builds a CNN. Convolution is a mathematical operation on two functions to produce a third function. Filters or kernels are the base units in this system. These layers consist of a series of learnable filters, which have a small receptive field, but extend through the full depth of the input volume. During the forward pass, each filter is convolved across the input volume. The input volume contains the width and height. Convolution is done by computing the dot product between the entries of the filter and the volume. Thus, the networks acquire the properties of the filter and they can detect a specific type of feature at a particular position in the input. Let us understand the process of convolution with an example.

2.1 EXAMPLE

Consider the input matrix in Fig. 2.3. The input matrix is convolved with the filter to produce a feature map. Since the shape of the filter is 3×3 , this convolution is called 3×3 convolution. Similarly, it can be 5×5 or 1×1 convolution, depending on the application.

0	1	1	0	0
0	0	1	0	0
0	1	1	0	0
0	0	1	0	0
0	1	1	0	0

(a)

0	1	1
0	0	1
0	1	1

(b)

FIGURE 2.3 (a) Input matrix; (b) filter or kernel.

The following are a few parameters that can be adjusted for a kernel:

1. **Size:** The size of the filter (e.g., 5×5).

2.1 EXAMPLE (Continued)

2. **Stride:** The rate at which kernel passes (a stride of 2 will move the kernel by 2 increments).
3. **Padding:** Zero-padding on the outside of the image to make sure that kernel pass is perfect on the edges.
4. **Output layers:** Number of kernels applied.

All 3×3 subsets of the input are convolved with the 3×3 filter to produce a feature map. The first 3×3 subset of the input matrix is matrix-multiplied with the filter and summed up to get the first value of the feature map.

The blue [grey in Fig. 2.4(a)] area is the current subset of the input feature being convolved. This is the receptive field. Once the first value is obtained, the filter is slid to the right for further computation.

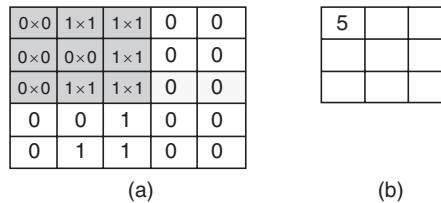


FIGURE 2.4 (a) First subset of the input feature; (b) first value of the feature map.

In Fig. 2.5, the receptive field is changed to the next 3×3 subset and convolved to get the next value of the feature map. Once the second value is obtained, the filter is again slid to the right.

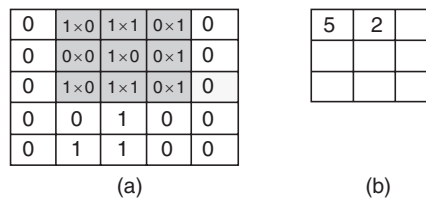


FIGURE 2.5 (a) Second subset of the input feature; (b) second value of the feature map.

Again, the convolution of the third receptive field and filter produces the third value of the feature map. By repeating these steps, the big input feature is reduced to a small feature map which implies the properties of the input. In the above example, we use a 3×3 filter that can be placed over the 5×5 input in 9 ways. So, we represent the features of the 5×5 matrix in a 3×3 feature map. In a different example, if we use a 2×2 filter, it can be placed over 16 different ways over the input matrix. So the 5×5 input matrix is reduced to 4×4 matrix in this case. Figure 2.6 shows the step-by-step visualization of the entire convolution.

2.1 EXAMPLE (Continued)

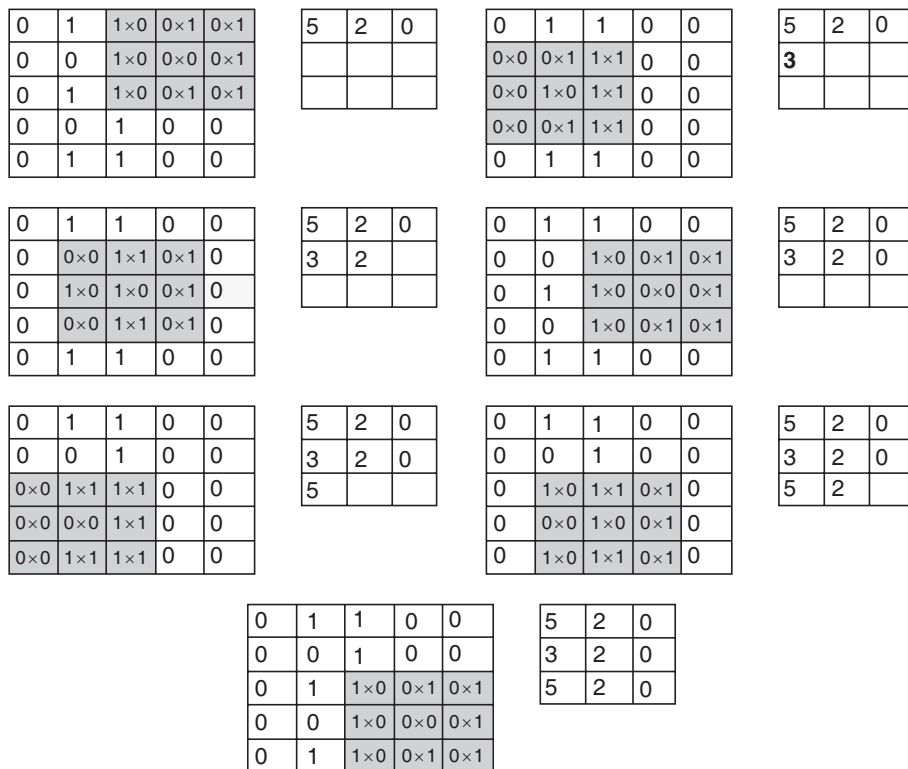


FIGURE 2.6 Complete process of convolution.

In this example, a 2D 3×3 filter is used. But in reality, an image is represented as a 3D matrix with width, height, and depth (color channels like RGB) as the dimensions; hence the filter used should also be 3D (i.e., the filter depth should be equal to the input channel size). Usually, many filters are applied on the input image which results in multiple distinct feature maps that are stacked together. These feature maps become the output of the convolution layer as shown in Fig. 2.6. Multiple convolutions are performed independently to get disjoint maps as represented in Fig. 2.7. As seen in the figure, the first layer filters convolve around the input matrix, and tries to identify a specific feature. When the feature is found, it is recorded in a feature map. Likewise, independent disjoint maps are created for each unique feature.

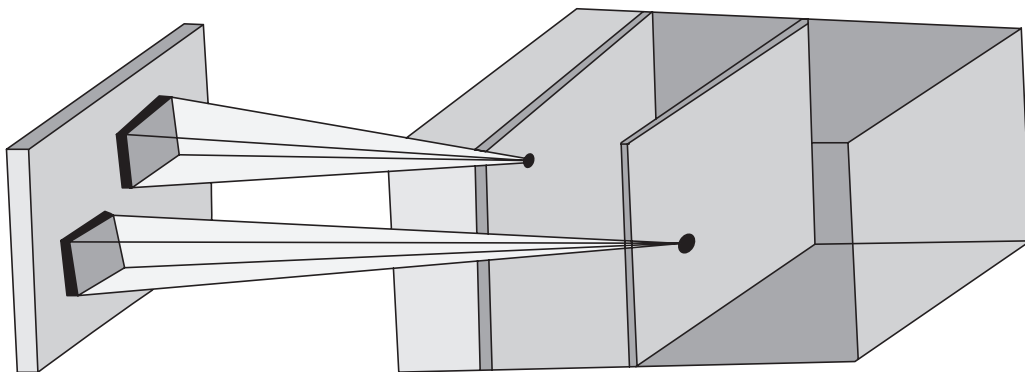


FIGURE 2.7 Independent convolutions producing disjoint maps.

2.2.1.1 Receptive Field

If there is high dimensional input image, then it is impossible to connect all neurons with all possible regions of the input size. This would result in the need to train too many weights resulting in high computational complexity. As an alternative, each neuron is connected only to a local region of the input volume. The spatial extent of this connectivity is a hyperparameter called the receptive field of the neuron. Consider the following example: the size of input image is $[32 \times 32 \times 3]$. If the filter size is 5×5 , then each neuron in the convolution layer will need to be trained for $5 \times 5 \times 3 = 75$ weights (and +1 bias parameter). The extent of the connectivity along the depth axis must be 3, since this is the depth of the input volume. The number of output features in each dimension is calculated based on the number of input features and the convolution properties using the following formula:

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

where n_{in} is the number of input features, n_{out} is the number of output features, k is the convolution kernel size, p is the convolution padding size, and s is the convolution stride size.

2.2.1.2 Feature Map

The size of the feature map (convolved feature) is controlled by three parameters, namely, depth, stride, and zero-padding. These parameters have to be decided before the convolution step is performed:

1. **Depth:** Depth corresponds to the number of filters used for the convolution operation. If convolution is performed on an original image using n distinct filters, then it produces n different feature maps. Thus, the depth of the feature map would be n .
2. **Stride:** Stride is the number of pixels by which the filter matrix is slid over the input matrix. When the stride is 1 then the filters are moved one pixel at a time. When the stride

is 2, then the filters jump 2 pixels at a time as shown in Fig. 2.8. A larger stride will produce smaller feature maps.

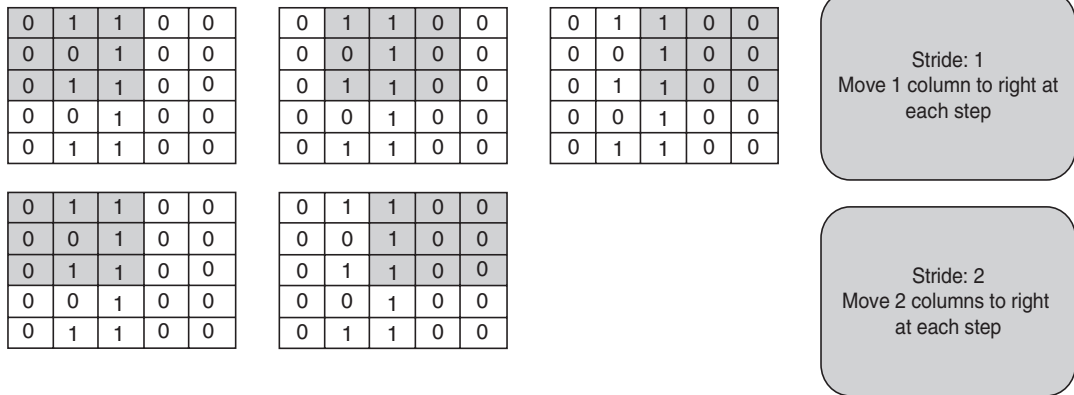


FIGURE 2.8 Representation of stride 1 and stride 2.

- Zero-padding:** Zero-padding is the process of adjusting the input size as per the requirement by adding zeros to the input matrix. It is mostly used in designing the CNN layers when the dimensions of the input volume need to be preserved in the output volume. Sometimes filter does not perfectly fit the input image. In that case, we need to either pad the picture with zeros so that it fits or drop the part of the image where the filter did not fit. This is called *valid padding* which keeps only the valid part of the image. When we add zero-padding, convolution is called *wide convolution*, and when zero-padding is not added, it is called *narrow convolution*. The representation of an input matrix with zero-padding is shown in Fig. 2.9.

2.2.2 | Pooling or Downsampling Layer

It is common to periodically insert a pooling layer in-between successive convolutional layers in a CNN architecture. Its function is to progressively reduce the spatial size of the representation to reduce the number of parameters and computation in the network, and hence to also control overfitting. The pooling layer operates independently on every depth slice of the input and resizes it spatially, using the MAX operation. The most common form is a pooling layer with filters of size 2×2 applied with a stride of 2 downsamples every depth slice in the input by 2 along both width and height, discarding 75% of the activations. In this case, every MAX operation would be taking at max over 4 numbers (little 2×2 region in some depth slice). The depth dimension remains unchanged. The following are general features of the pooling layer:

1. Accepts a volume of size $W_1 \times H_1 \times D_1$.
2. Requires two hyperparameters – spatial extent F and stride S .
3. Produces a volume of size $W_2 \times H_2 \times D_2$ where

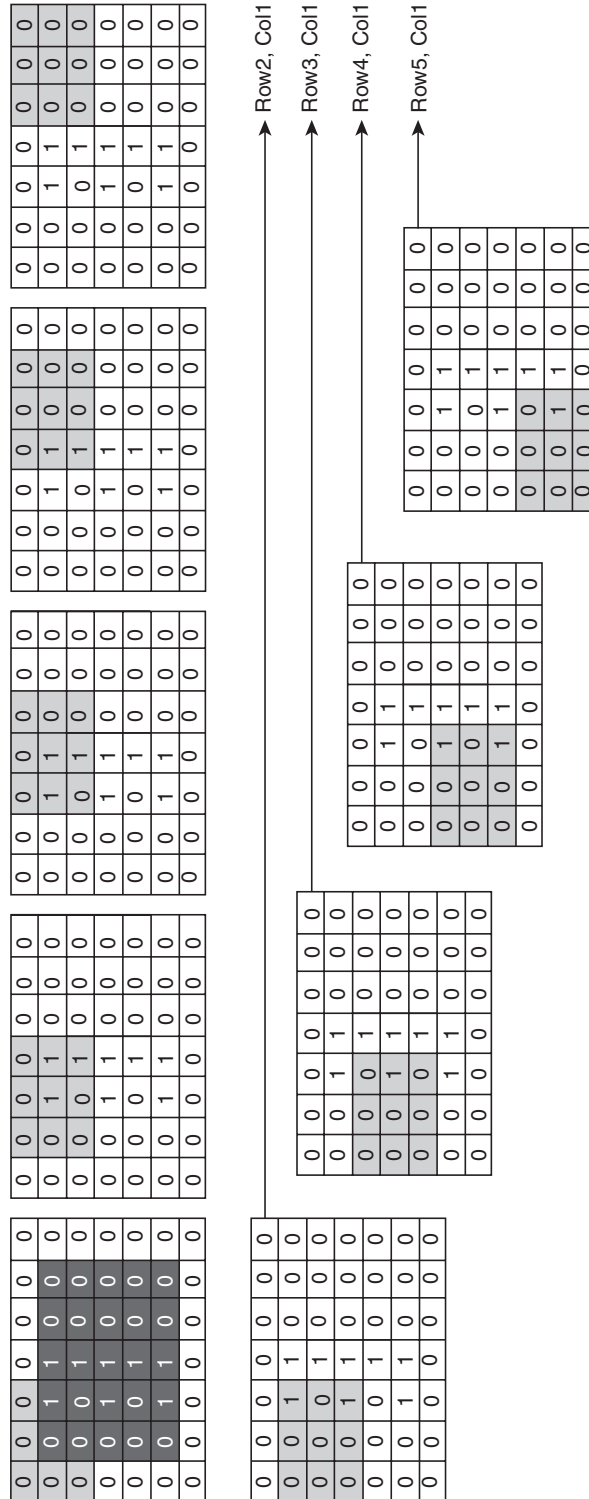


FIGURE 2.9 Representation of zero-padding.

$$W_2 = \frac{(W_1 - F)}{S} + 1$$

$$H_2 = \frac{(H_1 - F)}{S} + 1$$

$$D_2 = D_1$$

4. Introduces zero parameters since it computes a fixed function of the input.

Pooling layers section would reduce the number of parameters when the images are too large. Spatial pooling is called *subsampling* or *downsampling*; it reduces the dimensionality of each map but retains the important information. Spatial pooling can be of different types:

1. **Max-Pooling:** Pooling layers are generally placed in between convolution layers. Pooling layers reduce the spatial size of the representation to reduce the computation required and puts a check on overfitting. Max pooling determines the important features of the input image, like sharp edges and curves. Max-pooling takes the largest element from the rectified feature map.

Consider the example where the input volume of size $[224 \times 224 \times 64]$ is pooled with filter size 2 and stride 2 into output volume of size $[112 \times 112 \times 64]$. Notice that the volume depth is preserved.

The most common downsampling operation is max, giving rise to max-pooling with a stride of 1, as shown in Fig. 2.10. That is, each max is taken over 4 numbers (little 2×2 square).

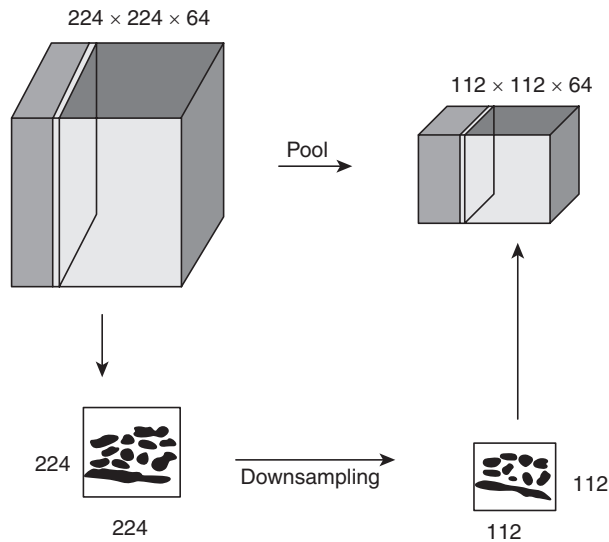


FIGURE 2.10 Max-pooling.

2. **Average Pooling:** Average pooling identifies the features in a smooth fashion. It takes into account all the features to produce an average value that may or may not detect sharp

features. Average pooling works well for straight lines and smaller curves, but cannot detect extreme features like sharp edges.

3. **Sum Pooling:** The sum of all elements in the feature map is called sum pooling. Max pooling takes the maximum value from the input region, whereas sum pooling uses the sum of the values in the input region. Both max pooling and sum pooling decrease the dimension of the input data while efficiently retaining the features. On the other hand, average pooling may or may not detect all the features.

2.2.2.1 *Getting Rid of Pooling*

Many people dislike pooling operation and try to work without it. To reduce the size of the representation without using a pooling layer, people may want to use a larger stride in convolutional layer once in a while. Discarding pooling layers has also been found to be important in training good generative models, such as Variational AutoEncoder (VAE) and Generative Adversarial Network (GAN). It seems likely that future architectures will feature very few to no pooling layers.

2.2.3 | Flattening Layer

Having extracted the features from the convolutional layer, and reduced the dimension by the pooling layer (optionally), it is the time for classification. The fully-connected layers cannot process multi-dimensional data though so that data should be reduced down to single dimension (flattened) for each channel before processing.

2.2.4 | Fully Connected Layer

Neurons in a fully connected layer have full connections to all activations in the previous layer, as seen in regular neural networks. Hence, their activations can be computed with a matrix multiplication followed by a bias offset.

The fully connected layer is a traditional MultiLayer Perceptron (MLP) that uses a softmax activation function in the output layer. The term ‘fully-connected’ implies that every neuron in the previous layer is connected to every neuron on the next layer.

The output from the convolutional and pooling layers represent high-level features of the input image. The purpose of the fully connected layer is to use these features for classifying the input image into various classes based on the training dataset.

Apart from classification, adding a fully connected layer is also a (usually) cheap way of learning non-linear combinations of these features. Most of the features from convolutional and pooling layers may be good for the classification task, but combinations of those features might be even better.

The sum of output probabilities from the fully connected layer is 1. This is ensured by using the softmax as the activation function in the output layer of the fully connected layer. The softmax function takes a vector of arbitrary real-valued scores and squashes it to a vector of values between zero and one that sum to one.

2.3 | RECTIFIED LINEAR UNIT (ReLU) LAYER

Activation layer is applied immediately after each convolution layer to introduce non linearity in the convolution layers. Non-linear functions like tanh and sigmoid were used initially to induce non-linearity. Now Rectified Linear Units (ReLU) are used, as they are found to be faster in training the network without compromising with accuracy. It also alleviates the vanishing gradient problem, which is the issue where the lower layers of the network train very slowly because the gradient decreases exponentially through the layers. The ReLU layer applies the function

$$f(x) = \max(0, x)$$

to all of the values in the input volume. Basically, ReLU just changes all the negative activations to 0. It introduces the non-linear property in the overall network without affecting the receptive fields of the convolution layer.

ReLU is the most commonly used activation function for the outputs of CNN neurons. For negative input, the function returns 0, but for any positive value x it returns that value back.

This function can be used by neurons just like any other activation function. A node using the rectifier activation function is called ReLU node. The purpose of ReLU is to introduce non-linearity in CNN. There are other non-linear functions such as tanh or sigmoid, but performance-wise ReLU is better than the other two without making a significant difference to generalization accuracy. ReLU is important because it does not saturate; the gradient is always high (equal to 1) if the neuron activates. As long as it is not a dead neuron, successive updates are fairly effective. ReLU is also very quick to evaluate.

2.3.1 | Leaky ReLU and Randomized ReLU

Leaky ReLU helps increase the range of the ReLU function. Usually, the activation value a is 0.01 or so. When a is not 0.01 then it is called *randomized ReLU*. Therefore, the range of the leaky ReLU is $-\infty$ to ∞ . Both leaky and randomized ReLU functions are monotonic in nature. Also, their derivatives also monotonic in nature.

Leaky ReLU has a small slope for negative values, instead of altogether zero. For example, leaky ReLU may have $y = 0.01x$ when $x < 0$. Leaky ReLU is not always superior to plain ReLU, and should be considered only as an alternative, because the result is not always consistent.

Leaky ReLU has two benefits:

1. It fixes the “dying ReLU” problem, as it does not have zero-slope parts.
2. It speeds up training. Unlike ReLU, leaky ReLU is more “balanced,” and may therefore learn faster.

2.4 | EXPONENTIAL LINEAR UNIT (ELU, OR SELU)

Exponential linear units (ELUs) try to make the mean activations closer to zero which speeds up learning. It has been shown that ELUs can obtain higher classification accuracy than ReLUs. ELU has a small slope for negative values, which is similar to leaky ReLU. Instead of a straight line, it uses a log curve as shown in Fig. 2.11.

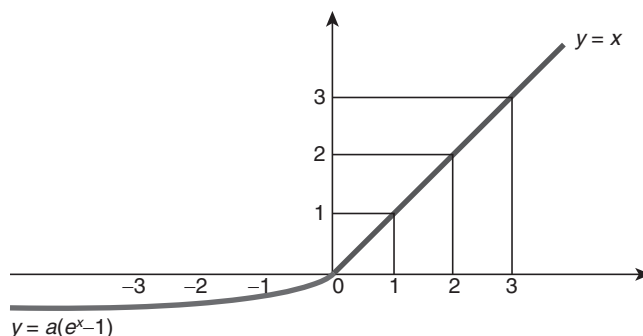


FIGURE 2.11 Exponential linear unit.

It is designed to combine the good parts of ReLU and leaky ReLU. While it does not have the dying ReLU problem, it saturates for large negative values, allowing them to be essentially inactive. It is sometimes called Scaled ELU (SELU) due to the constant factor a .

2.4.1 | Maxout

The Maxout neuron computes the function $\max(w_1Tx + b_1, w_2Tx + b_2)$. Notice that both ReLU and leaky ReLU are a special case of this form (e.g., for ReLU we have $w_1, b_1 = 0, w_2, b_2 = 0$). The Maxout neuron therefore enjoys all the benefits of a ReLU unit (linear regime of operation, no saturation) and does not have its drawbacks (dying ReLU). However, unlike the ReLU neurons it doubles the number of parameters for every single neuron, leading to a high total number of parameters.

2.5 | UNIQUE PROPERTIES OF CNN

2.5.1 | Weight Sharing

A property of convolutional neural networks is to share the same weight vectors across the receptive fields in a particular layer as shown in Fig. 2.12. It is known that a filter is applied to all the receptive fields in a layer to produce a feature map. By sharing the weights, the filter is learned irrespective of the position in the receptive field. Weight sharing is advantageous in some of the following ways:

1. It reduces the number of parameters to be learned or optimized.
2. Model converges faster.
3. It avoids overfitting since the neurons share the same weights.

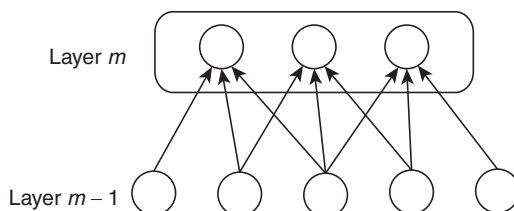


FIGURE 2.12 Neurons sharing same weights.

SOURCE: *Convolutional Neural Networks (LeNet)*; <http://deeplearning.net/tutorial/lenet.htm>

2.5.2 | Translation Invariance

Once the human eye recognizes an object, it can recognize that same object under any circumstance (even if it is moved, rotated, under different lighting, and so on). Translation invariance provides CNN with one similar property, which allows CNN to recognize the identified object even when it is moved to any different place.

In Fig. 2.13, the statue is moved to different places. A well-trained neural network can recognize this statue in all the three pictures even though the pixel values are moved to different places. The output of the model remains the same irrespective of shifts in the input image. For example, face detector module in OpenCV identifies the face from any image regardless of the position of the face (it may differ from the reference shape predictor image that OpenCV uses) and says “FACE FOUND”.

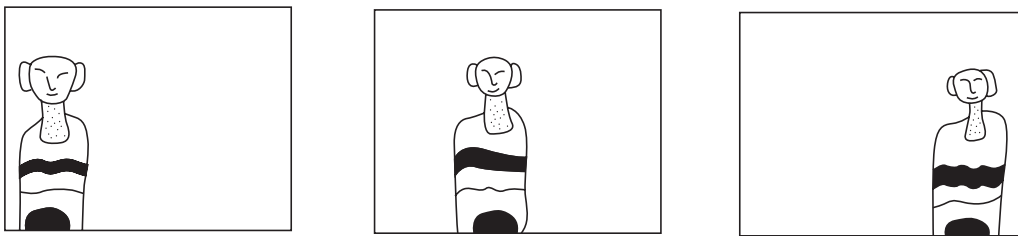


FIGURE 2.13 Translation invariance.

SOURCE: *Stack Exchange*; <https://stats.stackexchange.com/questions/208936/what-is-translation-invariance-in-computer-vision-and-convolutional-neural-network>

2.6 | ARCHITECTURES OF CNN

CNNs are special type of networks specifically designed to identify patterns from images. There are several CNN architectures designed to solve a particular image processing problem such as optical character recognition (OCR), object detection, face recognition, etc. The architectures differ in the number of parameters, number of layers, and type of layers. The error rate decreases as the architecture gets deeper. But some architectures, such as GoogLeNet, achieved minimal error rates with reduced number of parameters.

2.6.1 | LeNet

Introduced in 1998 by LeCun *et al.*, LeNet was the first deep convolutional architecture. It was used to perform OCR by several banks to recognize handwritten numbers on cheques digitized in 32×32 grayscale images. Though its ability to solve was admirable, it was constrained by the availability of computing resources at that time because of its high computational costs. Its architecture is shown in Fig. 2.14.

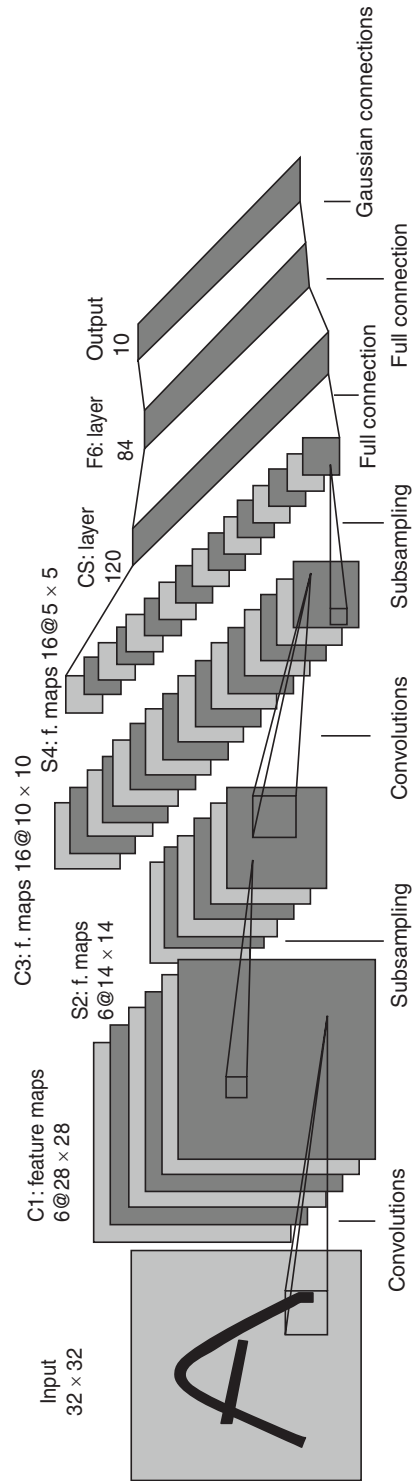


FIGURE 2.14 Architecture of LeNet.

SOURCE: LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998) Gradient-Based Learning Applied to Document Recognition. *Proceedings of IEEE*.

2.6.2 | AlexNet

AlexNet was designed by the SuperVision group consisting of Alex Krizhevsky, Geoffrey Hinton, and Ilya Sutskever. It was the first CNN architecture that triumphed in the ImageNet Large Scale Visual Recognition Competition (ILSVRC) 2012 contest with error rates half that of its nearest competitors (from 26% to 15.3%). This victory dramatically stimulated the trend toward deep learning architectures in computer vision.

From Fig. 2.15, we can see that it has a similar architecture as LeNet but deeper and with more stacked convolutional layers. It has 5 convolutional layers and 3 fully connected layers summing up to 8 layers in total. ReLU was applied after each convolutional and fully connected layer whereas dropout was applied before the first and second fully connected layers. It was trained for 6 consecutive days on two NVIDIA Geforce GTX 580 GPUs and so the processing was split into two pipelines.

2.6.3 | ZFNet

ZFNet is the winner of the ILSVRC 2013. It achieved a top-5 error rate of 14.8% which is already a significant improvement over AlexNet. It was mostly an achievement by tweaking the hyperparameters of AlexNet while maintaining the same structure with additional deep learning elements as shown in Fig. 2.16.

2.6.4 | GoogLeNet

Since 2012, CNN architectures have been coming out with flying colors in the ILSVRC contest. GoogLeNet (also known as Inception) is the winner of the ILSVRC 2014 contest. It achieved an error rate of 6.67% with its 22 layers, which was very close to the human level performance.

The architecture shown in Fig. 2.17 used a CNN inspired from LeNet but introduced 9 inception modules. They used batch normalization, image distortions and RMSprop, and were based on several but very small convolutions. The idea was to cover a bigger area but to keep a fine resolution on the images. So it convolved different sizes from 1×1 to 5×5 in parallel.

2.6.5 | VGGNet

The runner-up at the ILSVRC 2014 competition was dubbed VGGNet by the community. It was developed by Simonyan and Zisserman. VGGNet consists of 16 convolutional layers as shown in Fig. 2.18 and is very appealing because of its very uniform architecture. Similar to AlexNet, it has only 3×3 convolutions, with lots of filters. Trained on 4 GPUs for 2–3 weeks, it is currently the most preferred choice in the community for extracting features from images. The weight configuration of the VGGNet is publicly available and has been used in many applications and challenges as a baseline feature extractor. However, VGGNet consists of 138 million parameters, which can be a bit challenging to handle.

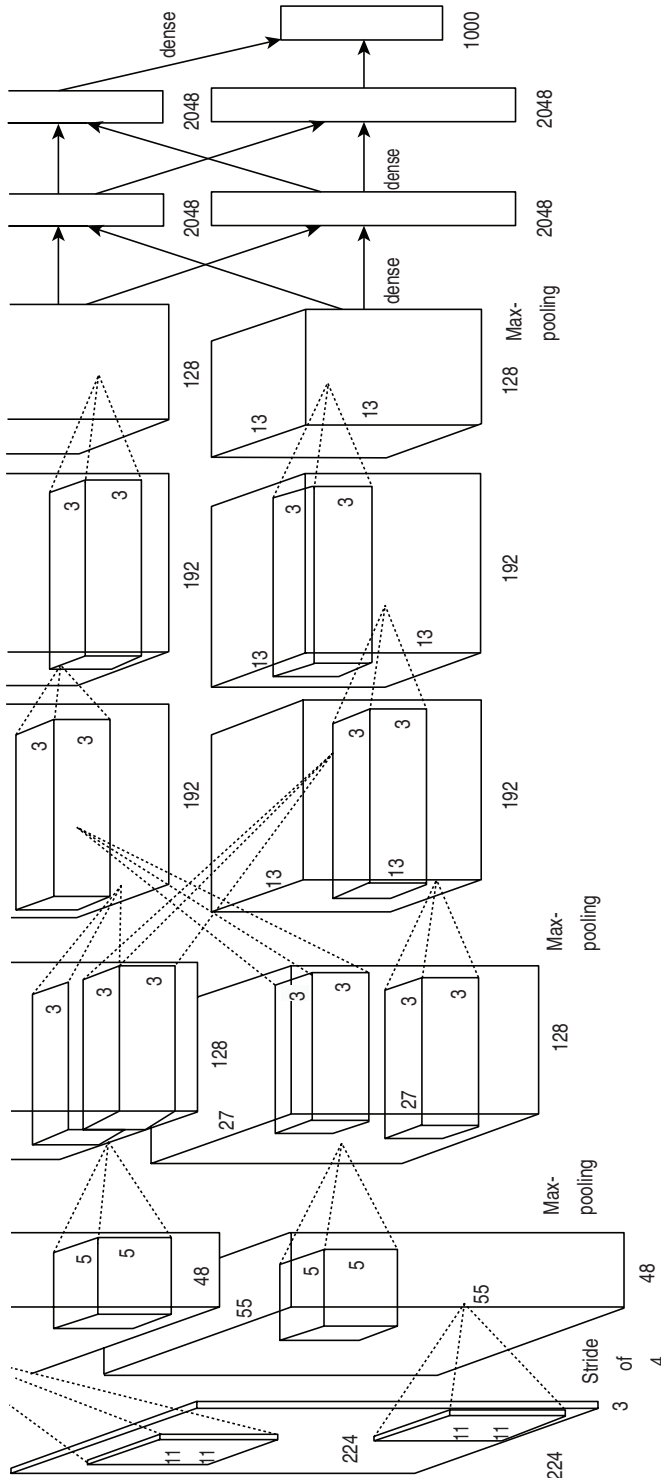


FIGURE 2.15 AlexNet architecture.

SOURCE: Krizhevsky, A., Sutskever, I. and Hinton, G. E. (2012) Imagenet classification with deep convolutional neural networks. *Advances In Neural Information Processing Systems*, 1097–1105.

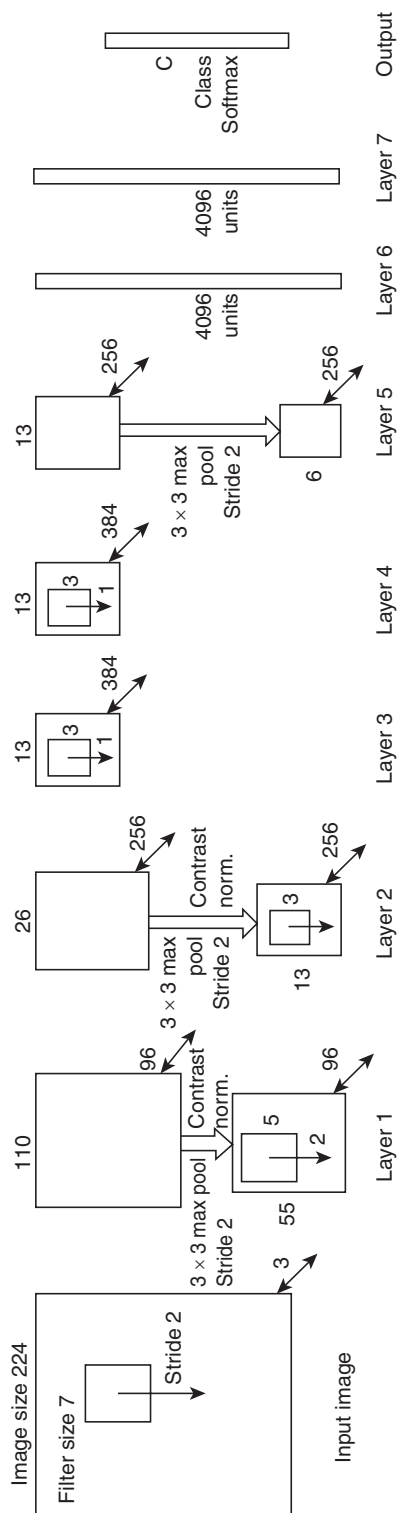


FIGURE 2.16 ZFNet architecture.

SOURCE: Zeiler, M. D. and Fergus, R. (2014) Visualizing and Understanding Convolutional Networks. *European Conference on Computer Vision*, 818–833.

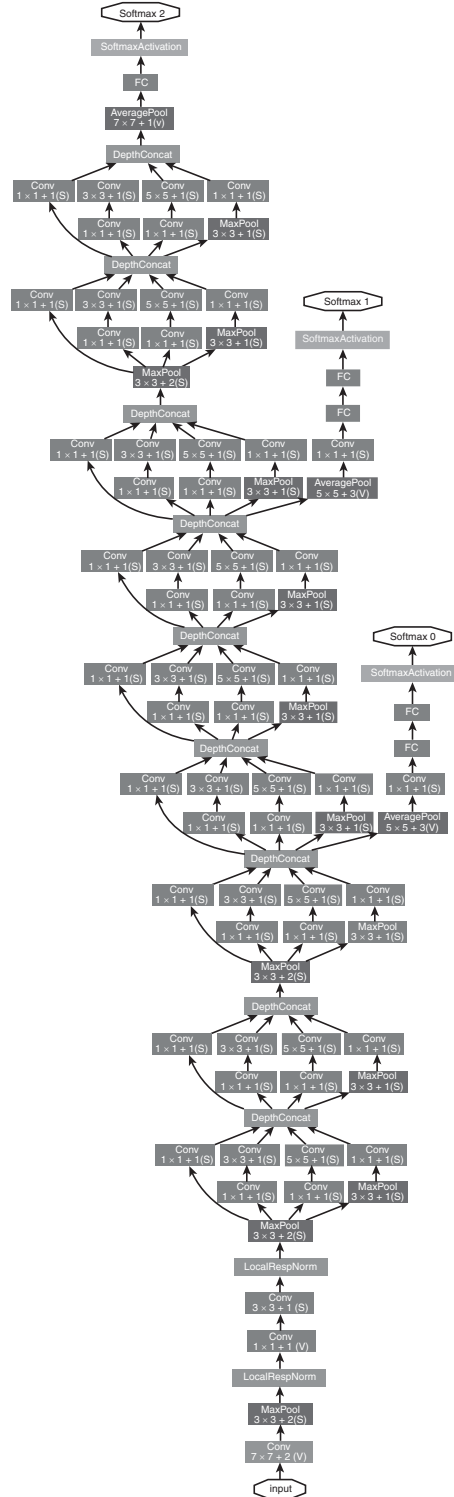


FIGURE 2.17 Architecture of GoogLeNet.

SOURCE: Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V. and Rabinovich, A. (2015) Going deeper with convolutions. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1–9.

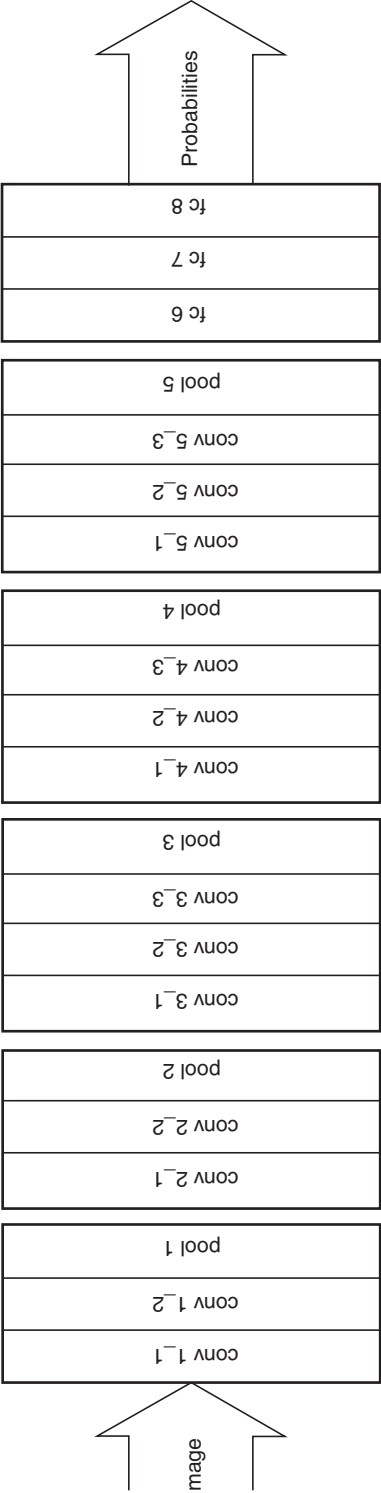


FIGURE 2.18 Architecture of VGGNet.

SOURCE: Simonyan, K. and Zisserman, A. (2015) Very Deep Convolutional Networks for Large-Scale Image Recognition. *International Conference on Learning Representations, International Conference on Learning Representations*.

2.6.6 | ResNet

ResNet was designed by Kaiming He *et al.* It is the 2015 winner of ILSVRC. It introduced ‘identity shortcut connection’ that skipped one or more layers, which were called ‘skip connections’, as shown in Fig. 2.19. The vanishing gradient problem, the main problem of deep networks, was solved by the use of skip connections. ResNet featured heavy batch normalization, and with 152 layers, it reduced the error rate to 3.57% which beat human performance. Residual network boosted the performance of many computer vision applications like object detection, face recognition, etc. Due to its success, many research communities started finding out the reason for its effectiveness and many refinements were made like ResNeXt, DenseNet, etc.

2.6.7 | DenseNet

The idea behind dense convolutional networks may be useful to reference feature maps from earlier in the network. Thus, each layer’s feature map is concatenated to the input of every successive layer within a dense block. This allows later layers within the network to directly leverage the features from earlier layers, encouraging feature reuse within the network, which thereby improves the efficiency. The architecture of DenseNet is shown in Fig. 2.20.

2.7 | APPLICATIONS OF CNN

CNNs are multistage architectures with convolution, pooling, and fully connected layers. They have become an integral part of computer vision which aims at imitating the functionality of the human eye and its brain insights by a machine to understand and process images. Its applications include object detection, action recognition, scene labelling, character or handwriting recognition, etc. Some important applications are discussed in the following sub-sections.

2.7.1 | Object Detection

Object detection is a technology that deals with detecting real-world objects from a given scene. Technically, it detects the instances of a particular class like animals, birds, cars, etc. from the given image. As CNNs are getting deeper, many complex computer vision problems can be solved using these deep CNNs. All applications of CNN are built on top of object detection. Figure 2.21 shows the detection of dogs from an input image.

2.7.2 | Face Recognition

Face recognition is a problem that predicts whether there is a match between the face that is input and those available in the database. The common facial features include eyes, nose, mouth, and chin but sometimes the background of the image is also taken into account. Face recognition is affected by many problems, including the following:

1. Identifying all faces possible.
2. Focus on each face regardless of lighting and perspective.

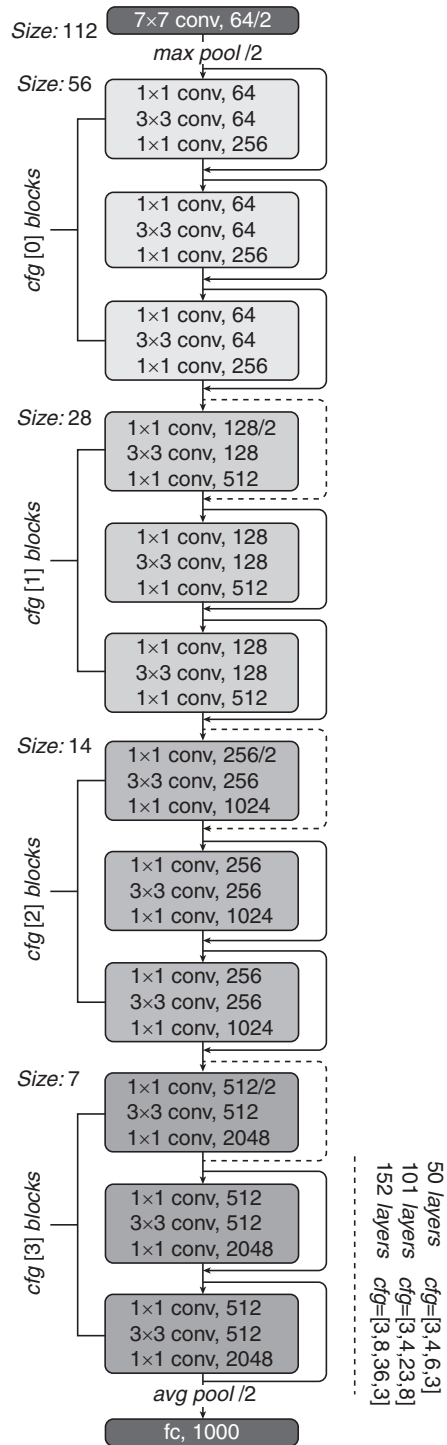


FIGURE 2.19 Architecture of ResNet.

SOURCE: He, K., Zhang, X., Ren, S. and Sun, J. (2015) *Deep Residual Learning for Image Recognition*. [arXiv preprint arXiv:1512.03385].

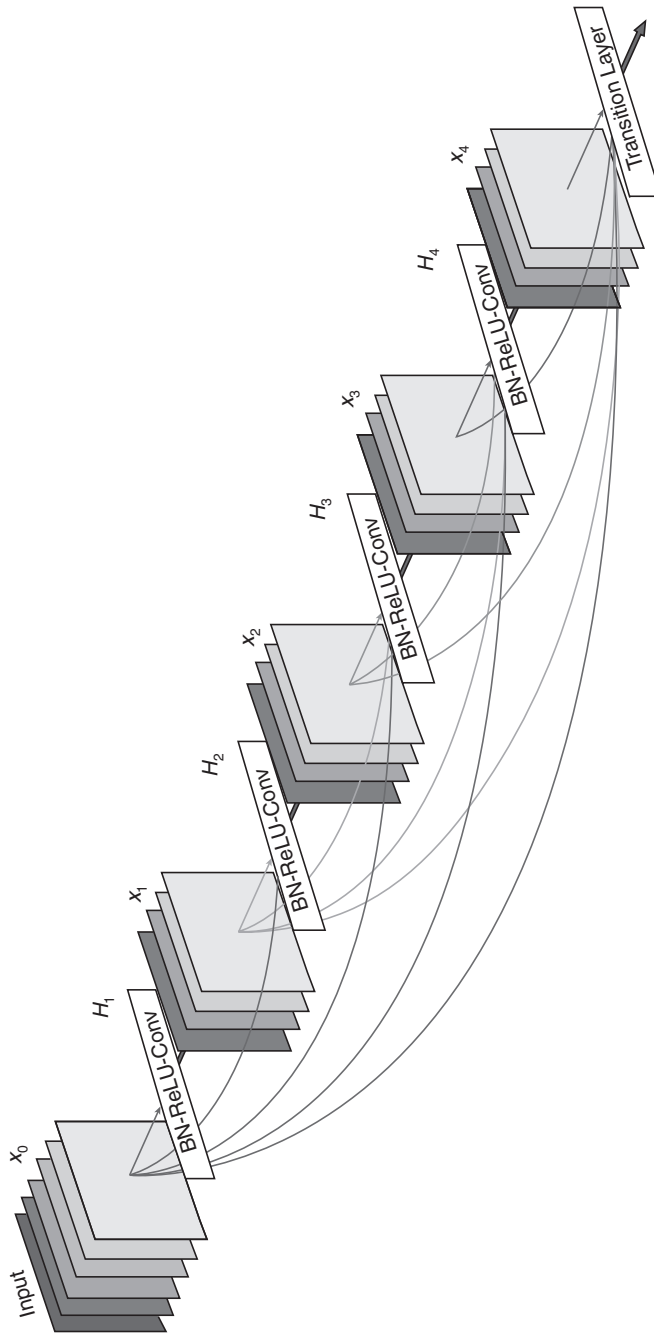


FIGURE 2.20 Architecture of DenseNet.

SOURCE: Huang, G., Liu, Z., van der Maaten, L. and Weinberger, K. Q. (2017) Densely Connected Convolutional Networks. *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition (CVPR)*.

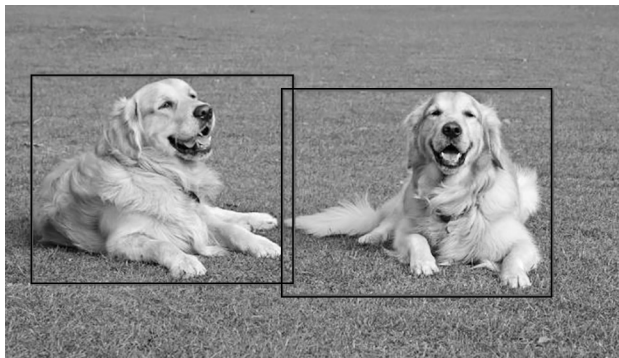


FIGURE 2.21 Object detection.

3. Finding unique features to a face.
4. Comparing identified features with that available in the database.

Usually, face recognition turns out to be an N -way classification problem where it is able to recognize N identities. However, it is also applied for recognizing single or two identities depending on the application. There are many datasets available for face recognition such as Labeled Faces in the Wild (LFW) dataset with 13,233 images of 5,749 identities, and YouTube Faces (YTF) dataset with 3,425 videos of 1,595 identities. An overview of the steps of CNN based face recognition are as follows:

1. Detect and crop faces on the input to feed aligned faces to CNN improving performance.
2. Extract vector representation of faces called embeddings.
3. Compare input vector embeddings to labeled vector embeddings in the dataset using a classifier.

Note that CNN is only used to extract facial features and only a classifier recognizes class labels. The cropped and aligned face of Jacques Chirac from LFW dataset is shown in Fig. 2.22.

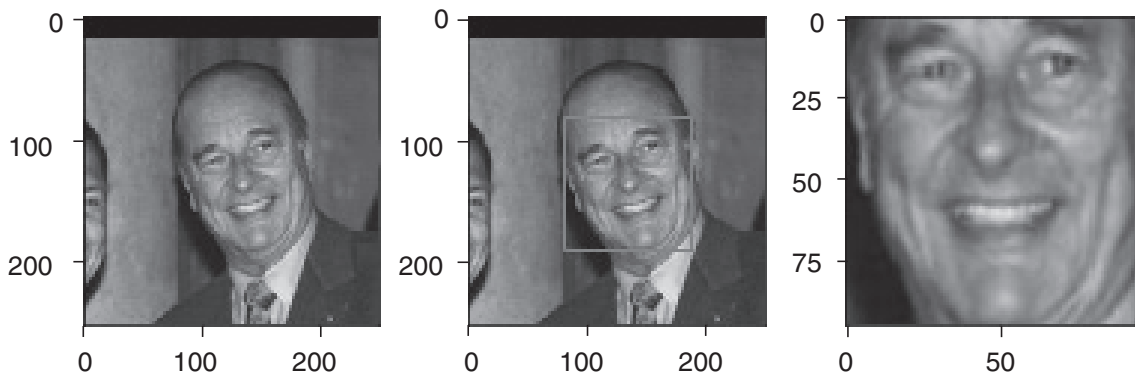


FIGURE 2.22 Cropped and aligned face of Jacques Chirac from LFW dataset.

2.7.3 | Scene Labeling

Scene labeling is the process of labeling every pixel in the given image with the category of the object it belongs. Let us consider the image in Fig. 2.23, which depicts the idea of scene labeling. The task is to take each pixel in the image and categorize it to the object of the class it belongs. Each pixel holds information about the object it is part of.

State-of-the-art systems incorporate CNN with a recurrent architecture. A recurrent system means the network has its networks share the same set of weights. The network is made to self-correct. It learns to smoothen the predicted values. As the size of the system increases, the system becomes more self-correcting.

An algorithm proposed by UCB used R-CNN performed 30% better, but yet was simple and scalable.

Recent developments in deep neural networks prove to be beneficial for machine learning applications. Likewise, deep learning convolution networks have greatly improved the performance of computer systems on problems in image classification. Although the solution is not end-to-end, this promises a better outcome.

The end-to-end, pixels-to-pixels trained CNNs overcome the shortcomings of previous approaches used for semantic segmentation where each pixel was labelled with the class of either the encoding object or region. Fully convolutional networks such as AlexNet, GoogleNet, and VGG Net achieve the state-of-the-art segmentation of PASCAL VOC, NYUDv2, and SIFT Flow datasets (20% relative improvement), where inference takes less than one-fifth of a second for a particular image.

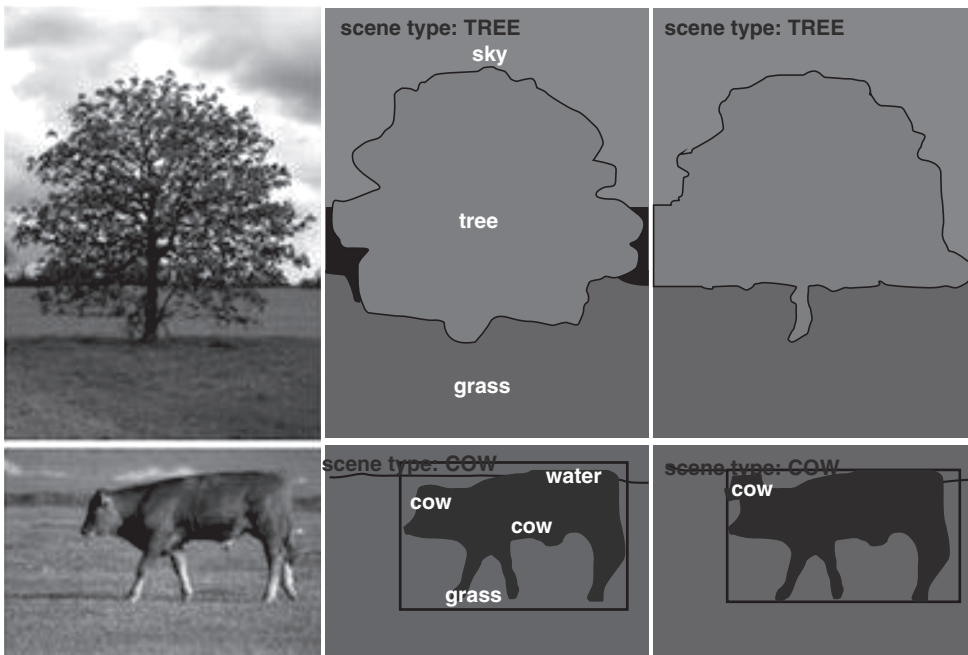


FIGURE 2.23 Scene labelling.

2.7.4 | Optical Character Recognition (OCR)

OCR is one of the domains where CNN gives the best result. Traditional systems rely on lengthy methodologies which need a large amount of training knowledge. But a system that uses multilayer neural networks with CNN can be used to design highly accurate text detectors and character modelers.

A simple system can provide extraction of text from scanned documents. However, we need a system that can recognize text in unconstrained images, where characters can be found anywhere on the image randomly with different formats (e.g., characters may be rotated through different degrees, or have different pixel density, or have different foreground and background color, or no restriction on noise level). For such cases, CNNs have proven to provide higher accuracy than most traditional approaches and other neural networks.

2.7.5 | Handwritten Digit Recognition

Handwritten digit recognition problem is one of the object recognition problems. A popular dataset that is available for this problem is the Modified National Institute of Standards and Technology (MNIST) dataset. It is a modified version of the NIST dataset in the sense that the digits are size-normalized and centered in the image of fixed size. It contains about 60,000 train examples and 10,000 test examples. This problem is like the “Hello World” problem for object detection. As it is a classification problem, regular machine learning classifiers can also be used. However, CNN proves its utmost efficacy in this problem by reducing the error rate to around 0.2%. Being a digit recognition problem, it becomes a 10-class (digits 0–9) classification problem. Figure 2.24 shows an example image of digit 2 available in MNIST dataset.

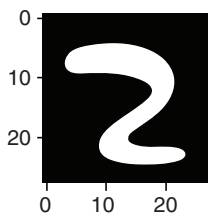


FIGURE 2.24 A sample from MNIST dataset corresponding to digit 2.

SUMMARY

This chapter has introduced the concept of CNN followed by a detailed description of CNN architecture, various components of CNN, unique CNN properties, CNN variants and a few of its infinite number of applications. Section 2.2 discussed in detail about each component (convolutional layer, pooling layer, fully connected layer and the activations) in a CNN architecture. Sections 2.3 and 2.4 discussed the various activation functions that could be used in CNN hierarchy. Section 2.5 discussed the two most important properties of any CNN architecture: weight sharing and translation invariance. The chapter concludes by discussing few real-time applications of CNN such as object detection, handwritten OCR, face recognition, scene labeling. The effectiveness of CNN in

image detection and processing is mentioned repeatedly throughout the chapter with explanation for some famous architecture including LeNet, AlexNet, VGGNet.

REVIEW QUESTIONS

1. What is the purpose of using filters?
2. What does the dimension of receptive field depend on?
3. How do the negative activations influence the gradient?
4. How can a CNN identify an object even when it is transformed?

ASSIGNMENT PROBLEMS

1. Given the input matrix and the kernel, perform convolution with stride being 1 and 2.

1	0	1	1	0
0	0	0	1	1
1	0	0	0	1
0	1	1	1	0
1	1	0	1	0

Input matrix

1	0	0
0	0	1
1	1	0

Kernel

2. Apply max-pooling, average-pooling, and sum-pooling to the results from above convolutions.
3. Visualize the flattened version of the pooled feature maps from Question 2.
4. With weights being randomly chosen and bias being 1, learn the weights and bias over an epoch for the flattened arrays from Question 3.

REFERENCES

1. He K., Zhang X., Ren S., and Sun J. 2015. Deep residual learning for image recognition. [arXiv preprint arXiv:1512.03385]
2. Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations, International Conference on Learning Representations (ICLR)*
3. Matthew D. Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. *European Conference on Computer Vision (ECCV)*, 8689: 818–833
4. Krizhevsky, I. Sutskever, and G. E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 1097–1105
5. Szegedy C., Liu W., Jia Y., Sermanet P., Reed S., Anguelov D., Erhan D., Vanhoucke V., and Rabinovich, A. 2015. Going deeper with convolutions. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1–9
6. Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based Learning Applied to Document Recognition, *Procedure of IEEE*

Recurrent Neural Network: Basic Concepts

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Know the basics of recurrent neural network and its architecture.
- Understand the concept of recurrent neural network.
- Apply recurrent neural network in the real-time applications.

3.1 | INTRODUCTION

Recurrent Neural Networks (RNNs) are a class of neural network architectures that received a lot of focus in research and development areas during the 1990s. Artificial neural networks are described as computational structures built from weighted connections among simple and functionally similar non-linear processing units or nodes, denoted using artificial neurons. A recurrent network is simply an artificial neural network with feedback (closed loop) connections. Examples include bidirectional associative memory, Hopfield Boltzmann machine, and recurrent backpropagation nets. These RNNs are designed to learn sequential or time-varying patterns. One of the reasons for the achievements of deep learning is the implementation of RNN algorithms. A salient feature of RNN is that it can retain information of its input, due to internal memory, and can figure out a much deeper realization of a sequence and its context, compared to other algorithms. Thus, problems that involve sequential data such as text, financial data, time series, speech, audio, video, weather, etc. can be solved using RNNs. They are widely used in the software for Google Translate and Siri of Apple.

Few applications that widely use RNNs are as follows:

1. **Speech Recognition:** Identifies words and phrases spoken and converts them into machine readable format. For example, an audio clip X is taken as input and it is mapped to text transcript Y.
2. **Music Generation:** Here, the input may be the genre of music to be generated and the output is music sequence.
3. **Machine Translation:** Conversion of a sentence from one language to another.
4. **DNA Sequence Analysis:** DNA is represented by the alphabets A, C, G, and T. A DNA sequence is given as input and RNN labels the protein DNA sequence.

5. **Video Activity Recognition:** Identifies activity from the sequence of video frames.
6. **Sentiment Classification:** Classification of text data according to the sentimental polarities of views contained in it.

3.1.1 | RNN versus CNN

A main limitation of convolutional neural networks (CNNs) (and also vanilla neural networks, covered in Chapter 2) is that their API is much constrained, that is, they admit a predetermined-sized vector as input (e.g., an image) and generate a predetermined-sized vector as output (e.g., probabilities of different classes). This input–output mapping is performed using a fixed number of computational steps (e.g., number of layers in the model). The main reason that recurrent nets are more stimulating is that they permit us to function over sequences of vectors (i.e., sequences in the input, the output, or both) (Nicholson and Gibson, 2001).

3.1.2 | Feedforward Neural Network versus RNN

In a feedforward neural network, the sequential data moves from the input layer to the output layer through the hidden layer, that is, the information flows in only one straight direction through the network. Thus, the information never reaches the same node twice.

Feedforward neural networks are inadequate in forecasting the future inputs as they have no memory of the input they received in the past. They consider only the present input and have no conception of order in time. They can retain information only about their training and cannot retain any other past information. In short, feedforward networks have forgetfulness regarding their recent past; they can remember nostalgically only the shaping moments of training.

In an RNN, the information goes through a loop. When it formulates a decision, it takes the current input and the learning from past inputs into consideration. The dissimilarities in the information flow between an RNN and a feedforward neural network are illustrated in Fig. 3.1 (Nicholson and Gibson, 2001).

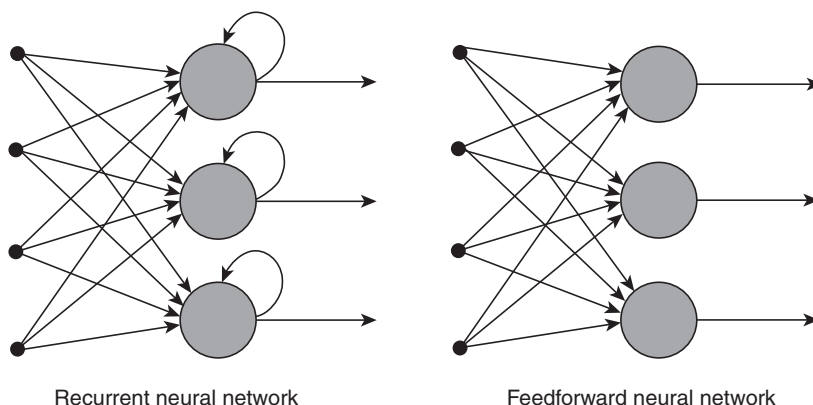


FIGURE 3.1 Recurrent neural network and a feedforward neural network flow.

3.2 | SIMPLE RECURRENT NEURAL NETWORK

A recurrent neural network takes the current input and also the previous observation as input. The Elman network was introduced by Elman in 1990. The Elman network has connections from the hidden layer to a context layer and also has connections from input to the hidden layer, as shown in Fig. 3.2. The role of the context units is to provide dynamic memory so as to encode the information contained in the sequence of phonemes necessary for prediction. It stores the past output.

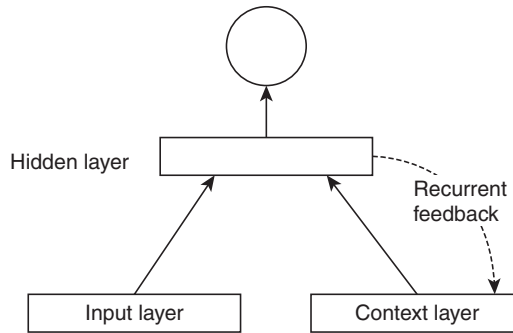


FIGURE 3.2 Simple recurrent net proposed by Elman.

The decision a simple recurrent net takes at t th time instant is affected by the decision taken at $(t - 1)$ th instant. So, recurrent networks have two input sources – the present and the recent past – which jointly determine how they respond to new data.

Recurrent networks are differentiated from feedforward networks by the feedback loop which is connected to their past decisions, receiving their own outputs moment after the moment as input has been repeatedly said that recurrent networks have the memory which takes information in the sequence, and recurrent nets utilize it to perform functions that cannot be performed in feedforward networks.

This sequential information is maintained in the recurrent network's hidden state, which handles to span many time steps as it cascades forward to influence the processing of each new example. It finds correlations among events parted by several moments, and these correlations are called *long-term dependencies*, because an event downstream in time is dependent on, and is a function of, one or more events that occurred before. RNN is a technique to consider that shares weights over time.

The process of transmitting memory forward is described mathematically as

$$h_t = \phi(Wx_t + Uh_{t-1}) \quad (3.1)$$

Preferably,

$$h_t = \tanh(Wx_t + Uh_{t-1}) \quad (3.2)$$

At a time step t , the hidden state is represented as h_t . It is a non-linear mathematical function of the input X_t , multiplied by a weight matrix W (like the one that was used for feedforward nets) added to the hidden state of the past time step h_{t-1} multiplied by its own hidden-state-to-hidden-state matrix U . The matrix U is also known as *transition matrix*. The choice of tanh over other non-linearities has to do with its second derivative decaying very gradually to zero. This helps to resolve the vanishing gradient problem as it keeps the gradients in the linear region of activation function.

The output vector y_t at time t is the product of the weight matrix V and the hidden state h_t , with softmax, a function that takes as input a vector of K real numbers, and normalizes it into a probability distribution consisting of K probabilities applied to the product so that the resulting vector is a set of output probabilities:

$$\hat{y}_t = \text{softmax}(Vh_t)$$

An RNN can be seen as the repetition of a single cell. The computation for a single time step of an RNN cell is shown in Fig. 3.3.

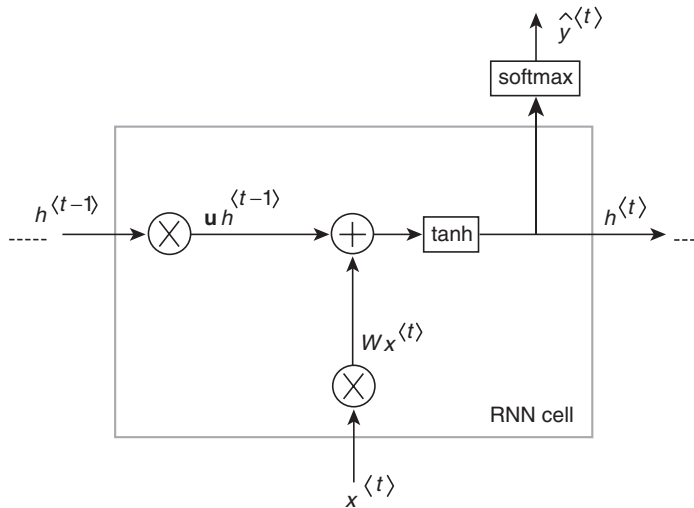


FIGURE 3.3 Single time step of an RNN cell.

To implement a single RNN cell, the following pseudo-code is used:

1. Compute the hidden state h_t with tanh activation.
2. Using your new hidden state h_t compute the prediction $\hat{y}_t$.
3. Store h_t , h_{t-1} , $x(t)$ and parameters.
4. Return h_t and $\hat{y}_t$.

After implementation of a single RNN cell, a basic RNN network is built as shown in Fig. 3.4.

We can also represent the RNN cell graphically as shown in Fig. 3.5 (left). At time t , the cell has an input x_t , and an output $\hat{y}_t$. Fraction of the output $\hat{y}_t$ (the hidden state h_t) is fed back into the cell for use at a later time step at $t + 1$. The parameters of RNNs are given by three weighted matrices just as a traditional neural network's parameters are contained in its weight matrix. These matrices are U , V , and W , corresponding to the input, output, and hidden state, respectively.

Another way to look at an RNN is to unroll it, as shown in Fig. 3.5 (right). By unrolling we mean that we draw the network for the complete sequence. The network shown here is a three-layer RNN suitable for processing three-element sequences. Note that the weight matrices U , V , and W are shared across the steps. This is because we are applying the same operation on different inputs at each time step. Being able to share these weight vectors across all the time steps greatly reduces the number of parameters that the RNN needs to learn.

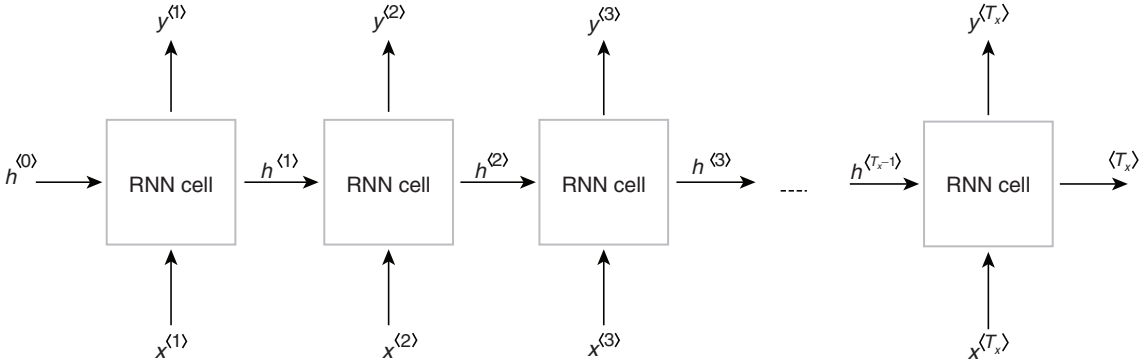


FIGURE 3.4 Basic RNN model.

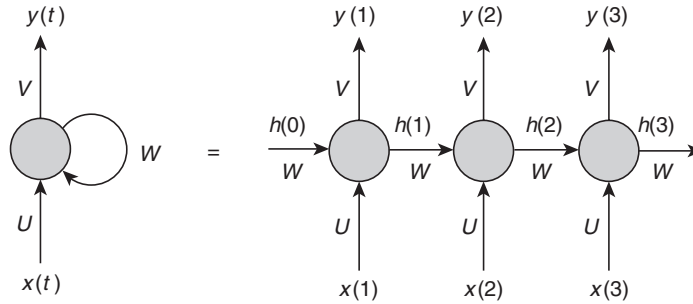


FIGURE 3.5 Unrolled RNN.

3.2.1 | Training an RNN

Training an RNN is comparable to training a CNN. The backpropagation algorithm is utilized, but with a slight twist. The gradient at all output relies not only on the computations of the current time step but also on the previous time steps as the parameters in the network are shared across all time instances.

For example, in order to determine the gradient at $t = 6$, we would need to backpropagate five steps and sum up the gradients. This is called *backpropagation through time* (BPTT). Thus, RNNs trained with BPTT algorithm have difficulties in learning long-term dependencies (i.e., dependencies among steps that are far apart). This complication arises because of vanishing/exploding gradient problems. These problems are dealt by a few technologies, and certain types of RNNs [like long short-term memory (LSTM)] were precisely designed to get around them.

In a simple (fully connected) neural network, backpropagation is used to compute the derivatives with respect to the cost function J to update the parameters W , U . Similarly, in RNN the derivatives are calculated with respect to the cost in order to update the parameters (i.e., the derivatives of the cost function J backpropagate through the RNN by following the chain rule from calculus to update the weights W , U).

3.2.2 | Backpropagation Through Time (BPTT) Illustration

The implementation steps of BPTT are as follows:

STEP 1: Generate input and output data.

STEP 2: Normalize the data with respect to maximum and minimum values.

STEP 3: Assume the number of hidden layers and number of neurons in the hidden layer.

STEP 4: Initialize the weights between 0 and 1. Let $[v]$ be the weights connecting input neurons and hidden neurons, and $[w]$ be the weights connecting hidden and output neurons in the case of a single hidden layer.

STEP 5: Compute the input and output at every neuron present in each layer. Let the last hidden layer compute the output using sigmoidal activation function given by.

$$O^h = \frac{1}{1 + e^{-I^h}}$$

STEP 6: Find the error and check the tolerance. If the error is above tolerance, update the weights.

STEP 7: Repeat from Step 4 till the error is within tolerance.

Consider a training example with two inputs $x_1 = 0.4$, $x_2 = -0.7$ and output = 0.1. Since the input and output data is within -1 and $+1$, normalization need not be carried out.

Considering a neural network with one input, one hidden and one output layer, let the weights be initialized as

$$[v]^0 = \begin{bmatrix} 0.1 & 0.3 \\ -0.2 & 0.1 \end{bmatrix}$$

$$[w]^0 = \begin{bmatrix} 0.2 \\ -0.4 \end{bmatrix}$$

Computing the input to hidden layer, we get

$$I^h = [v]^T \{O\}^i = \begin{bmatrix} 0.1 & -0.2 \\ 0.3 & 0.1 \end{bmatrix} \begin{bmatrix} 0.4 \\ -0.7 \end{bmatrix} = \begin{bmatrix} 0.18 \\ 0.05 \end{bmatrix}$$

Computing the output of the hidden layer using sigmoid function, we get

$$O^h = \begin{bmatrix} 0.5448 \\ 0.5125 \end{bmatrix}$$

Similarly, calculating input and output of the output layers, we get

$$I^o = -0.09604 \text{ and } O^o = 0.476$$

The actual output of the training example is 0.1. Thus,

$$\text{Error} = (0.1 - 0.476)^2 = 0.141376$$

To adjust the weights, we need partial derivatives of error:

$$d = (\text{True output} - O^0)(O^0)(1 - O^0) = -0.09378$$

To update the weights,

$$[\Delta w]^{t+1} = \alpha [\Delta w]^t + \phi[y]$$

where

$$[y] = O^h \times d$$

Let us assume momentum term, $\alpha = 0.5$ and learning rate, $\phi = 0.6$. Then substituting the values, we get

$$[\Delta w]^1 = \begin{bmatrix} -0.03065 \\ -0.02884 \end{bmatrix}$$

The updated weights are

$$[w]^1 = [w]^0 + [\Delta w]^1 = \begin{bmatrix} -0.16935 \\ -0.42884 \end{bmatrix}$$

Similarly, update the weights of $[v]$. The iterations are carried out until error comes under tolerance range. Once these iterations are over, the network can predict test data.

3.2.3 | RNN Topology

RNNs do not have the limitation of performing the transformation from input to output in a constant number of steps given by the constant number of layers in the model. Sequences in the input, the output, or both are possible. This means that RNNs can be organized in various ways to resolve specific problems. This property of being able to work with sequences gives rise to a number of common topologies, some of which are shown in Fig. 3.6.

Consider each rectangle as a vector and arrows as functions (e.g., matrix multiply). Let us study the RNN topologies, from left to right:

1. **One-to-One** represents vanilla neural network (CNN) of processing without recurrent nets, from constant-sized input to constant-sized output (e.g., image classification).
2. **One-to-Many** represents a sequence output (e.g., image captioning acquires an image as input and outputs a sentence of words).
3. **Many-to-One** denotes sequence input (e.g., sentiment analysis where a known sentence is classified as stating positive or negative sentiment).
4. **Many-to-Many for Sequence Input and Sequence Output** (e.g., language translation: an RNN examines a sentence in English and then outputs it in French).
5. **Many-to-Many for Synced Sequence Input and Output** (e.g., video classification where we want to label every frame). It should be noted that there are no specific constraints on the lengths of sequences because the recurrent transformation (green) is not fixed and can be applied as many times as we want.

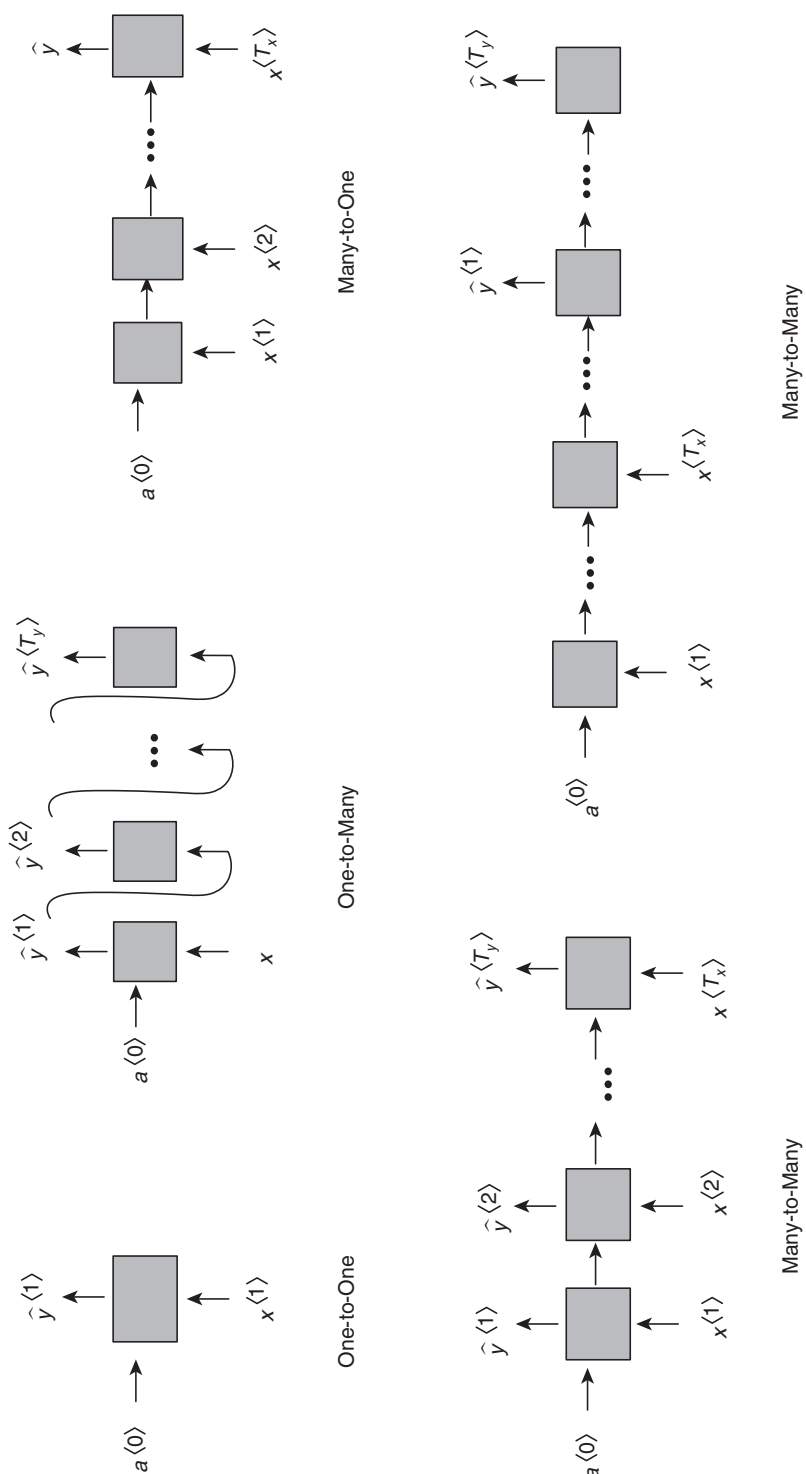


FIGURE 3.6 RNN topologies.

3.2.4 | Challenges with Vanishing Gradients

By the early 1990s, the *vanishing gradient problem* appeared as a major difficulty in recurrent net performance. Just as a straight line conveys a change in x alongside a change in y , the gradient conveys the change in all weights with respect to the change in error. If the gradient is unknown, weights cannot be tuned in a direction that will decrease error, and the network stops to learn.

Recurrent nets that are looking to establish connections between a final output and events many time steps before were shambled because of the complexity involved to know how much significance to accord to remote inputs. This is partly because the information flowing via the neural nets passes through several stages of multiplication.

For example, let us consider an example from language modelling. Note the two sentences: ‘The **dog** that ate a bunch of apples, pears, carrots and bananas **was** full’ and ‘The **dogs** that ate a bunch of apples, pears, carrots and bananas **were** full’. This is an example of when language can have very long-term dependencies, but basic RNN is not very good at capturing very long-term dependencies because of vanishing gradient problems.

People who have learnt compound interest know that any quantity multiplied recurrently by a value slightly greater than 1 can become infinitely large (indeed, this simple mathematics emphasizes inevitable social inequalities and network effects). Its inverse, that is, multiplying by a value less than 1 will consequently vanish, is also true. For example, gamblers lose all their money when they win just 97 paise on each rupee they put in a game.

Because the time steps and layers of deep neural networks communicate to each other via multiplication, derivatives may vanish or explode in most scenarios.

The gradients may also increase exponentially with the number of hidden layers; this is called *exploding gradient problem*. It turns out that vanishing gradients tends to be a bigger problem with training RNNs, although when exploding gradients happens, it leads to exponentially large gradients which makes the neural network parameters get really large or NaN.

To solve these issues, exploding gradient problem can be trimmed or squashed and are thus relatively easy to solve. Similarly, vanishing gradients can evolve too tiny for computers to work with or for networks to learn. Thus, it is tedious to solve vanishing gradients problems.

Figure 3.7 is helpful in observing the effects of applying a sigmoidal function over and over again. The data is levelled unless, for large stretches, the slope is not detectable. This resembles gradient vanishing as it passes through many layers.

Just as in traditional neural networks, training the RNN involves backpropagation as well. The only difference is that the parameters are shared across all time steps. The gradient at every output depends not only on the present time step, but also on the previous ones. As already mentioned in Section 3.2.1, this procedure is called backpropagation through time, or BPTT in short.

Consider the small three-layered RNN shown in Fig. 3.8. During the forward propagation (shown in solid lines), the network creates predictions that are compared to the labels to calculate a loss L_t at each time step. During backpropagation (shown in dotted lines), the gradients of the loss with regard to the parameters U , V , and W are calculated at every time step and the parameters are updated with the summation of the gradients.

Equation (3.3) shows the gradient of the loss with respect to W , the matrix that encodes weights for long-term dependencies. This part of the update has to be focused because it is the reason for the

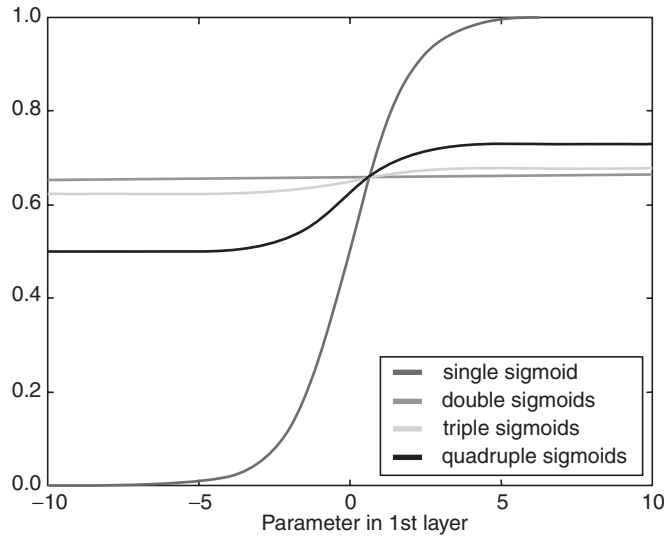


FIGURE 3.7 Sigmoid function applied multiple times.

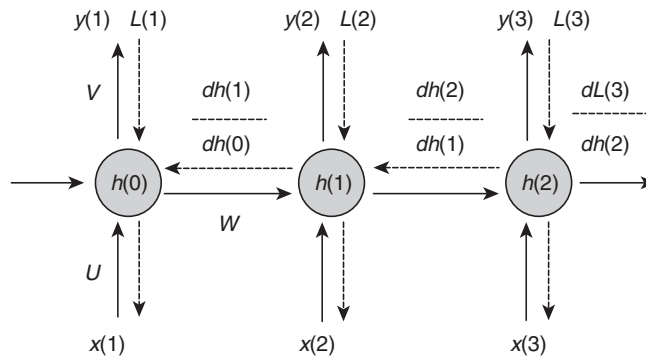


FIGURE 3.8 Three-layered RNN.

vanishing and exploding gradient problems. The other two gradients of loss with respect to matrices U and V are also summed up across all time steps in a similar way:

$$\frac{\partial L}{\partial W} = \sum_t \frac{\partial L_w}{\partial t} \quad (3.3)$$

Let us now look at what happens to the gradient of loss at the last time step ($t = 3$). As seen, this gradient can be decomposed to a product of three subgradients using the chain rule. The gradient of the hidden state h_2 with regards to W can still be decomposed as the summation of the gradient of every hidden state with regard to the previous one. Finally, each gradient of the hidden state with regard to the previous one can be further decomposed as the product of gradients of the present hidden state against the previous one.

$$\begin{aligned}
\frac{\partial L_3}{\partial W} &= \frac{\partial L_3}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial W} \\
&= \sum_{t=0}^2 \frac{\partial L_3}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_t} \cdot \frac{\partial h_t}{\partial W} \\
&= \sum_{t=0}^2 \frac{\partial L_3}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_2} \cdot \left(\prod_{j=t+1}^2 \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_t}{\partial W}
\end{aligned} \tag{3.4}$$

Similar calculations are made to compute the gradient of losses L_1 and L_2 (at time steps 1 and 2, respectively) with regard to W and to add them into the gradient update for W . The last form of the gradient given by Eq. (3.4) tells us why RNNs have the drawback of vanishing and exploding gradients. Consider the case where the individual gradient of a hidden state with regard to the previous one is less than one. As we backpropagate across several time steps, the product of gradients gets smaller, causing the problem of vanishing gradients. Likewise, if the gradients are greater than one, the products get larger, causing the problem of exploding gradients.

The consequence of vanishing gradients is that the gradients from steps that are at a distant do not contribute anything to the learning process. Thus, the RNN does not learn long-range dependencies. Vanishing gradients might happen for traditional neural networks as well; it is just more visible in case of RNNs, as they tend to have many more layers (time steps) over which backpropagation occurs.

Exploding gradients are more effortlessly detectable; the gradients will become very big and then turn into NaN and the training process will collapse. Exploding gradients can be controlled by cutting them at a predefined threshold. While there are a few approaches to diminish the problem of vanishing gradients, such as using ReLU instead of tanh layers, proper initialization of W matrix, and pre-training the layers using unsupervised methods, the most accepted solution is the use of LSTM or gated recurrent unit (GRU) architectures. These architectures have been designed to treat the vanishing gradient problem and to learn long-term dependencies more effectively. These architectures are discussed in subsequent sections in this chapter.

3.2.5 | Bidirectional and Stateful RNNs

At time step t , the output of the RNN is dependent on the outputs of all previous $t - n$ time steps where $n = 1, 2, \dots, n - 1$. However, it is also possible that the output is reliant on future outputs as well. This is very true for applications related to Neuro-Linguistic Programming, where the attributes of the word or phrase we are trying to predict may be dependent on the context given by the entire enclosing sentence, and not only the words that come before it.

Bidirectional RNNs also help the network architecture to provide equal emphasis on the start and end of the sequence and increase the data accessible for training. Bidirectional RNNs are nothing but two RNNs stacked on top of each other, which interpret the input in opposite directions (Fig. 3.9). For example, one RNN will interpret the words right to left and the other will interpret the words left to right. The hidden state of both RNNs will decide the output at every time step.

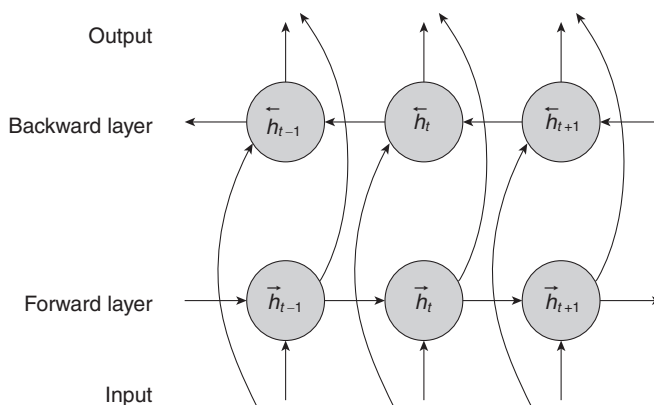


FIGURE 3.9 Bidirectional RNN.

RNNs may be stateful. This means that they can maintain state across various batches while training. The hidden state calculated for a batch of training data shall be used as the initial hidden state for the subsequent batch of training data. However, this has to be explicitly set, as RNNs are stateless by default and resets the state after every batch.

Making an RNN to be stateful means that it can assemble a state across its training sequence and even preserve that state while doing predictions. The benefits of utilizing stateful RNNs are lower training time and/or smaller network sizes. The disadvantage of stateful RNNs is that network has to be trained with a batch size that replicates the periodicity of the data, and the state is reset after each epoch. In addition, data must not be jumbled up while training the network, since the order in which the data is offered is relevant for stateful networks.

3.2.6 | Long Short-Term Memory (LSTM)

A variation of the recurrent net with LSTM units was projected by German researchers Sepp Hochreiter and Juergen Schmidhuber in the mid-1990s. LSTM serves as a remedy to the vanishing gradient problem.

LSTM helps to store up the error that can be backpropagated through time and layers. By preserving a steady constant error, recurrent nets are allowed to continue to learn over several time steps (over 1000), thereby opening a channel to connect causes and effects remotely. This is one of the vital challenges to artificial intelligence and machine learning since algorithms are often confronted by environments where reward signals are sparse and delayed, such as life itself. (Religious thinkers have handled this equivalent problem with ideas of karma or divine reward, theorizing invisible and distant cost to our actions).

LSTMs hold information in a gated cell which is outside the normal flow of the recurrent network. Information can be accumulated in, written to, or read from a cell, much like data in a computer's memory. The cell makes its decisions on what to accumulate, and when to allow reads, writes and deletes, through gates that open and close. Generally, storage in computers is in digital mode. However, these gates are analog in nature, executed with element-wise multiplication by sigmoids that are all in the range between 0 and 1. The advantage of being analog is that it is differentiable, and therefore suitable for backpropagation.

The gates act upon the signals they receive, and like neural network's nodes, they block or pass on information based on its strength and import, which are filtered with their own sets of weights. Those weights, like the weights that modulate input and hidden states, are altered via the recurrent network's learning process. That is, the cells learn when to permit the data to leave, enter or be deleted via the iterative process of assuming guesses, backpropagating errors, and altering the weights by gradient descent.

Figure 3.10 demonstrates how data flows through a memory cell and is controlled by means of its gates.

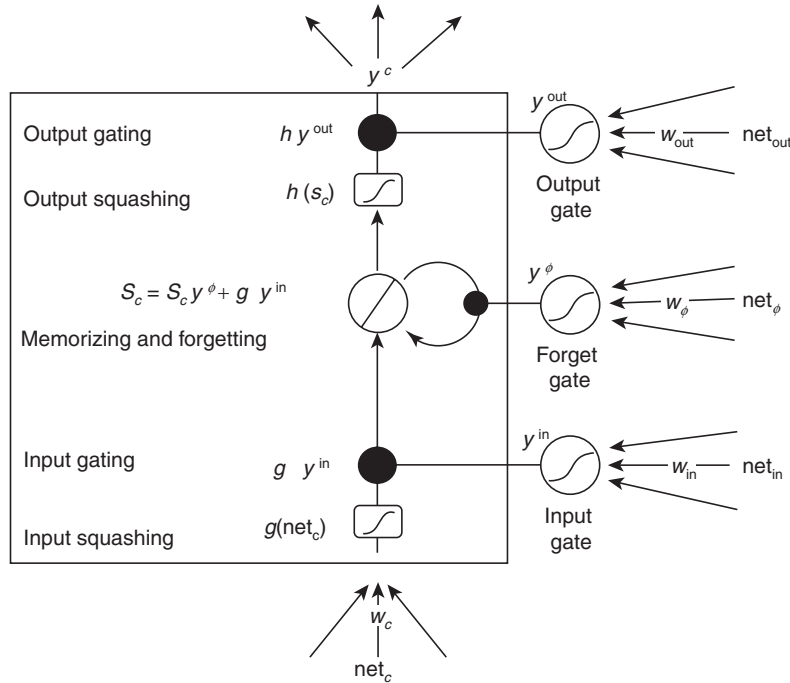


FIGURE 3.10 Data flow through a memory cell.

In Fig. 3.10, the three arrows at the bottom indicate where information flows into the cell at multiple points. That arrangement of present input and past cell state is fed not only to the cell itself, but also to all the three gates, which will decide on handling inputs. The black dots represent the gates themselves. These analog gates decide whether to permit new input in, remove the present cell state, and/or let that state impact the network's output at the current time step. S_c denotes the current state of the memory cell, and $g y_\text{in}$ denotes the current input to it. It should be noted that a gate might be shut or open, and memory will recombine their shut and open states at every step. The cell can fail to remember its state or not; be written to or not; and be read from or not, at each time step. These flows are shown in Fig. 3.11.

Figure 3.11 compares a simple recurrent network [Fig. 3.11 (a)] to an LSTM cell [Fig. 3.11(b)]. The lines coming to summing points can be ignored for now.

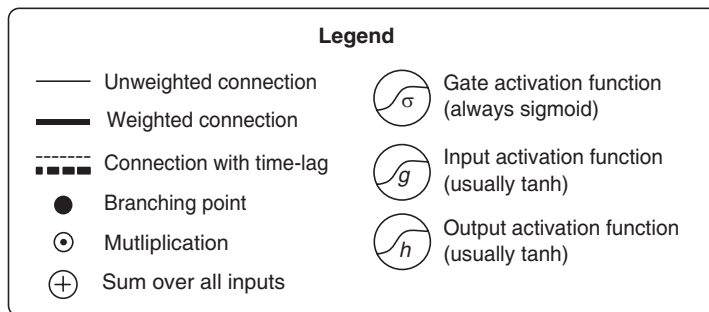
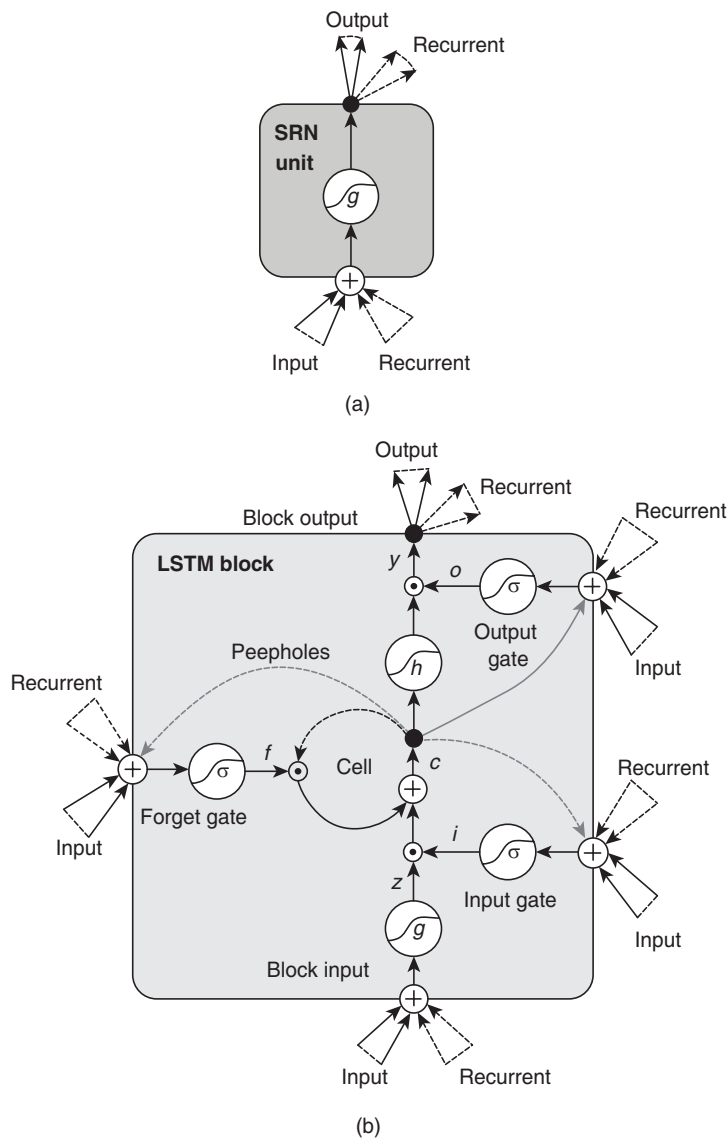


FIGURE 3.11 Detailed schematic of (a) simple RNN and (b) LSTM.

Moreover, while we are on the topic of simple hacks, including a bias of 1 to the forget gate of each LSTM cell is also shown to improvise performance.

In general, feedforward networks map one input to one output. On the other hand, recurrent nets map one-to-many, as mentioned in Fig. 3.12 (one image to many words in a caption), many-to-many (language translation), or many-to-one (segregating a voice).

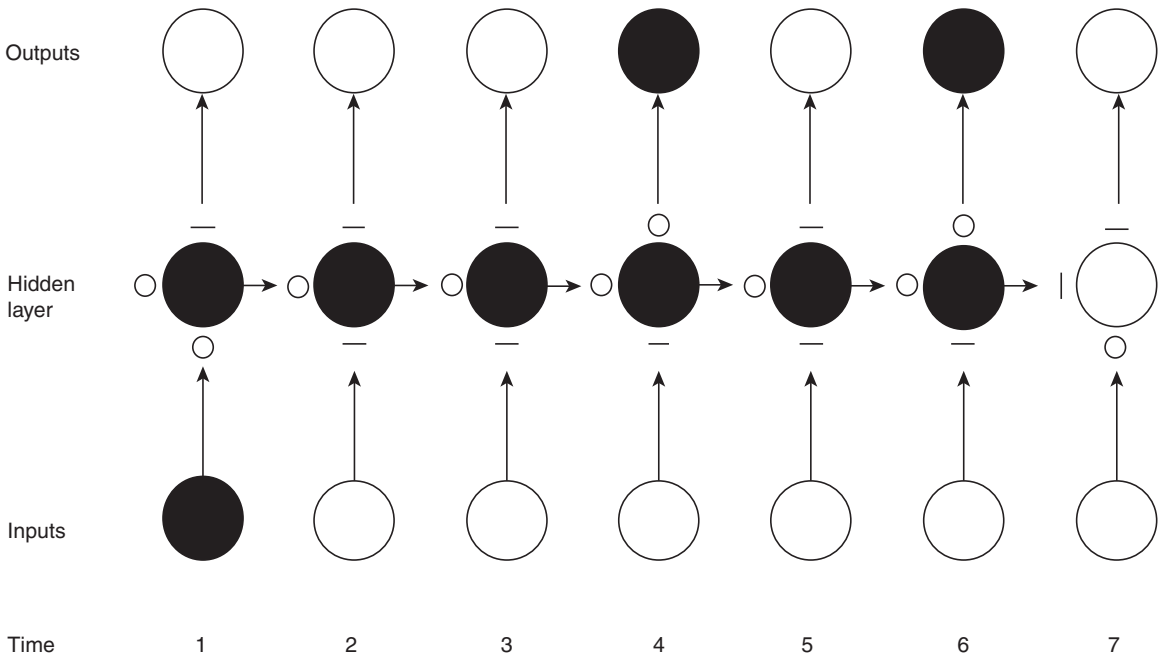


FIGURE 3.12 Input, output and forget gates.

A simple RNN utilizes the hidden state from the previous time step and the present input in a tanh layer to execute recurrence. Similar to simple RNN, LSTMs also execute recurrence. But instead of a single tanh layer, there are four layers acting together in a particular way. The transformations that are applied on the hidden state at time step t are illustrated in Fig. 3.13.

The line across the top of the figure is the cell state c ; it signifies the internal memory of the unit. The line across the bottom is the hidden state, and the i , f , o , and g gates are the means by which the

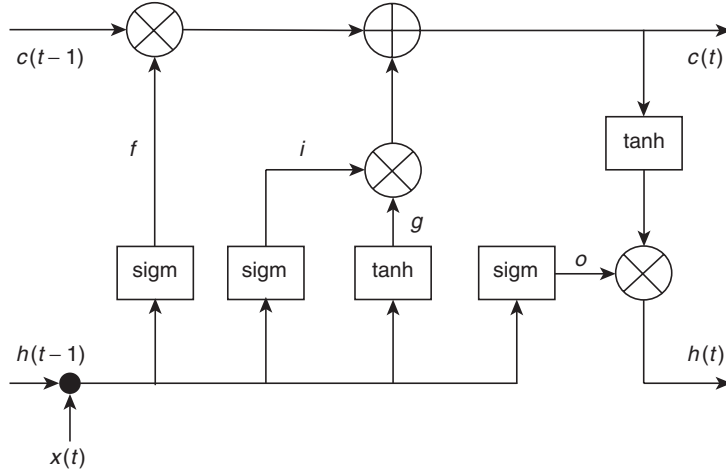


FIGURE 3.13 Hidden state transformations.

LSTM works on the vanishing gradient problem. During training, the LSTM trains the parameters for these gates.

In order to achieve a deeper knowledge of how these gates adjust the LSTM's hidden state, let us consider Eqs. (3.5)–(3.10) that show how state h_{t-1} at the previous time step is utilized to calculate the hidden state h_t at time t :

$$i = \sigma(W_i h_{t-1} + U_i x_t) \quad (3.5)$$

$$f = \sigma(W_f h_{t-1} + U_f x_t) \quad (3.6)$$

$$o = \sigma(W_o h_{t-1} + U_o x_t) \quad (3.7)$$

$$g = \tanh(W_g h_{t-1} + U_g x_t) \quad (3.8)$$

$$c_t = (c_{t-1} \circ f) + (g * i) \quad (3.9)$$

$$h_t = \tanh(c_t) \circ o \quad (3.10)$$

Here i , o and f are the input, output, and forget gates, respectively. They are calculated by the same equations but with different parameter matrices. The sigmoid function adjusts the output of these gates between 0 and 1, so the output vector generated can be multiplied element-wise with another vector to identify how much of the second vector can pass via the first one.

The forget gate describes how much of the previous state h_{t-1} you want to allow to pass through. The input gate describes how much of the newly calculated state for the present input x_t you wish to let through. The output gate describes how much of the internal state you wish to render to the next layer.

The present input x_t and the previously hidden state h_{t-1} is utilized to calculate the internally hidden state g . Note that the equation for g is similar to that for the simple RNN cell, but in this case, we will adjust the output by the output of the input gate i .

Given i , f , o and g , the cell state c_t is computed at time t in terms of c_{t-1} at time $(t-1)$ multiplied by the forget gate and the state g multiplied by the input gate i . So, this is basically a method to merge

the previous memory and the new input; resting the forget gate to 0 disregards the old memory and resting the input gate to 0 disregards the newly computed state. Finally, the hidden state h_t at time t is calculated by multiplying the memory c_t with the output gate.

3.3 | LSTM IMPLEMENTATION

The implementation steps of LSTM are as follows:

STEP 1: Implement a single LSTM cell by concatenating h_{t-1} and x_t matrices and by computing Eqs. (3.5)–(3.10) given above.

STEP 2: Compute the prediction at single time step $y(t)$ using softmax function.

STEP 3: Iterate this single LSTM cell over and over using a for-loop to process a sequence of T_x inputs.

We need to note that an LSTM is a drop-in substitute for a simple RNN cell, the only difference being LSTMs are resistant to the vanishing gradient problem. An RNN cell in a network can be replaced with an LSTM without being concerned about any side effects. Better results can be seen along with longer training times.

3.4 | GATED RECURRENT UNIT (GRU)

The GRU is a variant of the LSTM and was put forth by K. Cho. It preserves the LSTM's resistance to the vanishing gradient problem, but its internal structure is simpler. Thus, it can be trained more rapidly, since fewer calculations are required to make updates to its hidden state. Figure 3.14 represents the gates for a GRU cell.

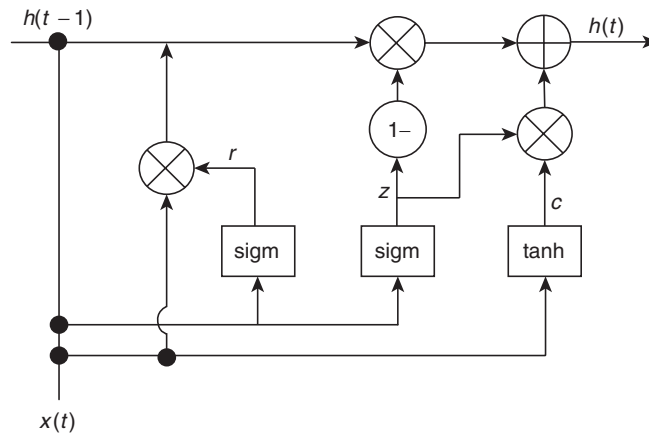


FIGURE 3.14 Gates for a gated recurrent unit (GRU) cell.

It is known that LSTM has three gates: input, forget, and output gates. The GRU cell has two gates: a reset gate r and an update gate z . The update gate describes how much previous memory to maintain around and the reset gate describes how to merge the new input with the previous memory.

GRU has no persistent cell state different from the hidden state as in LSTM. The gating mechanism in GRU is described by Eqs. (3.11)–(3.14).

$$z = \sigma(W_z h_{t-1} + U_z x_t) \quad (3.11)$$

$$r = \sigma(W_r h_{t-1} + U_r x_t) \quad (3.12)$$

$$c = \tanh[W_c (h_{t-1} \circ r) + U_c x_t] \quad (3.13)$$

$$h_t = (z \circ c) + [(1 - z) \circ h_{t-1}] \quad (3.14)$$

GRU and LSTM have equivalent performance and there is no easy way to recommend one or the other for a specific task. While GRUs are quicker to train and need fewer data to generalize, in situations where there is enough data, an LSTM's better expressive power might lead to enhanced results. Like LSTMs, GRUs are drop-in alternatives for the simple RNN cell.

The general idea of utilizing a gating mechanism to learn long-term dependencies is almost the same as in a LSTM, but a few key differences are as follows:

1. GRU has two gates (reset gate r and update gate z), while LSTM has three gates (input, forget, and output gates).
2. GRUs do not have an internal memory (c_t) which is different from the exposed hidden state. They do not include the output gate that is present in LSTMs.
3. An update gate z is the combination of input and forget gates and the reset gate r is applied directly to the previous hidden state. Thus, the responsibility of the reset gate in GRU is really split up into both r and z . A second non-linearity is not applied for output computation.

Thus, the vanishing gradient problem can be cracked either by using LSTM or GRU. GRUs are quite recent (2014), and their effects have not been fully investigated yet. There is no clear winner between GRU and LSTM. In many situations, both the architectures yield comparable performances. Tuning hyperparameters like layer size is probably of much importance than picking an ideal architecture. Since GRUs have less parameters compared to LSTMs (U and W are smaller), they may train a bit quicker or need lesser data to generalize. On the other hand, if you have enough data, the superior expressive power of LSTMs may lead to improved results.

3.5 | DEEP RECURRENT NEURAL NETWORK

Deep artificial neural networks are a group of algorithms that solve several important problems with much higher accuracies such as sound recognition, image recognition, recommender systems, etc. *Deep* is a technical word referring to the quantity of hidden layers in a neural network. A shallow network has only one hidden layer, while a deep recurrent neural network has more than one.

Depth is an important parameter that differentiates deep learning networks from the more common single-hidden-layer neural networks. Depth refers to the number of node layers via which data travels in a multistep process of pattern recognition. Former versions of neural networks such as the first perceptrons were shallow, comprising single input and single output layer, and with a maximum of one hidden layer in between. Any network that has greater than

three layers (including input and output) qualifies as ‘deep’ learning. So, the technical term ‘deep’ refers to more than one hidden layer (Simon, 2008).

In deep neural networks, the previous layer’s output is used to train every layer of nodes on a unique set of features. As we advance into a neural net, the nodes can recognize more complex features since they cumulate and recombine features from the previous layer. This is called *feature hierarchy* – it is a hierarchy of rising complexity and abstraction. Feature hierarchy enables deep learning neural networks capable of handling huge, multidimensional datasets with millions of parameters passing along non-linear functions.

Majority of data generated in the world are categorized under unlabeled, unstructured data. Unstructured data is also termed as *raw media*; these include texts, pictures, video and audio recordings. Processing and clustering of the world’s raw, unlabeled media, identifying the similarities and anomalies in data that no human can organize in a relational database can be done by utilizing deep networks. These deep networks are skilled in discovering patterns or structures within such datasets (Simon, 2008).

For example, deep learning can acquire a million images, and cluster them based on their similarities: dogs in one cluster, ice breakers in another, and in a third all the photos of your grandfather. This is the basis of smart photo albums.

Deep learning can cluster raw text such as news articles or emails. Emails can be classified as spam/not spam or emails filled with angry complaints from unhappy customers may be clustered in one corner of the vector space, while satisfied customers, or spambot messages, may be clustered in another vector space. This is the foundation of several messaging filters and can be utilized in customer relationship management (CRM). Voice messages can also be clustered using equivalent principle. With time series, data might cluster around healthy/normal behavior and dangerous/anomalous behavior. If the time-series data is being produced from a smartphone, it will give an insight to the user’s habits and health; if it is being produced by a vehicle, it might be used to prevent catastrophic breakdowns.

Deep learning neural network performs automatic feature extraction without human interventions, unlike all traditional machine-learning methods. Given that feature extraction is a task that can take teams of data scientists years to finish, deep learning is a mode to circumvent the chokepoint that limits experts. It enhances the powers of small data science teams, which by their nature do not scale.

When training on raw data, every node layer in a deep network learns features automatically by constantly trying to rebuild the input from which it draws the samples, trying to minimize the error between the network’s estimation and the probability distribution of the input data itself. An example of such reconstruction happens in restricted Boltzmann machines (RBMs).

In this process, these networks are trained to recognize correlations between some relevant features and optimal results – they illustrate connections between feature signals and what those features represent, whether it be a full reconstruction or has labeled data.

A deep learning network trained on labeled data can then be implemented on unstructured data, providing its access to much more input than machine-learning nets. This is a recipe for enhanced performance: the more data a net can train upon, the more precise it is likely to be. (Bad algorithms trained on lots of data can do better than good algorithms trained on very little data). Deep learning’s capability to process and learn from vast quantities of unlabeled data gives it a unique advantage over previous algorithms.

The last layer of deep neural network is an output layer which assigns a logistic, or softmax, classifier that assigns likelihood to a particular label or outcome. We call that predictive, but it is predictive in a broad sense. Given raw data in an image form, a deep-learning network may decide, for example, that the input data is 90% likely to represent a person. Computational intense is one of the attributes of deep learning, and it is one cause why GPUs are in need to train deep learning models.

An RNN remembers each and every information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called long short term memory (LSTM). RNNs are even used with convolution layers to extend the effective pixel neighborhood. RNN converts the independent activations into dependent activations by providing the same weights and biases to all the layers, thus reducing the complexity of increasing parameters and memorizing each previous outputs by giving each output as input to the next hidden layer.

SUMMARY

This chapter introduces the basic architecture of RNN and how it differs from the previously known ANN and CNN architectures. The backbone behind the RNN architecture, namely, BPTT, is explained in detail with an example. Other known variants like stateful RNN, LSTM and GRU have also been explained to help the reader analyze the pros and cons of each architecture.

REVIEW QUESTIONS

1. How is RNN different from other artificial neural network architectures?
2. What are the different RNN topologies, and why do we need them?
3. Name the major problem that hinders the performance of an RNN, and show how it can be overcome.
4. Find few applications where LSTM implementation gives better results when compared to other models and mention how LSTM performs better for that application.

ASSIGNMENT PROBLEMS

1. Given the truth table for XOR logic, use a simple RNN with weights and architecture same as that explained in Section 3.2.2, and calculate the output of the hidden layer.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

2. Apply sigmoid activation to the output of the hidden layer and find the output.
3. Compute error and repeat the process until the model converges.

REFERENCES

1. Britz, D. (2016). *Recurrent Neural Network Tutorial, Part 4 – Implementing a GRU/LSTM RNN with Python and Theano*. Available: <http://www.wildml.com/2015/10/recurrent-neural-network-tutorial-part-4-implementing-a-grulstm-rnn-with-python-and-theano>.
2. Gulli, A. and Pal, S. (2017). *Deep Learning with Keras*. Packt Books. Available: <https://www.packtpub.com/big-data-and-business-intelligence/deep-learning-keras>.
3. Jain, C. and Medsker, L. (2000). *Recurrent Neural Networks: Design and Applications*. Boca Raton, Florida: CRC Press.
4. Nicholson, C. V. and Gibson, A. (2001). *A Beginner's Guide to Recurrent Networks and LSTMs*. Available: <https://deeplearning4j.org/lstm>.
5. Nicholson, C. V. and Gibson, A. (2002). *Introduction to Deep Neural Networks (Deep Learning)*. Available: <https://deeplearning4j.org/neuralnet-overview>.
6. “Recurrent Neural Networks TensorFlow,” TensorFlow. Available: <https://www.tensorflow.org/tutorials/recurrent>.
7. Simon, H. (2008). *Neural Networks: A Comprehensive Foundation*. New Delhi: Prentice-Hall of India.

Autoencoder

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand the features and architecture of autoencoders.
- Understand the types of autoencoders and their architecture.
- Understand the use of autoencoders in the real world.

4.1 | INTRODUCTION

Deep learning architectures like Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN) are useful for supervised learning. In this chapter, we will introduce a deep learning architecture useful in unsupervised learning – an autoencoder. An autoencoder is one such unsupervised learning technique. It is useful for pre-training deep networks and for dimensionality reduction. Dimensionality reduction is an important step in data preprocessing to improve the process of machine learning. Given a dataset, dimensionality reduction reduces the dimensionality of the dataset. A dataset contains many features, but only some features are helpful to process the data further. Dimensionality reduction is the technique used to identify the required features.

This chapter focusses mainly on how autoencoders can be used for dimensionality reduction. Dimensionality reduction reduces the number of dimensions of the input data. Previously known dimensionality reduction techniques like Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA) and Non-negative Matrix Factorization (NMF) try to find the feature space that best describes the data. The objective of the autoencoder is to find the best latent representation of the input data.

An autoencoder is a feedforward network that uses the backpropagation algorithm to learn weights. It has a simpler architecture when compared to the deep learning architectures seen so far. It is a two-layer architecture [remember that the input layer is not counted as a layer in the artificial neural network (ANN) terminology]. The autoencoder has an input layer, a hidden layer and an output layer. The only difference is that the output should be the same as the input. This differentiates autoencoders from the architectures seen so far. That is, if (1, 1, 0, 0, 0, 1) is given as input, the autoencoder neural network tries to output (1, 1, 0, 0, 0, 1). This validates the requirement that the number of input neurons should be the same as the number of output neurons.

The hidden layer captures the best representative features of the input. That is, the hidden layer stores the representation of the input. Let us assume that the number of hidden nodes are much less than the number of input nodes (we will see later that this is not always the case). This type of autoencoder in which the dimension of the hidden layer is less than the dimension of the input layer is called *undercomplete autoencoder*. The values of the hidden layer are viewed as a compressed version of the input. In this example, let the number of input and output neurons be 6. The hidden layer has a smaller number of neurons than the number of neurons in the input layer. Let us have 2 neurons in the hidden layer. The autoencoder takes 6 features and encodes it using just 2 features. These 2 features are enough to reconstruct the 6 features. That is, we have just performed dimensionality reduction. The dimension of the original data is 6. The dimension of the dataset is reduced from 6 to 2; this is termed as *dimensionality reduction*.

4.2 | FEATURES OF AUTOENCODER

Autoencoders exhibit the following features:

1. **Data Dependent:** Autoencoders are compression techniques where the model can be used only on data in which they have already been trained. For example, the model of an autoencoder used to compress house images cannot be used to compress human faces.
2. **Lossy Compression:** Reconstruction of the original data from the compressed representation would result in a degraded output. This is illustrated in Fig. 4.1.

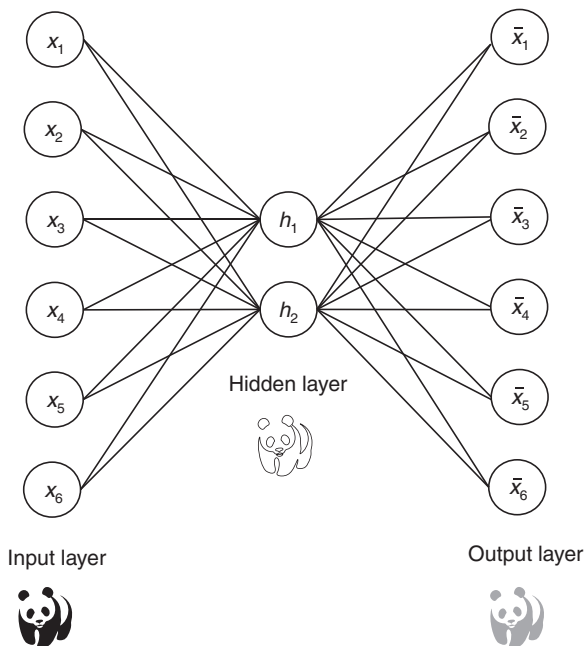


FIGURE 4.1 Lossy compression.

4.3 | TYPES OF AUTOENCODER

Let us learn about the following types of autoencoder:

1. Vanilla autoencoder.
2. Multilayer autoencoder.
3. Stacked autoencoder.
4. Deep autoencoder.
5. Denoising autoencoder.
6. Convolutional autoencoder.
7. Regularized autoencoder.

4.3.1 | Vanilla Autoencoder

Figure 4.2 is an illustration of a vanilla autoencoder. This is an ordinary autoencoder with no added features. It may be noted that the layers are fully connected.

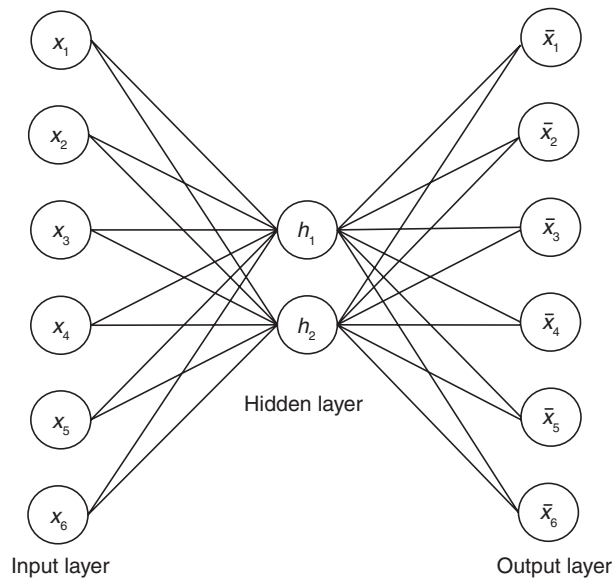


FIGURE 4.2 Vanilla autoencoder.

4.3.1.1 Structure of Vanilla Autoencoder

Autoencoders have three parts: the encoder, code and the decoder.

1. The **encoder** takes input from the input neurons. It then encodes the input and gives the output to the code. The encoder encompasses of the input layer and one or more hidden layers immediately after the input layer, finally resulting in the code. The encoder compresses the data.

2. The **code** is a hidden layer with reduced required number of nodes to represent the input.
3. The **decoder** is symmetric to the encoder. The final layer of the decoder is the output layer, where you get back the input data. It decodes the output of the code to produce a lossy reconstruction of the input and produces the final output.

The encoder takes $(1, 1, 0, 0, 0, 1)$ as input and outputs (h_1, h_2) . Code has the output (h_1, h_2) . In an ideal case, the decoder takes (h_1, h_2) as input and outputs $(1, 1, 0, 0, 0, 1)$. The vanilla autoencoder with weights and bias is shown in Fig. 4.3.

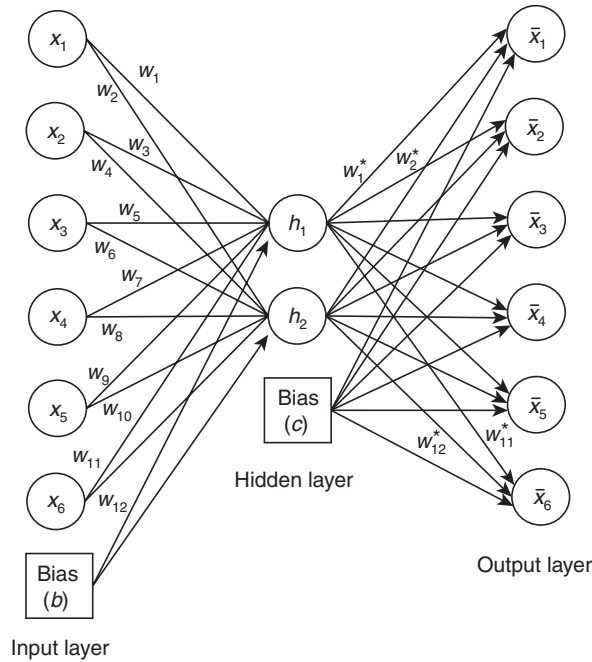


FIGURE 4.3 Vanilla autoencoder with weights and bias.

Let us assume there are n input nodes and d hidden nodes. The encoder is the function

$$f(wx + b)$$

where w is the real-valued vector $(w_1, w_2, \dots, w_{12})$ and can be generalized as $w \in R^{d \times n}$; b is the real-valued vector (b_1, b_2) and can be generalized as $b \in R^d$; x is the input vector and can be generalized as $x \in R^n$.

Here $wx + b$ is a linear transformation followed by a non-linear transformation produced by the activation function f .

Similarly, the decoder is the function

$$g(w^* \bar{x} + c)$$

where w^* is the real-valued vector $w_1^*, w_2^*, \dots, w_{12}^*$ and can be generalized as $w^* \in R^{n \times d}$; c is the real-valued vector $(c_1, c_2, \dots, c_6)$ and can be generalized as $c \in R^n$; y is the output vector and can be generalized as $\bar{x} \in R^n$.

Here $w * \bar{x} + c$ is a linear transformation followed by a non-linear transformation produced by the activation function g .

The choice of the activation function can differ depending on the input. For example, if the input values are binary then the activation function can be a logistic function.

4.3.1.2 Loss Function for Vanilla Autoencoder

The loss function is written in such a way that the output is the same as the input. The loss function used here is the mean squared error. For an autoencoder, it is the average of the squared error along all the input dimensions. If there are n input dimensions, then the mean squared error is

$$\min_{w, w^*, B, C} \frac{1}{n} \sum_{i=1}^n (\bar{x}_i - x_i)^2$$

For a lossless compression, the loss function would be closer to 0. If the activation function is logistic, then the output would be in the range $[0, 1]$. Here, the cross-entropy loss function can be used, which is given by

$$-\sum_{i=1}^n x_i \log(\bar{x}_i) + (1 - x_i) \log(1 - \bar{x}_i)$$

4.3.2 | Multilayer Autoencoder

When the vanilla autoencoder has more than one hidden layer it is termed the multilayer autoencoder. Multilayer autoencoder is a multilayer perceptron (MLP) with symmetry in the encoder and decoder sides. Figure 4.4 shows the architecture of a simple multilayer autoencoder.

The value in any layer can be used as an intermediate representation of the features. But usually it is symmetric and the middle layer is used for feature representation. The loss function is like that of a vanilla autoencoder.

4.3.3 | Stacked Autoencoder

Stacked autoencoders stacks various layers of the hidden layer in the encoder and the decoder. The training does not involve training end-to-end as in multilayer perceptrons using the backpropagation algorithm. Rather, when there are multiple hidden layers, the first hidden layer (h^1) is trained and the parameters are identified using backpropagation. That is, the stacked autoencoder looks like a simple autoencoder as given in Fig. 4.5.

To train the second hidden layer (h^2), h^1 is the given in the input and output layers and h^2 is used as the code layer. To find h^3 , we use h^2 and so on. This is shown in Fig. 4.6.

The size of the hidden layer continuously decreases, and each hidden layer is expected to learn an abstract feature. The final code layer can be given as input to a supervised learner to perform classification and regression. Figure 4.7 shows a binary classification.

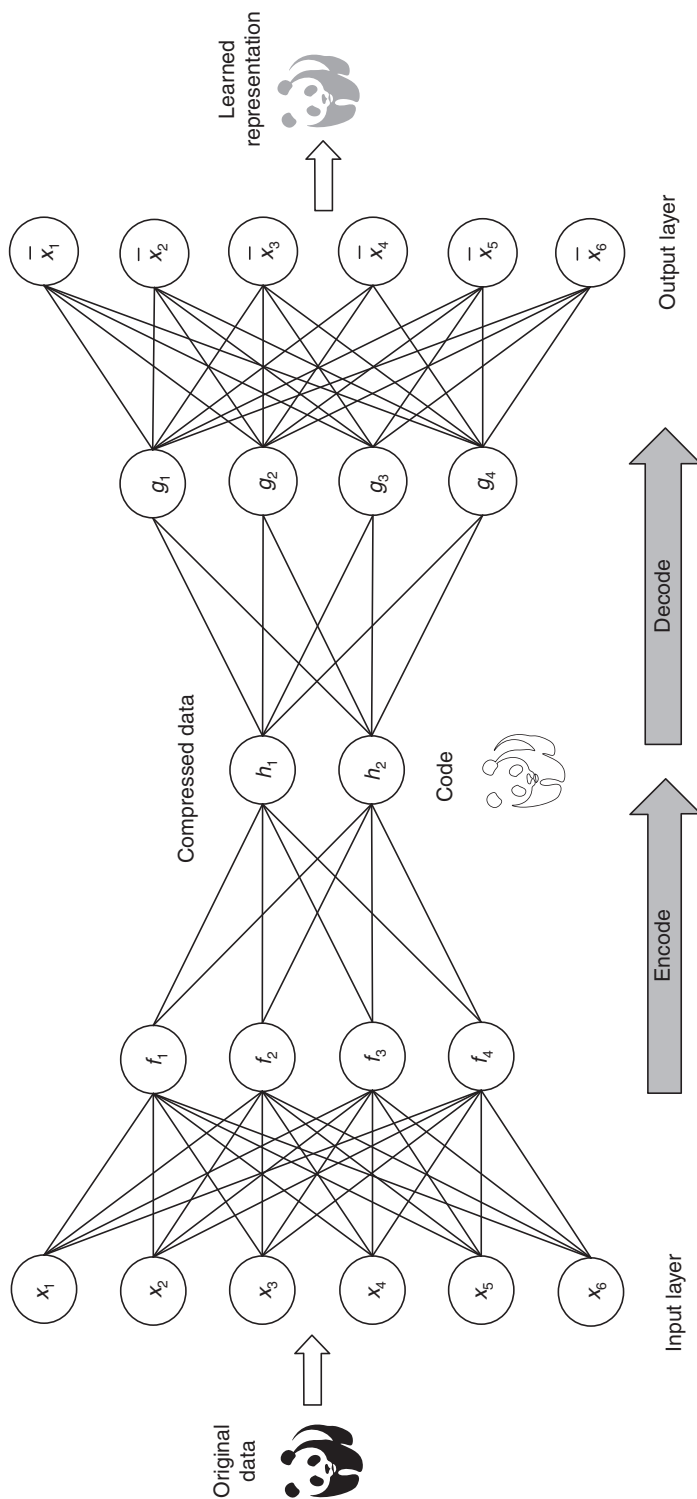


FIGURE 4.4 Simple multilayer autoencoder.

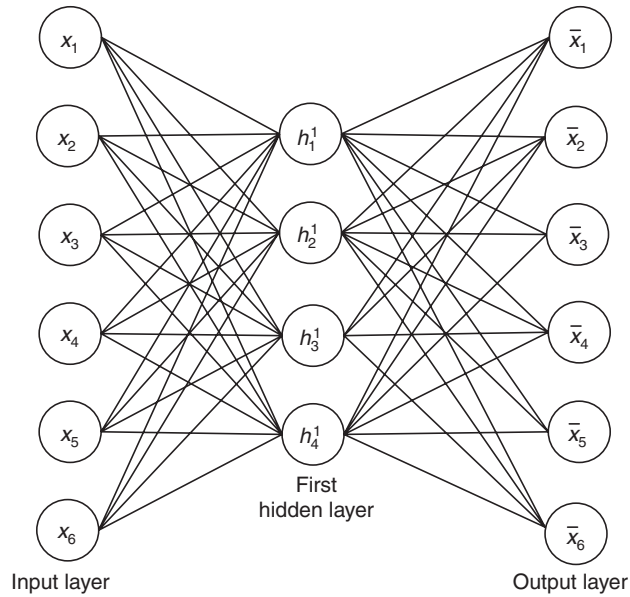


FIGURE 4.5 Stack autoencoder (training first hidden layer).

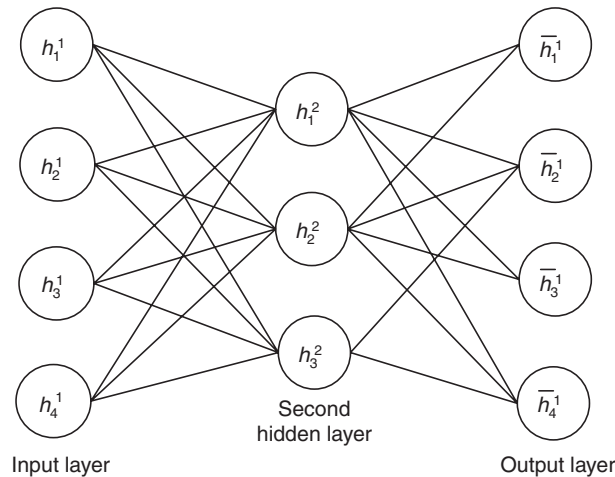


FIGURE 4.6 Stack autoencoder (training second hidden layer).

4.3.4 | Deep Autoencoder

A deep autoencoder comprises of two symmetric deep belief networks. Deep belief networks have been explained in Chapter 5. The layers are restricted Boltzmann machines because they are the building blocks of deep belief networks. The encoder is a deep belief network with four or five hidden layers. The decoder has the same number of layers as the encoder but it is a mirror image as given in Fig. 4.4.

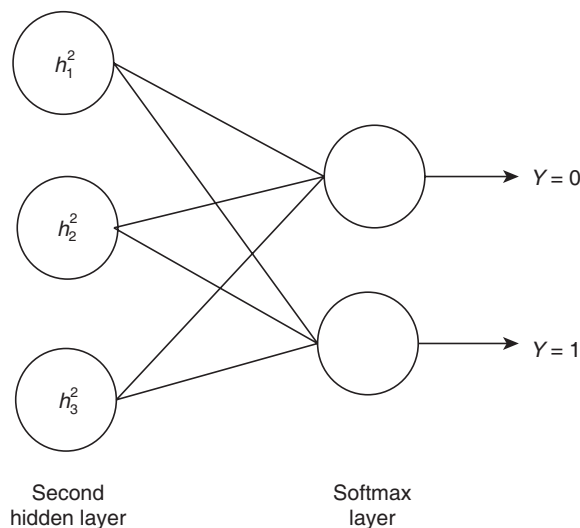


FIGURE 4.7 Stack autoencoder (training classifier).

4.3.5 | Denoising Autoencoder

Denoising autoencoder adds random noise to the input and forces the autoencoder to learn the original data after removing the noise. The autoencoder is trained in such a way that it identifies the noise, removes the noise and learns only the required features of the original data. The general architecture of the autoencoder is given in Fig. 4.8.

The loss function still checks the difference between the input data and output data. This ensures that there is no overfitting of data and the autoencoder can remove the noise and learn the important features of the input data after removing noise.

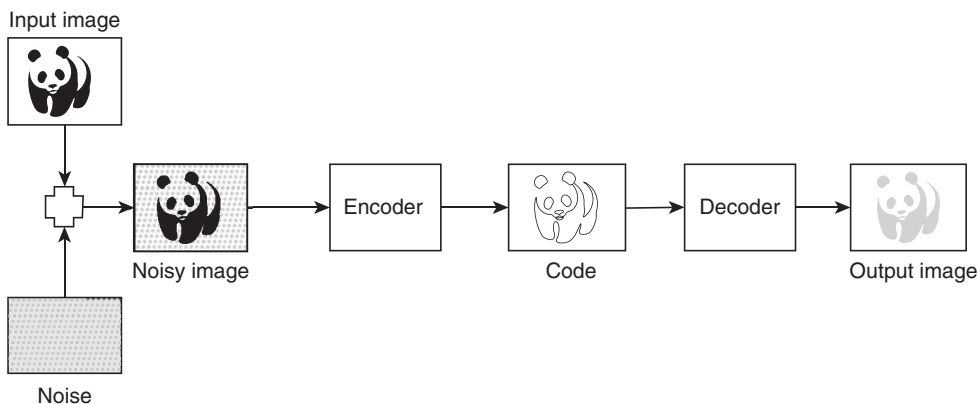


FIGURE 4.8 Denoising autoencoder.

4.3.5.1 Denoising Autoencoder in Overcomplete Autoencoder

It is not always necessary to have less hidden nodes than the number of input nodes in the code layer. When there are equal or more hidden nodes than the input nodes, the autoencoder is called *overcomplete autoencoder*. We expect the autoencoder to learn new features. But what might happen is that the values in the input nodes will be copied to the hidden nodes without learning any useful information. That is, the input data is stored without any modification in the hidden nodes and is subsequently transferred to the output layer and is found to have learnt the identity function. This condition known as identity encoding is shown in Fig. 4.9.

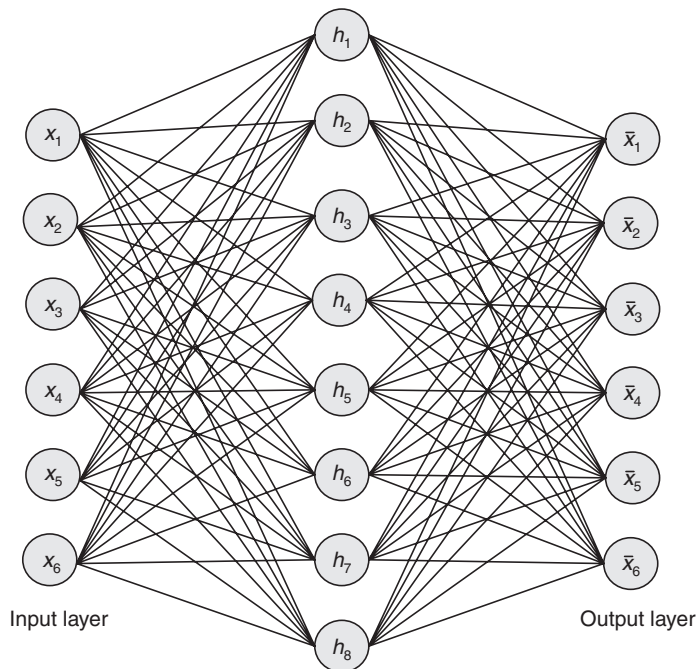


FIGURE 4.9 Overcomplete autoencoder with identity encoding.

Just as an undercomplete autoencoder is used to compress the input data by extracting useful features, an overcomplete autoencoder can be used to separate the jumbled features in an input data. Identity encoding can be avoided using denoising autoencoders.

4.3.6 | Convolutional Autoencoder

As the name suggests, a convolutional autoencoder combines the architecture of CNN and the vanilla autoencoder. While CNN create supervised machine learning models, autoencoders create unsupervised machine learning models. The encoder is a combination of the convolution layers and pooling layers as in a normal CNN. The pooling layers perform downsampling, while the decoder performs deconvolution and upsampling. The final layer gives the output which is expected to be the same as the input. A simple architecture is shown in Fig. 4.10.

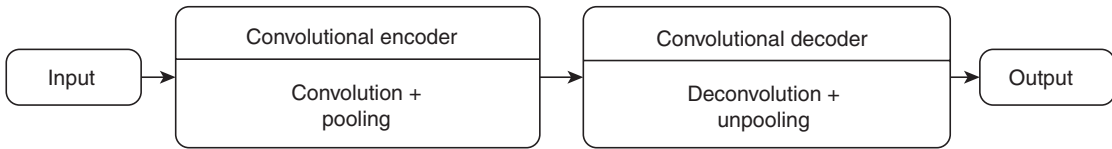


FIGURE 4.10 Convolutional autoencoder.

4.3.7 | Regularization in Autoencoder (Regularized Autoencoder)

Autoencoders can overfit the data when the loss function is close to zero. Overfitting can lead to poor generalization. This can also occur when there are too many parameters that fit the training data. Many variations of the vanilla autoencoder have been proposed with various regularization factors. When a regularization factor is added, the optimization technique does not try to take the loss function to zero. But rather, the sum of the loss function and the regularization term is made to tend to zero. This avoids overfitting of the data. The structure of the autoencoder remains the same. Only the loss function varies across the various autoencoders that use regularization.

4.3.7.1 Weight Decay

The simplest regularization term to add to the loss function is the *weight decay*, which is given by $\sum_{ij} w_{ij}^2$ and referred to as L2-penalty. This is also termed as *L2-regularization*. The overall loss function now becomes

$$L_{\text{new}} = L + \lambda \sum_{ij} w_{ij}^2$$

where λ determines the strength of the regularization term. Varying λ decides how much importance can be given to the loss function.

4.3.7.2 Sparse Autoencoder

As the name suggests, sparse autoencoders introduce sparsity into the autoencoders for the purpose of regularization. This method is also a type of regularization because we add a penalty term to the loss function to ensure that most of the nodes are inactive. A node is said to be inactive when it produces an output closer to zero. The sparsity constraint ensures that only a fraction of the input nodes is active at a time. The rest of the nodes are zero. The nodes should be active only when they must learn an important feature.

Overfitting occurs due to the numerous weight parameters that can take any value to learn the training data. Restricting the parameters helps avoid overfitting. The regularization term added in a sparse autoencoder is based on the Kullback–Leibler divergence (KL divergence). This is shown in Fig. 4.11, where the shaded nodes are inactive.

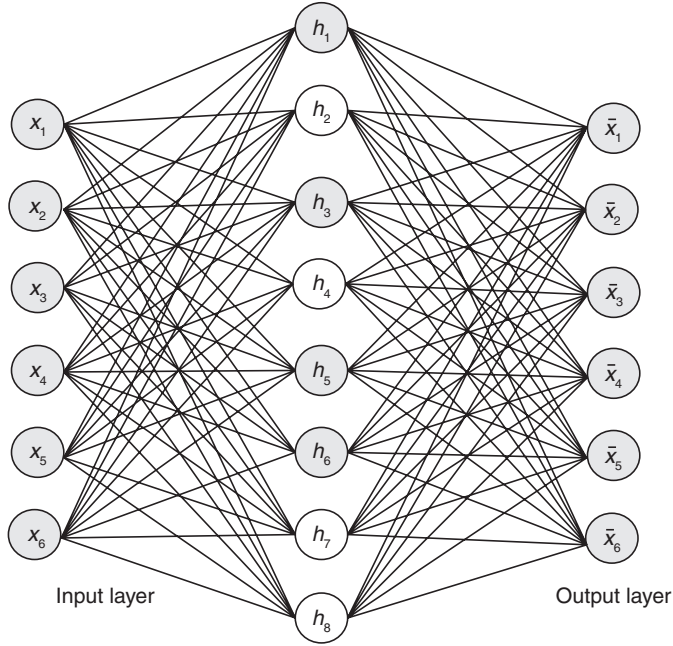


FIGURE 4.11 Sparse autoencoder.

Regularization Term in Sparse Autoencoder

Let h_l represent a neuron l in the hidden layer.

Let $a(h_l)$ be the activation of the h_l .

Let $a(h_l)_{x_i}$ be the activation of h_l for the input vector x_i .

Then, the average activation of the node is taken over all the samples in training set and is given by

$$\rho_l = \frac{1}{n} \sum_{i=1}^n a(h_l)_{x_i}$$

where n is the number of samples.

Sparse autoencoders make the average activation of the node tend to a value closer to zero. This value near zero is given by the sparsity parameter ρ . If $\rho = 0.03$, then the sparse autoencoder tries to make $\rho_l = \rho$.

To ensure this, the regularization term is chosen in such a way that ρ_l does not deviate much from ρ . The KL divergence is used for this purpose. Based on the KL divergence, the regularization term added to the loss function is given by

$$\sum_{j=1}^m \text{KL}(\rho \| \rho_j) = \rho \log \frac{\rho}{\rho_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \rho_j}$$

where m is the number of neurons in the hidden layer.

Loss Function of Sparse Autoencoder

The overall loss function including the regularization term now becomes

$$L_{\text{new}} = L + \lambda \sum_{j=1}^m \text{KL}(\rho \| \rho_j)$$

where λ is the weight parameter that controls the importance of the sparse regularization term. Here, loss function can be the mean squared error or the cross-entropy loss.

4.3.7.3 Contractive Autoencoder

Contractive autoencoders also use regularization to ensure that the data does not overfit nor does the code learn the identity function. Contractive encoders use the Frobenius norm of the Jacobian function for this purpose.

Jacobian Matrix

The Jacobian matrix is the matrix of the first order partial derivatives. Here, we measure the activation in the hidden layer (h) with respect to the activation in the input layer (x). The Jacobian matrix for k hidden nodes and n input nodes is given as follows:

$$J_x(h) = \begin{pmatrix} \frac{\partial h_1}{\partial x_1} & \frac{\partial h_1}{\partial x_2} & \dots & \frac{\partial h_1}{\partial x_n} \\ \frac{\partial h_2}{\partial x_1} & \frac{\partial h_2}{\partial x_2} & \dots & \frac{\partial h_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_k}{\partial x_1} & \frac{\partial h_k}{\partial x_2} & \dots & \frac{\partial h_k}{\partial x_n} \end{pmatrix}$$

The first column is the partial derivative of every neuron in the hidden layer with respect to the first neuron in the input layer. The first column shows how much the neuron in the hidden layer reacts to neuron x_1 . The second column is the partial derivative of every neuron in the hidden layer with respect to the second neuron in the input layer and so on.

Frobenius Norm

The Frobenius norm or Euclidean norm of a matrix (M) of order $n \times m$ is the square root of the sum of the squares of the elements of the matrix. The Frobenius norm of matrix M is given by

$$\|M\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m |m_{ij}|^2}$$

The regularization term used in contractive autoencoders is the square of the Frobenius norm of the Jacobian matrix $J_x(h)$. It is given by

$$\|J_x(h)\|_F^2$$

The overall loss function becomes

$$L_{\text{new}} = L + \|J_x(h)\|_F^2$$

The partial derivative is $\partial h_1 / \partial x_1 = 0$ when h_1 does not change but x_1 changes. That is, it tries not to capture the variations. In other words, it tries to not capture minor variations in the input data. Also, the first additive term tries to capture the variations in the input data. This ensures that the autoencoder learns only the important, distinguishing features and ignores the minor feature variations.

4.3.7.4 Variational Autoencoder

Variational autoencoders are very successful generative models. Generative models are models that generate or create new data based on numerous data already available. For example, a new face can be generated when the model is trained on a large number of human faces.

In a vanilla autoencoder, the encoder learns the features of the input in a compressed form. In variational autoencoders, the encoder also learns the features of the input, but outputs a vector of means and standard deviations. If c is a sample in the unit Gaussian distribution (a unit Gaussian distribution has mean 0 and standard deviation 1), then $\sigma x + \mu$ is a sample with mean μ and standard deviation σ . The decoder takes as input a sample from the vector of means and standard deviations. This helps the decoder to generate an output in the category of the input data, but different from the actual input data. This is shown in the following example.

Let

Output vector (μ) = [0.2, 0.9, 0.3, 0.7, ...]

Output vector (σ) = [0.3, 0.4, 1, 1.3, ...]

Then the samples generated will be

$$[X_{1 \sim N(0.2, 0.3^2)}, X_{2 \sim N(0.9, 0.4^2)}, X_{3 \sim N(0.3, 1^2)}, X_{4 \sim N(0.7, 1.3^2)}, \dots]$$

From this, a sample of encoding vectors are stochastically generated. The architecture of the variational autoencoder is represented in Fig. 4.12.

Loss Function of Variational Autoencoder

The loss function is a combination of reconstruction loss and latent loss. The reconstruction loss is the usual mean squared error. The latent loss is the KL divergence and it measures how closely the latent variables match the unit Gaussian distribution ($N(0,1)$). The loss function can be written as

$$L_{\text{new}} = -\frac{1}{2} \sum_{j=1}^J [1 + \log(\sigma_j^2) - \mu_j^2 + \sigma_j^2] + E \left[\sum_{i=1}^D [x_i \log y_i + (1 - x_i) \cdot \log(1 - y_i)] \right]$$

where J is the size of the standard deviation and mean vectors.

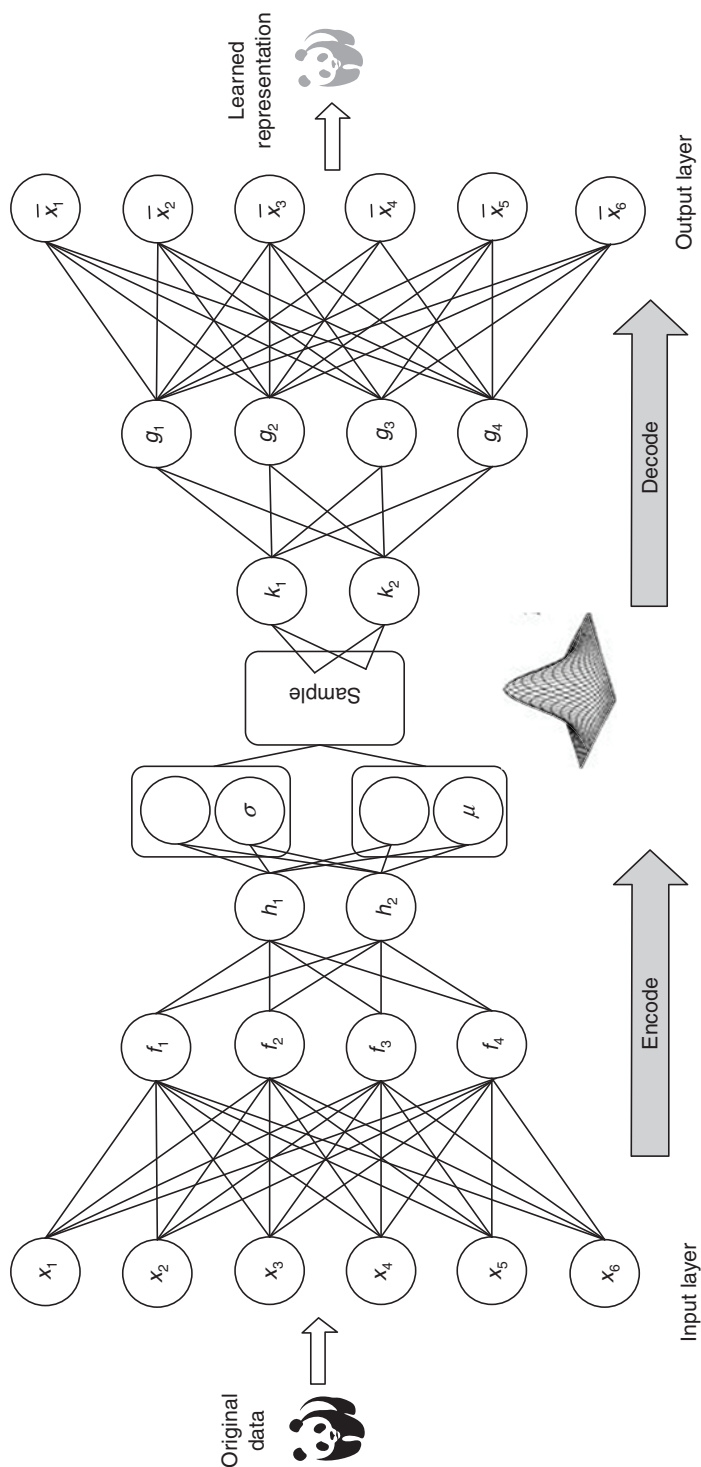


FIGURE 4.12 Variational autoencoders.

4.1 EXAMPLE Real-World

1. Dimensionality Reduction using Autoencoders in MNIST Dataset

As a real-world example, let us consider the MNIST dataset. The MNIST dataset is a common dataset used for classification and image recognition tasks. It has images of handwritten digits. It has 60,000 examples in the training set and 10,000 examples in the test set. Each digit is a 28×28 image. The input to an autoencoder is $28 \times 28 = 784$ neurons. The images can be compressed using autoencoders. The compression depends on the number of hidden neurons. If the number of hidden neurons is 50, the image can be reconstructed using just 50 features instead of 784 features. There have been cases where the authors have reconstructed the MNIST dataset using just 25 hidden neurons. The original input and the output of the autoencoder are given in Fig. 4.13.



FIGURE 4.13 MNIST dataset reconstructed using autoencoders.

2. Image Retrieval using Autoencoders

Content-based image retrieval (CBIR) is a type of image retrieval system. When a query image is given, based on the amount of similarity in the visual contents, images are retrieved. Convolution autoencoder is used to encode the query image into the latent space representation. The images from the search dataset are also encoded to the latent space representation. Subsequently, the encoded query is compared with encoded images from the search dataset using the nearest neighbor algorithm that uses the Euclidean Distance. The images having the closest distance will be displayed.

4.1 EXAMPLE (Continued)

The dataset used is the MNIST dataset and images with similar digits and handwriting style are retrieved.

Architecture

Two encoders E1 and E2 take 28×28 grayscale images as input and the architecture consists of four convolutional layers with 3×3 kernels, followed by ReLU activations. The outputs of the last convolutional layers are $4 \times 4 \times 256$ tensors that are mapped to a 16-dimensional latent representations $z1$ and $z2$ using a single fully-connected layer with ReLU activation. The final layer is a fully-connected layer with linear activation behind E2. Batch normalization is employed for all the convolutional layers and not for the fully-connected layers. The decoder is the reverse of the encoders. It starts with a fully-connected layer to increase the dimensionality of the latent representation to 4096 dimensions and reshape to form a $4 \times 4 \times 256$ tensor. This tensor is mapped back to the input image space using four layers of transposed convolutions. Batch normalization and ReLU activations are employed in all the layers excluding the output layer. The output layer uses hyperbolic tangent activation.

3. Reconstruction of Video Frames using LSTM Autoencoders

Sequence prediction problems are problems in which the input is a sequence of data and the output is to predict the next data in the sequence. The data can be text, image or sound. Nitish Srivastava, *et al.* took videos as input data and used LSTM autoencoders to predict the next frame in a video sequence.

Architecture

The architecture comprises of an encoder LSTM and a decoder LSTM. The encoder LSTM reads a frame sequence. This is given as input to a decoder LSTM. The decoder predicts the target sequence in the reverse order. This helps to reconstruct the input. It is also possible to remove the decoder part and use the architecture to predict a new target sequence.

SUMMARY

This chapter introduced autoencoders and its features. The various types of autoencoders were dealt in detail and the differences were highlighted. Readers should have a clarity on the different autoencoder architectures and must be able to use the appropriate autoencoder for their application.

REVIEW QUESTIONS

1. How does the autoencoder differ from the other deep learning architectures?
2. How are the weights updated in stacked autoencoders?
3. How does adding noise help to overcome overfitting in autoencoders?
4. When does an autoencoder learn the identity function?
5. What are generative models? Why are variational autoencoders said to be generative models?

ASSIGNMENT PROBLEMS

1. What is the effect on the regularization term λ as the value of λ changes? Show its effect on using $L2$ regularization when $\lambda = 1$, $\lambda = 10$ and $\lambda = 100$.
2. Write a program using Python to generate images of faces using variational autoencoders.
3. Write a program using Python to compress MNIST images using autoencoders. Find the minimum number of features required for best possible decompression of data.

REFERENCES

1. Domingos, Pedro. 2015. *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World*. Penguin
2. Dorta, G., Vicente, S., Agapito, L., Campbell, N. D. F., and Simpson, I. 2018. Training VAEs under structured residuals. [arXiv:1804.01050 (stat.ML)]
3. Liou, Cheng-Yuan; Cheng, Wei-Chen; Liou, Jiun-Wei; Liou, Daw-Ran. 2014. Autoencoder for words. *Neurocomputing*. 139: 84–96
4. Makhzani, Alireza and Frey, Brendan. 2013. k-sparse autoencoder. [arXiv:1312.5663]
5. Vincent, Pascal; Larochelle, Hugo; Lajoie, Isabelle; Bengio, Yoshua; Manzagol, Pierre-Antoine. 2010. Stacked denoising autoencoders: learning useful representations in a deep network with a local denoising criterion. *The Journal of Machine Learning Research*. 11: 3371–3408

Restricted Boltzmann Machine

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand the architecture of Boltzmann machines and Restricted Boltzmann Machines (RBMs).
- Understand the training phase of RBM.
- Identify the variant of RBM to use for a real-world problem.

5.1 | BOLTZMANN MACHINE

Boltzmann machines are symmetrically connected recurrent neural networks developed by Stanford scientist Geoff Hinton. They are said to be symmetrically connected because they are undirected edges between the nodes and the weight from node i to node j is the same as the weight from node j to node i . The input vectors in Boltzmann machines is binary vectors. They are said to be recurrent neural networks because they apply the same learning algorithm between the input nodes and the hidden nodes. The structure of a Boltzmann machine is given in Fig. 5.1.

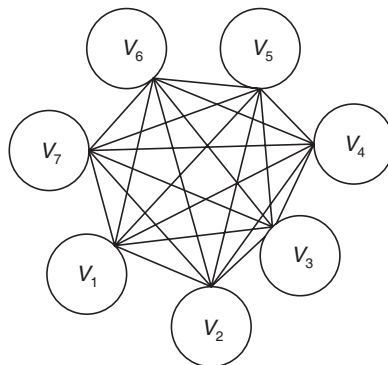


FIGURE 5.1 Structure of a Boltzmann machine.

5.1.1 | Relation to Hopfield Networks

The similarity between Hopfield networks and Boltzmann machines is that they both have a fully connected network of nodes and an energy is associated with the entire network. However, Boltzmann machines are stochastic. That is, the output of Boltzmann machine is a probability distribution and not continuous values as in regression or categorical values as in classification.

5.2 | RBM ARCHITECTURE

The RBM was introduced by Salakhutdinov *et al.* in 2007. They used RBMs to improve the accuracy in collaborative filtering. RBM helps to perform unsupervised training of data. That is, it is possible to train an RBM on an unlabeled training data. RBM is a special case of Boltzmann machine in which we have two layers, namely, the visible (input) layer and the hidden layer. The input vectors are not connected to each other and are connected to a set of hidden nodes. Similarly, the hidden nodes are not connected to one another. This results in the term *restricted*. All the connections are symmetric. An example of a Boltzmann machine with four input nodes (v_1, v_2, v_3, v_4) and three hidden nodes (h_1, h_2, h_3) is given in Fig. 5.2.

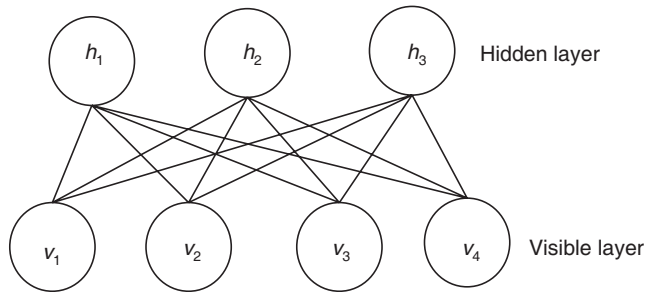


FIGURE 5.2 Example of a restricted Boltzmann machine.

As seen in the figure, this is a bipartite graph because it has two sets of nodes where one set of nodes is fully connected to the nodes in the second set.

5.2.1 | Energy-Based Model

Boltzmann machines are closely associated with quantum physics. They are energy-based models because energy is associated with the restricted Boltzmann machines. The state of the entire network is now given in terms of the energy of the network, termed the *energy function*. The energy is determined by configuration of the variables. The variables associated with the RBM are the values in the visible node and values in the hidden node. The nature of the energy function is such that the energy becomes large when the compatibility between the variables is low. The objective is to increase the compatibility between the variables. That is, the energy value is decreased iteratively till it reaches the local minima.

5.2.2 | Gibbs Distribution

The output of an RBM is a Boltzmann distribution also called as Gibbs distribution. The probability distribution function gives the probability with which a system given by x is in a certain state as a function of that state's energy and the temperature of the system. The probability distribution is given by

$$P(x) = \frac{e^{-E(x)}}{Z}$$

Z is the normalizing factor and is called the partition function. It is given by

$$Z = \sum_x e^{-E(x)}$$

In the case of an RBM, the input are the values in the visible and hidden neurons given by v and h , respectively. The probability that the system is in the state given by v and h is a joint probability and can be written as

$$P(v, h) = \frac{e^{-E(v, h)}}{Z}$$

where $Z = \sum_x e^{-E(v, h)}$. Here v is the input vector and h is the hidden layer vector. We consider all possible pairs between the visible and hidden units to calculate Z . The joint probability $P(v, h)$ is difficult to compute because the number of possible pairs between v and h would be $v \times h$. The Gibbs sampler is used to sample from $P(v, h)$ during the training phase.

5.2.3 | Gibbs Sampler

Gibbs sampling makes use of conditional probability to simplify the calculations. Initially, the input vector termed v_0 is a value taken from the training data. The value of h can be predicted using the input vector (v). The prediction can be formulated as the conditional probability $P(h|v)$. Similarly, if we know h , v can be found in a similar manner using the conditional probability $P(v|h)$. The new visible layer vector found is termed as v_0^1 . When it is continued k times, we get k such variations of the input vector $v_0, v_0^1, v_0^2, \dots, v_0^k$. These are termed as forward and backward passes. Each such cycle is termed as *Markov Chain Monte Carlo*. This is shown diagrammatically in Fig. 5.3.

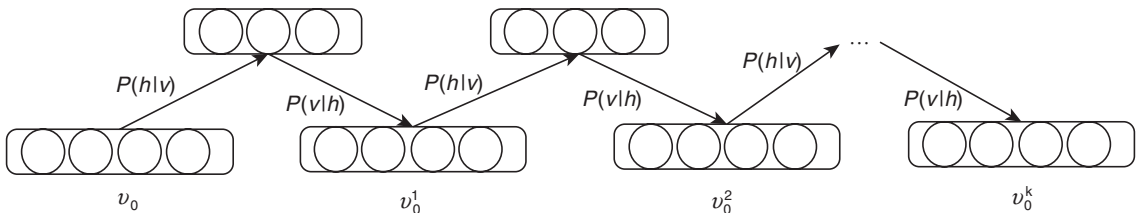


FIGURE 5.3 Gibbs sampling.

The new vector created in the k th iteration is a recreation of the original input v_0 . The structure of the RBM is such that there is no connection between the units of the visible layer or between the units in the hidden layer. So, the conditional probability can be written as

$$P(h | v) = \prod_j P(h_j | v)$$

$$P(v | h) = \prod_i P(v_i | h)$$

The vanilla RBM assumes the visible and hidden units to exist in one of the two states, namely, 0 and 1. That is, a neuron is said to be active if it is in state 1. The input vector (v) can be of dimension N and each dimension can take the binary value 0 or 1. Every hidden unit has a bias b_j associated with it. The weight between unit i and unit j is given by w_{ij} . The weights between the input vector and hidden vector (h) are initialized randomly. The probability that a hidden layer is active is given by

$$P(h_j = 1 | v) = \frac{1}{1 + e^{-\left(\sum_{i=1}^N w_{ij} v_i + b_j\right)}} = \sigma\left(\sum_{i=1}^N w_{ij} v_i + b_j\right)$$

where σ is the sigmoid function. The hidden layer is of dimension M and every visible unit has a bias c_i associated with it. Similarly, the probability that a visible node is active is given by

$$P(v_i = 1 | h) = \frac{1}{1 + e^{-\left(\sum_{j=1}^M w_{ij} h_j + c_i\right)}} = \sigma\left(\sum_{j=1}^M w_{ij} h_j + c_i\right)$$

That is, the output of the sigmoid activation function determines the probability of activating a neuron. The output is a value between 0 and 1. The same weight as used in the calculation of the hidden unit is reused. The values are rounded to produce binary values. The same weight is used in all the iterations. The value converges when the output of the visible unit in the i th iteration is the same as the output of the visible unit in the $(i + 1)$ th iteration. That is, the model has learnt the input data.

5.2.4 | Contrastive Divergence

Having too many iterations like the one discussed in previous section is not practical when training real-world data. Hinton *et al.* showed that it is not necessary to wait till the chain converges. Since what was needed was just the direction in which the distribution moved towards so that the target distribution coincided with the estimated distribution, Hinton showed that even with small number of iterations the result could be achieved. The direction in which the estimated distribution should move such that it reflects the target distribution gives the approximate gradient. Contrastive divergence estimates the gradient of the energy function. The weight update is done in contrastive divergence.

The output in the visible units is a probability distribution which may or may not reflect the actual input. The difference between the estimated probability distribution (Q) and the original probability distribution (P) is given by the Kullback–Leibler divergence. The similarity of the distribution P to the distribution Q is given by $D_{KL}(P||Q)$. Contrastive divergence approximates the gradient achieved when we take the difference in the probability distribution in the initial step and the probability distribution after k iterations. This is given as

$$CD_k = KL(P_0||P_\infty) - KL(P_k||P_\infty)$$

It was shown that the result would be the same even when k was as small as $k = 1$. The update matrix is given as the outer product of two probabilities.

$$\nabla w = v_0 \otimes P(h_0|v_0) - v_k \otimes P(h_k|v_k)$$

The new_weight is calculated as

$$\text{new_weight} = \text{old_weight} + \nabla w$$

5.3 | EXAMPLE

A group of dog lovers were asked to give their preference for the dog that they would like to buy. The preference is given as 0s and 1s for different dogs such as poodle, mastiff, bull dog, hound, terrier, shih tzu. Of these poodle and shih tzu are companion dogs, mastiff and bull dog are guard dogs, while hounds and terriers are hunting dogs. If a person gives the input vector (1, 0, 0, 0, 0, 1) the preference for companion dogs is evident. Let us see how RBM is used to find this person's latent preference for companion dogs.

We construct an RBM with six visible units representing the different dogs (poodle, mastiff, bull dog, hound, terrier, shih tzu) and three hidden units. An RBM is created for each user separately. A sample RBM state is given in Fig. 5.4.

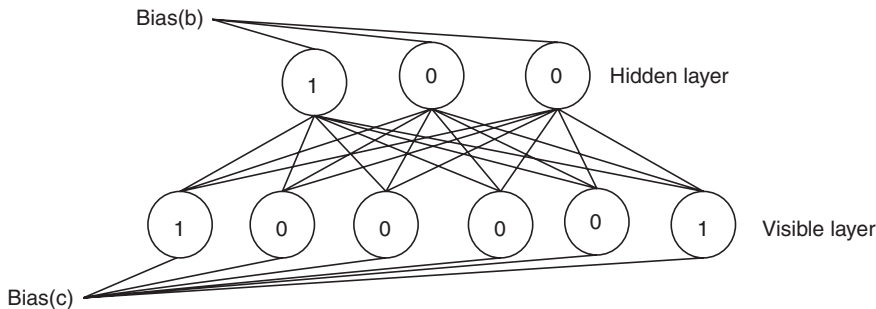


FIGURE 5.4 RBM example.

The final activation in the hidden layer is given as 0 or 1. But, it would be a probability value between 0 and 1. The probability value decides whether a node is activated or not. The feature latent in his/her selection of dog is activated in the hidden layer.

5.4 | TYPES OF RBM

Based on the distribution used and the structure of the hidden layers, many types of RBM are possible. Some of these are as follows:

1. **Bernoulli–Bernoulli RBM:** The units in the RBM considered so far are random variables taking binary values. The probability density function is conditioned to a Bernoulli distribution. This can be referred to as the Bernoulli–Bernoulli RBM where the visible and hidden units are modeled using the Bernoulli distribution.
2. **Gaussian–Bernoulli RBM:** There are variants of the RBM that model visible units as Gaussian and hidden units as Bernoulli and are referred to as Gaussian–Bernoulli RBM. This allows the visible units to take real-valued input that are modelled using a normal distribution.
3. **Conditional RBM:** In this RBM, the visible units are modelled using the Gaussian distribution and the hidden units use rectified linear unit transformation. Using binary values in the hidden layer restricts the number of latent features that can be represented. Rectified linear units help to represent more features.
4. **Deep Belief Network (DBN):** When the features can be represented as a hierarchy, deep belief networks are useful. They stack RBMs to represent the features of the training data as a hierarchy. The architecture of deep belief networks is given in Fig. 5.5.

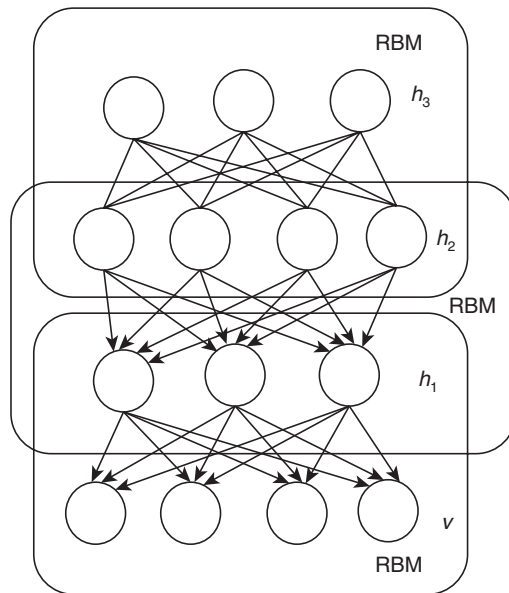


FIGURE 5.5 Deep belief networks.

The training in deep belief networks is a greedy layer-wise unsupervised training. During training, the first RBM trained has the input vector in the visible layer and the first hidden layer. Figure 5.6 shows the first level of training.

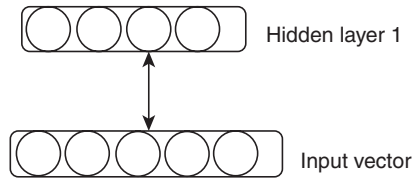


FIGURE 5.6 First level of training.

The output of this RBM is a representation of the input and is given as input to the next layer as shown in Fig. 5.7. The training is performed using contrastive divergence as usual.

The output of this layer can be used to train the next layer and is as shown in Fig. 5.8.

The process is repeated till the desired condition is met. Each layer has a higher level representation when compared to the previous layer.

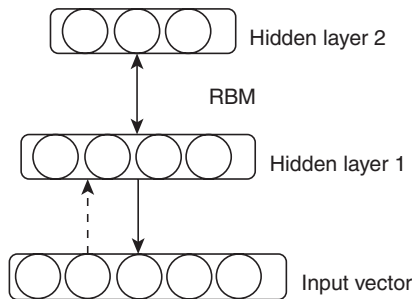


FIGURE 5.7 Second training.

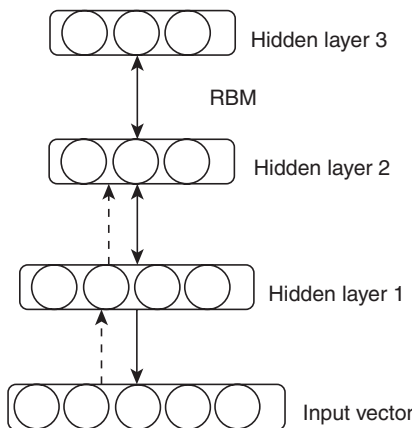


FIGURE 5.8 Three layer DBN.

SUMMARY

This chapter introduces the basis for restricted Boltzmann machine. The architecture of the RBM and the forward pass and the backward pass in the training phase have been explained with the outline of Gibbs sampling and contrastive divergence. The different variants of RBMs are seen and how RBM works is explained with an example.

REVIEW QUESTIONS

1. How do restricted Boltzmann machines differ from Boltzmann machine?
2. What is the advantage of using contrastive divergence?
3. How are energy-based models different from the other models?

ASSIGNMENT PROBLEMS

1. Is it possible to use activations other than the sigmoid activation?
2. Show how RBMs can be used for collaborative filtering.
3. Can RBMs be used to pretrain models? If so, how is it possible?

REFERENCES

1. Ruslan Salakhutdinov, Andriy Mnih, Geoffrey E. Hinton, (2007). Restricted Boltzmann machines for collaborative filtering. *Proceedings of the Twenty-Fourth International Conference (ICML 2007)*. Oregon, USA.
2. Hinton, G. E., (2002). Training products of experts by minimizing contrastive divergence. *Neural Computation* 14: 1711–1800.

Open-Source Frameworks for Deep Learning

CHAPTER 6

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand the importance of Python in deep learning.
- Understand the features and usage of popular deep learning frameworks.

6.1 | PYTHON – AN INTRODUCTION

Python (logo shown in Fig. 6.1) is a multipurpose, interactive, interpreted, scripting, object-oriented, and high-level programming language. It was created by Guido van Rossum and first released in 1991 at the National Research Institute for Mathematics and Computer Science in the Netherlands. It is copyrighted and the source code is available under the GNU General Public License (GPL). Guido van Rossum chose the name 'Python' since he was a big fan of Monty Python's Flying Circus, a British surreal comedy group. Python's design philosophy emphasizes readability using whitespaces. The constructs are more English-like compared to any other programming language, thus enabling a large range of audience to learn it faster and more accurately.

6.1.1 | Characteristics of Python

1. **Multipurpose:** It can be used for a wide range of applications.
2. **Interactive:** It has an interactive prompt called Python prompt where people can code just like that.
3. **Interpreted:** It is processed at runtime line-by-line, so it need not be compiled.
4. **Scripting:** Being a multipurpose language, it nicely fits for scripting.
5. **Object-oriented:** It supports the object-oriented style of programming such as encapsulation, inheritance, etc.
6. **High-level Language:** It needs to be processed before execution.



FIGURE 6.1 Python logo.

6.1.2 | Features of Python

1. **Open Source:** The source code is freely available at the official website (www.python.org).
2. **Easy to Read and Maintain:** Indentations and statements make it easy to read and maintain.
3. **Portable:** Python code can run on any platform with the help of Python Virtual Machine (PVM) with no difference in execution.
4. **Extensible:** New lower-level modules can be added to Python interpreter so that it can be easily extended.
5. **GUI Programming:** Desktop applications can be made for any platform with access to many system calls.
6. **Scalable:** It provides more support for larger programs than shell scripting which is basically a few lines of code.

6.2 | ENVIRONMENTAL SETUP

Python is available for deployment on a wide variety of platforms. The user needs to download the binary code compatible with the platform that he/she is working on.

UNIX/Linux

1. Visit www.python.org/downloads and follow the link to download zipped file for UNIX/Linux.
2. Download and extract the zip file.
3. Edit Modules/Setup file if you want to customize Python (setup.py is the file that tells that Python is packaged and distributed whereas Modules directory consists of all packages' codes).
4. Run *./configure script*.
5. Run *make install* (This installs Python at standard */usr/local/bin/python* and libraries at */usr/local/lib/pythonxx* where xx is the Python version).
6. Type *export PATH = "\$PATH:/usr/local/bin/python* to setup path.

Windows

1. Visit www.python.org/downloads and follow the link to download windows installer (.msi file).
2. Download and install the file (it installs at C:\PythonXX where XX is the Python version).
3. Do not change any options in the installer unless you want to customize the installation.
4. Setup path using *path %path%;C:\Python*.

Macintosh

For Mac, an older version of Python comes pre-installed. To use the current version, follow the instructions at www.python.org/downloads/mac.

After installing, type *python* in command prompt (or shell) of any platform to check whether or not Python is successfully installed, and the path is added to the system environment variable PATH.

6.3 | DEEP LEARNING WITH PYTHON

When it comes to data science, machine learning, or deep learning, R becomes Python's rival. Since R is simply a mathematical and statistical tool, it cannot be used to create front-ends (like a website or an application), handle database requests from front ends, etc. For each of these processes, a separate application has to be used. Mostly, R is useful for dataset exploration and visualization. Since Python is a general-purpose programming language, its focus is not only on mathematics and statistics. It can be used to create applications, handle database requests, write object-oriented codes, automate processes, and much more. Deep learning requires the basic knowledge in probability, linear algebra, calculus and some basic machine learning techniques. Python provides many such packages that work equally to R and some better than R.

6.4 | SCIENTIFIC PYTHON (SciPy)

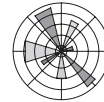
Scientific Python (SciPy) is a Python ecosystem for performing mathematical, scientific, and engineering computations. Its core packages are shown in Fig. 6.2.



NumPy
Base N-dimensional
array package



SciPY library
Fundamental library
for scientific computing



Matplotlib
Comprehensive 2D
plotting



IPython
Enhanced interactive
console



SymPy
Symbolic mathematics



Pandas
Data structures and
analysis

FIGURE 6.2 SciPy ecosystem.

SOURCE: SciPy, <https://www.scipy.org/>

Among these packages, NumPy, Matplotlib and Pandas are mostly used in pre-processing and visualization part of any deep learning application. In this section, we will discuss these packages.

6.4.1 | Numerical Python (NumPy)

Numerical Python (NumPy) is a powerful package for multidimensional array processing. All deep learning algorithms require inputs as NumPy arrays and return NumPy arrays as well. It contains the following:

1. An n -dimensional array object.
2. Tools for integrating C/C++ codes.
3. Linear algebra, Fourier transform, and random number capabilities.

NumPy can also be used as an efficient multidimensional container of generic data. Arbitrary data types can be defined. This allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

6.4.2 | Matplotlib

Matplotlib is a library for plotting 2D graphs and charts. It has a submodule named *pyplot* which plots beautiful graphs in a more customizable way. It provides features to manipulate and control line styles, line colors, formatting axes, fonts, etc. Matplotlib can be used in Python scripts, Python, IPython Shells, Jupyter notebook, web application servers, and graphical user-interface toolkits.

6.4.3 | Pandas

Pandas is the data analytics library for Python. It provides high-performance and efficient data structures to handle datasets. It is compatible with NumPy as it uses most of its functionalities. It can read and write most common file formats such as csv, json, html, excel and hdf5. The fast and efficient DataFrame object, intelligent handling of missing data, smart label-based indexing make developers use Pandas in fields like statistics, neurosciences, analytics, etc.

6.5 | FRAMEWORKS

With huge community support, many deep learning frameworks are available in Python for efficient processing. Some of them include the following:

1. TensorFlow by Google.
2. Caffe.
3. Microsoft CNTK.
4. PyTorch.
5. MXNet.
6. Keras.

As shown in Fig. 6.3, the most used frameworks are TensorFlow, Keras, and PyTorch, which are explained the following subsections.

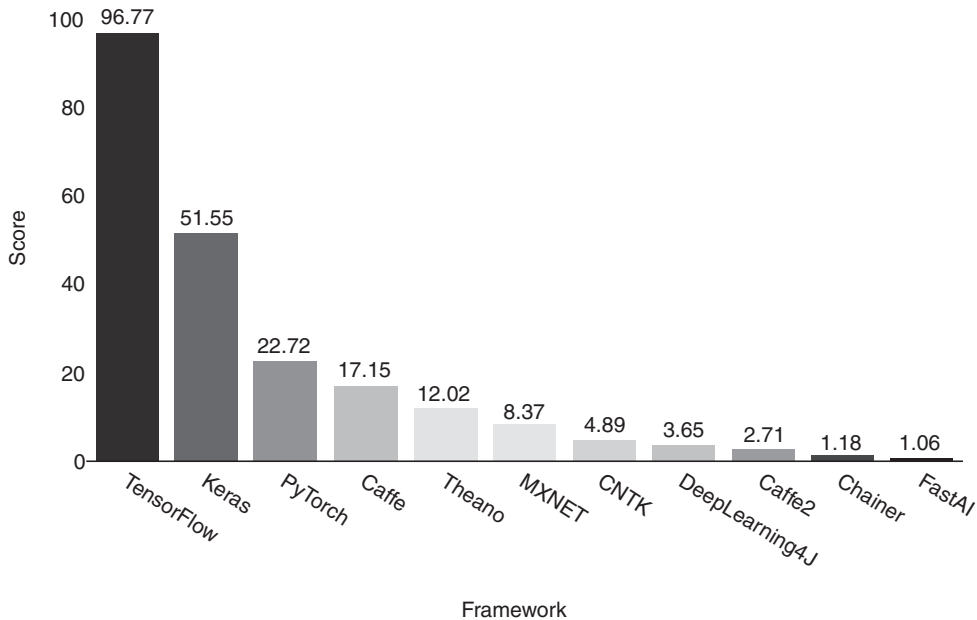


FIGURE 6.3 Deep learning frameworks ranking for 2018, computed by Jeff Hale, based on 11 data sources across 7 categories.

SOURCE: *Towards Data Science*, <https://towardsdatascience.com/deep-learning-framework-power-scores-2018-23607ddf297a>

6.5.1 | TensorFlow

TensorFlow™ (logo shown in Fig. 6.4) is Google Brain and Google’s Machine Intelligence research organization’s solution to high-performance numeric processing. It is an open-source Python library.

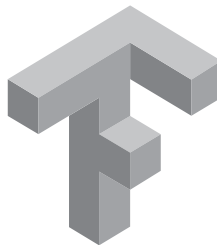


FIGURE 6.4 TensorFlow logo.

SOURCE: *TensorFlow*, <https://www.tensorflow.org/>

Tensor forms the basic data unit in TensorFlow. A simple tensor can be considered as a geometric vector. It can be seen that two vectors can be added to form a new vector. TensorFlow works on “tensor data structure”, which can be thought as an n -dimensional array of base objects or datatypes. A tensor has a rank, a shape and a static type, so a tensor can be represented as a multidimensional array of numbers.

Let us consider a simple tensor creation.

```
import tensorflow as tf
matrix1 = tf.constant([[1, 2, 3], [4, 5, 6]])
# matrix: rank 2 shape [2, 3]
matrix1.get_shape()
# Output: TensorShape([Dimension(2), Dimension(3)])
```

This paradigm of solving is called *data flow programming*, where the solution to the problem is modeled into a directed graph of data flowing between operations. The program input is converted into a tensor object that can be solved off using this paradigm. A data flow graph will have the following:

1. Nodes that represent the mathematical operation.
2. Edges that represent tensors communicated between nodes.

In the graph shown in Fig. 6.5, there is only one operation which is 'add', and there are three tensors communicating: tensors 'a' and 'b' are inputs and tensor 'c' is the output.

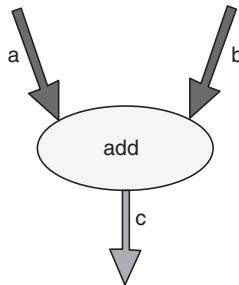


FIGURE 6.5 Flowgraph for the addition of two tensors.

The steps and code for the graph shown in Fig. 6.5 are explained in Section 6.5.1.3.

6.5.1.1 TensorFlow API

Multiple APIs are provided for implementing applications with TensorFlow. TensorFlow Core (the base tensor computation) is being used for more control and forward research by machine learning engineers and data scientists. TensorFlow Core mostly deals with low-level coding but at the same time it provides the most control over the application. For easier use and application, a simpler API is built over TensorFlow Core. Such a higher level API is called `tf.contrib.learn`. The most commonly used APIs are listed below:

1. A constant tensor can be created using `tf.constant(...)`: A Constant Tensor. The tensor will be created with the passed values with type *dtype*.
2. A computation graph can be launched using `tf.Session()`: A Session object. It creates the environment to run tensor operations and encapsulates within a session object

3. Mathematical operations are implemented using `tf.math`. It contains APIs for all mathematical operations such as add, sub, multiply, divide, negate and trigonometric functions as well with syntax, just as in the math module of any programming language.
4. The `tf.layers` contains APIs for all possible neural network layers like dense layer, convolutional layers, pooling layers, recurrent layers, dropout layers, normalization layers, etc.
5. Loss functions such as absolute difference, cosine difference, mean squared error (MSE), entropy loss, and log loss are available in `tf.losses` package.
6. The `tf.metrics` contains APIs for all evaluation metrics such as accuracy, mean distances, squared errors, etc.

For more details on APIs and syntaxes, visit www.tensorflow.org.

6.5.1.2 Setting up TensorFlow

Setting up TensorFlow is simple. Install Python 3 along with pip manager. Using Python's pip, TensorFlow can be directly downloaded; the versions of dependencies will be auto-managed. There are packages such as `tensorflow-gpu`, `tf-nightly` and `tf-nightly-gpu` that are preferred over the basic package (TensorFlow) depending on requirement such as whether GPU or nightly build is needed or not. Nightly builds are pre-compiled versions of any new changes made and are built and released every night.

Other than Python, there is language support for Java, C++, Go, JS and Swift.

6.5.1.3 Implementing TensorFlow

Let us see some examples in the following subsections to learn how to use TensorFlow package in Python and create deep learning applications.

Create Constants

Creating constants is very useful in TensorFlow while dealing with neural networks. `tf.constant` is used to create constants and then `tf.Session` is called to launch the computational graph with a session. To run this piece of code, `tf.run` is used so that the graph is evaluated.

```
#Import tensorflow
import tensorflow as tf
#Create a constant
hello = tf.constant('Hello, TensorFlow!')
#Launch the computational graph
sess = tf.Session()

#Evaluate the graph
print(sess.run(hello))
```

The output of the below program is

```
b'Hello, TensorFlow!'
```

Addition Program

Addition is a basic but important computation in TensorFlow. Addition and multiplication are the core operations of tensor manipulations. Create two tensor nodes 'a' and 'b' using `tf.constant`. Create an operation `add` that takes in two nodes and returns the sum as a node (`tf.add`). This forms the edges. Create another node 'c' to get the result. Run the above graph created using `tf.Session` and `tf.run`.

The following code segment shows how to implement these steps.

```
#Import tensorflow
import tensorflow as tf

#Create computational graph
a = tf.constant(1, dtype=tf.int32)
b = tf.constant(2, dtype=tf.int32)
c = tf.add(a, b)

#Execute the graph
ses = tf.Session()
print("Sum of node1 and node2 is:", ses.run(node3))
ses.close()
```

Neural Network Model

Let us create a simple multilayer perceptron for 10-class classification problem. This network having 512 neurons (or units) in the input layer followed by another fully connected layer with the 512 units is implemented as follows:

```
layer_1 = tf.layers.dense(x, 512) #x is input data
layer_2 = tf.layers.dense(layer_1, 512)
```

This is followed by the output layer with 10 units. The number of units in the output layer should be equal to the number of classes (in this case, 10 classes). This configuration is either by trial-and-error only or it can be referenced from similar previous implementations.

```
out_layer = tf.layers.dense(layer_2, 10)
```

Each node in the output layer calculates the corresponding probabilities (or likelihoods) of each class. The node with the maximum probability is the class predicted. The maximum likelihood is calculated using `tf.argmax` and `tf.nn.softmax` functions.

```

pred_classes = tf.argmax(logits, axis=1) #Get predicted classes
pred_probab = tf.nn.softmax(logits) #Get probabilities of
predictions

```

Once this is done, loss function, evaluation metric and optimizer are chosen. This multiclass classification (10-class classification) problem uses cross entropy loss (`tf.nn.sparse_softmax_cross_entropy_with_logits`) and gradient descent optimizer (`tf.train.GradientDescentOptimizer`). The aim of any deep learning application is to reduce the error which can be achieved using `tf.reduce_mean`.

```

#Add cross-entropy loss function
loss_op = tf.reduce_mean(
    tf.nn.sparse_softmax_cross_entropy_with_logits(
        logits=logits, labels=tf.cast(labels, dtype=tf.int32))
    )

optimizer = tf.train.GradientDescentOptimizer (
    learning_rate=learning_rate
    ) #Add Gradient Descent optimizer

```

Accuracy metric can be chosen for classification problem to check how accurate the model is, using `tf.metrics.accuracy`. The `tf.estimator.Estimator` function or class is responsible for evaluating part of the computation. All these metrics are set using `tf.estimator.EstimatorSpec` and the estimator is called. The complete code is as follows:

```

#Function creates the neural network architecture
def neural_net(x_dict):
    x = x_dict['images'] #Input data

    #A fully-connected layer with 512 neurons or units
    layer_1 = tf.layers.dense(x, 512)

    #Another fully-connected layer with 512 units
    layer_2 = tf.layers.dense(layer_1, 512)

    #Final fully-connected reg. layer with 10 units
    out_layer = tf.layers.dense(layer_2, 10)
    return out_layer

#Function that completes the net with loss and optimization def
model_fn(features, labels, mode):
    logits = neural_net(features)

```

```

pred_classes = tf.argmax(logits, axis=1) #Get predicted
classes
pred_probab = tf.nn.softmax(logits) #Get probabilities of
predictions

#Add cross-entropy loss function
loss_op =
tf.reduce_mean(tf.nn.sparse_softmax_cross_entropy_with_log-
its( logits=logits, labels=tf.cast(labels, dtype=tf.int32)))

optimizer = tf.train.GradientDescentOptimizer (learning_
rate=learning_rate) #Add Gradient Descent optimizer

train_op = optimizer.minimize(loss_op,global_step=tf.train.
get_global_step())

acc_op = tf.metrics.accuracy(labels=labels,
predictions=pred_classes)
#Set accuracy metric

estim_specs = tf.estimator.EstimatorSpec(
    mode=mode,
    predictions=pred_classes,
    loss=loss_op,
    train_op=train_op,
    eval_metric_ops={'accuracy': acc_op}) #Set the specifications

return estim_specs

model = tf.estimator.Estimator(model_fn)

```

6.5.1.4 Enactments in Other Languages

TensorFlow is usually used with powerful graphics processing unit (GPU) systems with Python support. But there are cases in machine learning that need a powerful framework like TensorFlow, so more adaptable implementations were released. They include the following:

1. **TensorFlow.js:** A JavaScript library capable of running in a browser, and still be able to host machine learning applications.
2. **Swift for TensorFlow:** Swift is an Apple-developed programming language for any Apple device. Being a first-class language, Swift needs advanced performance tools such as availability checking, deployment support, shape checking and named dimensions.
3. **TensorFlow Lite:** It is a simpler implementation of TensorFlow that is not taxing, and can provide a better real-time experience for mobile devices. It also enables machine learning in embedded computing.

6.5.1.5 Advantages

1. The entire computation scheme is graph based, so we get a better view of what is happening.
2. Has a highly pipelined structure. Can be used with NVIDIA GPU to improve performance.
3. Easier to execute and test subgraphs (subgraphs mean a part of the entire application).
4. Scalable and seamlessly managed.

6.5.1.6 Disadvantages

1. No support for non-NVIDIA GPUs.
2. Lag in computation speed with respect to similar packages.
3. Less support for Windows.
4. Lack of symbolic loops.

6.5.2 | Keras

Keras is an open-source, high-level deep learning library written in Python. It was initially developed as a part of the research project called ONEIROS (Open-ended Neuro-Electronic Intelligent Robot Operating System). It is built on top of TensorFlow, CNTK, and Theano, and so it can be used with any of these backends. It enables programmers to write convolutional networks, recurrent networks or a combination of both. It can run seamlessly on both CPU and GPU. Following are the characteristics of Keras:

1. **Turns Models into Products:** Keras models can be easily deployed across a wide number of platforms compared to any other frameworks. They can be deployed on iOS via Apple's CoreML, on Android via TensorFlow Android Runtime, in the browser via GPU-accelerated runtimes like Keras.js, on Google Cloud via TensorFlow-Serving, in a Python web app backend like Flask, on JVM via DL4J by SkyMind and on Raspberry Pi.
2. **Multiple Backends:** As already known, Keras is implemented using three backends (as of now), namely, TensorFlow, CNTK and Theano. Amazon is now working on implementing Keras using MXNet. Keras models can also be trained on a variety of hardware like NVIDIA GPUs, Google TPUs and OpenCL-enabled GPUs rather than just being a CPU.
3. **Multi-GPU Support:** Keras has built-in support for 'Multi-GPU data parallelism', for example, Horovod from Uber, which is a distributed training framework. Keras models can be converted into TensorFlow estimators (a class that encapsulates training, evaluation, prediction and export options) and trained on Google Cloud. It can also run on Spark via Dist-Keras.
4. **Support from Giants:** Keras development is supported by IT Giants like Google (TensorFlow backend), Amazon AWS (MXNet backend), Microsoft (CNTK backend) and other companies as shown in Fig. 6.6.



FIGURE 6.6 Companies supporting Keras development.

6.5.2.1 Features of Keras

1. **User Friendliness:** Keras is designed such that programmers can use it effortlessly. It reduces the cognitive load by offering simple APIs, reducing the number of user actions for common tasks, and throwing clear errors.
2. **Modularity:** A model is a sequence or graph of standalone modules that can be connected together with certain possible restrictions. For example, neural layers, cost functions, optimizers, initialization schemes, activation functions and regularization schemes are standalone Keras modules but can be combined to create user-specific models.
3. **Extensibility:** New modules can be easily added by means of writing classes and functions.
4. **Pythonic:** Models are not written in any model description language but in Python, hence making debugging and extending easier.

6.5.2.2 Setting Up Keras

1. Keras can be simply downloaded using *pip*:

```
sudo pip install keras
```

2. Or, GitHub repo can be cloned by

```
git clone https://github.com/keras-team/keras.git
cd keras
sudo python setup.py install
```

6.5.2.3 Configuration

By default, TensorFlow is the tensor manipulation backend for Keras. However, it can be configured to any other backends. This is done as follows:

1. Locate Keras configuration file by

```
$HOME/.keras/keras.json
```

Use %USERPROFILE% instead of \$HOME for Windows.

2. The default configuration looks like

```
{
  "image_data_format": "channels_last",
  "epsilon": 1e-07,
```

```

    "floatx": "float32",
    "backend": "tensorflow"
}

```

3. Simply change the 'backend' field to any desired backend like *tensorflow*, *cntk*, or *theano*. It can be done at the command line by defining the environment variable.

```
KERAS_BACKEND=tensorflow python -c "from keras import backend"
```

The change will take effect from the next run.

6.5.2.4 Keras API

Keras APIs are the same irrespective of the backend used. They only influence the way the tensors are manipulated. Some important APIs are listed below in the order in which they are used in a program:

1. Create a sequential model so that the layers can be appended, using `keras.models.Sequential()`: A Sequential model object.
2. Neural net layers are available in `keras.layers` package. Some layers available are Dense, Conv1D (or Conv2D or Conv3D), MaxPooling1D (or MaxPooling2D or MaxPooling3D), RNN, SimpleRNN, Dropout, and Flatten.
3. Compile the completed model using `model_object.compile(...)` method. It mostly accepts loss function, optimizer, and metrics which are available as `keras.losses`, `keras.optimizer`, and `keras.metrics` respectively. These can be passed as strings also.
4. To look into the architecture, use `model_object.summary()` which displays a table-like architecture summary. An example is shown in Fig. 6.7.

For more details, visit www.keras.io.

Layer (type)	Output Shape	Param #
fc1 (Dense)	(None, 32)	1632
act-relu (Activation)	(None, 32)	0
fc2 (Dense)	(None, 10)	330
act-softmax (Activation)	(None, 10)	0
Total params: 1,962		
Trainable params: 1,962		
Non-trainable params: 0		

FIGURE 6.7 Summary of neural network explained in the implementation section.

6.5.2.5 Implementing Keras (with TensorFlow Backend)

Simple Neural Network

The code shown below implements a two-layer fully-connected neural network with two activations *relu* and *softmax*. A keras model is an organization of the keras layers with activations and optimizers. A *Sequential model* is a model that allows users to linearly stack up the layers needed.

```
model = Sequential()
```

Since it is a multilayer perceptron, fully-connected neurons (units) are required. So, a *Dense* layer is added with 32 neurons in the input layer accepting the inputs of 50 features. *Activation* layers are used to apply activation functions on a layer. They can even be added by passing a value for *activation* argument of any layer API.

```
#A Fully connected network with 32 units
model.add(Dense(32, name = 'fc1', input_shape = (50, )))
model.add(Activation('relu', name = 'act-relu'))
```

Another fully-connected layer of 10 units is added on the output side. Note that the number of units in the output layer should be equal to the number of classes.

```
#A Fully connected network with 10 units
model.add(Dense(10, name = 'fc2'))
#Softmax activation on the final layer
model.add(Activation('softmax', name = 'act-softmax'))
```

The keras models need to be compiled before training and testing. So, compile the model using the *compile* method before using the model.

```
model.compile(optimizer='rmsprop', loss='categorical_
crossentropy',
              metrics=['accuracy'])
```

There are different loss functions available in Keras like categorical cross entropy, MSE, mean absolute error and so on where each application requires different loss function based on its nature. The architecture summary of this net using `model.summary()` is already shown in Fig. 6.7.

```
from keras.models import Sequential
from keras.layers import Dense, Activation
```

```
#Create a model instance
model = Sequential()

#Stack up models
#A Fully connected network with 32 units
model.add(Dense(32, name = 'fc1', input_shape = (50, )))
#Rectified Linear Unit activation
model.add(Activation('relu', name = 'act-relu'))
#A Fully connected network with 10 units
model.add(Dense(10, name = 'fc2'))
#Softmax activation on the final layer model.
add(Activation('softmax', name = 'act-softmax'))

#Compile the model
"""For a multi-class classification problem. Use "binary_
crossentropy" for binary classification"""
model.compile(optimizer='rmsprop',
              loss= 'categorical_crossentropy',
              metrics=['accuracy'])
```

Two-Dimensional Convolutional Neural Network (2D CNN)

The two-dimensional convolutional neural networks (2D CNNs) intake inputs of two dimensions. They can take input images of resolution equal to width $\times$ height. Several convolution, pooling, and dense layers are stacked up to build the 2D CNN architecture. Dropout layers are used in between to drop some of the feature vectors off the training process to avoid overfitting. So, a *Sequential* model is created and a 2D convolutional layer is added with 32 units accepting input images of dimension 100×100 with 3 channels (RGB color image) and with a 3×3 kernel. It is followed by a pooling layer of 2×2 kernel size (the dimensions of the kernels are taken arbitrarily).

```
model = Sequential()
model.add(Conv2D(32, kernel_size = (3, 3), activation='relu',
                input_shape=(100, 100, 3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

A dropout layer is added with 25% of data being dropped out randomly in each epoch.

```
model.add(Dropout(0.25))
```

Another pack of convolution-pooling-dropout layers is added.

```
#A Convolutional layer with 3 x 3 kernel
model.add(Conv2D(64, kernel_size = (3, 3), activation='relu'))
#A Maxpooling layer with 2 x 2 kernel
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
```

Before giving the features extracted by convolutional layers to dense layers, the input should be flattened (using *Flatten* layer) because the convolved features will be multidimensional.

```
model.add(Flatten())
```

Compile the model at last to use it.

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Dropout, Dense,
    Activation, Flatten
from keras.optimizers import SGD
#Create a model instance
model = Sequential()
#Stack up models
model.add(Conv2D(32, kernel_size = (3, 3), activation='relu',
    input_shape=(100, 100, 3))) #A Convolutional layer with 3x3 kernel
#A Maxpooling layer with 2x2 kernel
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25)) #A Dropout to avoid overfitting
#A Convolutional layer with 3x3 kernel
model.add(Conv2D(64, kernel_size = (3, 3), activation='relu'))
#A Maxpooling layer with 2x2 kernel
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25)) #A Dropout to avoid overfitting
model.add(Flatten()) #Flatten the layers before Fully connected
layers
#A Fully connected layer with 256 units
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
#A Fully connected layer with 10 units
model.add(Dense(10, activation='softmax'))
#Compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

Recurrent Neural Network (RNN)

Recurrent neural networks (RNNs) are sequential models used to model the sequence problems like speech recognition, time series prediction and so on. Many recurrent layers are available in Keras library like SimpleRNN, RNN, LSTM, GRU, etc., and some wrapper layers like Bidirectional. The following code shows a simple RNN. A *SimpleRNN* layer with 5 units is added accepting inputs of 5 features.

```
model = Sequential()
model.add(SimpleRNN(5, input_shape = (5, 5)))
```

Since it is a regression problem, one unit is enough at the output side.

```
model.add(Dense(1)) #A Fully connected layer with 1 unit
```

A sigmoid (or logistic regression) activation is used since it is generally with regression problems solved using RNN. Loss functions are the general metric for regression so that MSE loss is used up here.

```
model.compile(loss='mean_squared_error', optimizer='adam')
```

The complete code is given below.

```
from keras.models import Sequential
from keras.layers import SimpleRNN, Dense, Activation

#Create a model instance
model = Sequential()

#Stack up models
model.add(SimpleRNN(5,
input_shape = (5, 5))) #A Simple RNN with 5 units
model.add(Dense(1)) #A Fully connected layer with 1 unit
model.add(Activation('sigmoid')) #A Sigmoid activation on the
output

#Compile the model
model.compile(loss='mean_squared_error', optimizer='adam')
```

6.5.3 | PyTorch

PyTorch is a popular open-source deep learning framework. It was first released in October 2016. The name PyTorch was inspired by the Torch open-source machine learning framework, which

was written in Lua programming language. Torch was released way back in 2002 and was the most widely used deep learning framework of the time. Torch provided numerous deep learning algorithms based on LuaJIT scripting language supported by the underlying C implementation. The major hurdle faced by developers in effectively using Torch was learning Lua language. Lua was not powerful enough for complex artificial intelligence (AI) application development and it does not provide modularity required to interface with other libraries.

PyTorch was developed by Facebook's AI research team. The AI developers inspired by Torch's programming style decided to implement in Python and called it PyTorch. Similar to Lua, Python is a scripting language but it is more flexible and provides support to a wide range of machine learning and deep learning libraries. In addition to functionalities of Torch, PyTorch provides new features that make it capable to handle complex AI development.

6.5.3.1 Features

Two key features of PyTorch that help it handle complex learning tasks easily are imperative programming and dynamic computation graphs. These features of PyTorch have their roots in the properties of Python. PyTorch Dynamic Graphs can be built and rebuilt as necessary at runtime. Imperative programming allows performing computation as the model is being built. There is no distinction between defining the computation graph and compiling the computation graph for imperative programs.

1. **Imperative programming:** An imperative program performs computation when the program is being typed. In contrast to symbolic programming, which tells what task the program should do or what goals it should achieve without focusing much on how to do them, imperative programming focuses more on describing how the program works to achieve a goal. In symbolic programming, the program is converted into a computation or symbolic graph and actual computation occurs only at the last step in the execution. TensorFlow makes use of symbolic programming. Imperative programs are more flexible as computation occurs immediately when a statement is typed.
2. **Dynamic computation graphs:** PyTorch makes use of dynamic computation graphs and is defined by the run. It generates dynamic computation graphs to handle the changes with each run. Dynamic graphs are useful when the amount of work or computation that needs to be done is variable. This feature of PyTorch is useful for debugging when the model fails. Dynamic graphs work well with models of any complexity like with RNN.

6.5.3.2 Ecosystem

PyTorch provides a well-loaded ecosystem of tools and libraries that can be used for various AI applications. A few popular libraries are:

1. **AllenNLP:** An open-source library for natural language processing (NLP) research and applications built on PyTorch.

2. **GpyTorch:** It is a Gaussian process library developed by researchers at Cornell University using PyTorch to help build scalable, flexible Gaussian process models.
3. **Horovod:** A library for distributed training in deep learning frameworks.

A few popular tools are:

1. **Glow:** A machine learning compiler and execution engine that can perform state-of-the-art compiler optimizations and code generation for neural network graphs.
2. **TensorLy:** A high-level API for tensor methods and neural networks in Python.
3. **Pyro:** A universal probabilistic programming language (PPL) to build an elastic and expressive deep probabilistic model, thereby implementing the Bayesian model for deep learning.

6.5.3.3 PyTorch Implementation

The Python program given below is an implementation of a neural network using PyTorch. This neural network has an input layer with 10 units, a hidden layer with 5 units and a single output unit with a learning rate of 0.01.

The program starts with importing the PyTorch library and the Autograd package which will let the network automatically implement gradient calculation and backpropagation. The Autograd package provides classes and functions implementing automatic differentiation of arbitrary scalar-valued functions. It is enough to declare the tensors for which gradient should be computed, and the Autograd package performs the tensor differentiation.

The variables N , D_{in} , H and D_{out} indicate batch size, input dimension, hidden layer dimension and output dimension, respectively. The variables x , y are tensors to hold dummy inputs and outputs of the network. The network works with random training data. The next step is to define the model. By using sequential model, layers are stacked up in a linear fashion. This code uses rectified linear unit (ReLU) activation function in the hidden layer and sigmoidal activation in the output layer. It is also possible to define more complex models.

```
Model = torch.nn.Sequential(*args)
```

We then pass the layers to the constructor in the order they need to be added to the model. The next step is to define the loss function and optimizer. MSE loss function (`torch.nn.MSELoss()`) and stochastic gradient descent (`torch.optim.SGD()`) with a learning rate of 0.01 optimizer are used. Then train the model for 50 epochs. Each iteration performs forward propagation, loss computation, backward propagation and parameter updation sequentially.

`y_pred` holds the predicted values of each epoch and `y` holds the target value, which is used to compute the loss. To prevent previous gradients from accumulating in each step, `optimizer.zero_grad()` clears out the gradients. `loss.backward()` uses Autograd package to compute gradients for all tensors using dynamic computation graphs. This performs the backpropagation step.

Finally, the `optimizer.step()` updates all parameters using new gradients. At the end of all iterations, a simple two-layer neural network is trained based on the given target values.

```

#Import pytorch library
import torch
import torch.nn as nn

#Set the datatype of tensor
dtype = torch.FloatTensor
#Set the dimensions
N, D_in, H, D_out = 10, 10, 5, 1

#Define input and output variables
x = Variable(torch.randn(N,D_in).type(dtype), requires_
grad=False)
y = Variable(torch.randn(N,D_out).type(dtype),
requires_grad=False)

model = nn.Sequential(nn.Linear(D_in, H), nn.ReLU(),nn.
Linear(H, D_out), nn.Sigmoid())

criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
total_iteration = 50

for epoch in range(total_iteration):
    # Forward Propagation
    y_pred = model(x)

    # Compute and print loss
    loss = criterion(y_pred, y)
    print('epoch: ', epoch, ' loss: ', loss.item())

    # Zero the gradients
    optimizer.zero_grad()

    # perform a backward pass (backpropagation)
    loss.backward()

    # Update the parameters
    optimizer.step()

```

6.6 | HARDWARE SUPPORT FOR DEEP LEARNING

Neural networks involve processor intensive operations such as linear algebraic computations (e.g., dot product), matrix multiplications, repeated computations with a large amount of data, etc. So, dedicated hardware is required for these operations.

Many advancements have come to improve the functionality of the CPU from GPU to tensor processing unit (TPU), which is explained in the following subsections (Fig. 6.8).

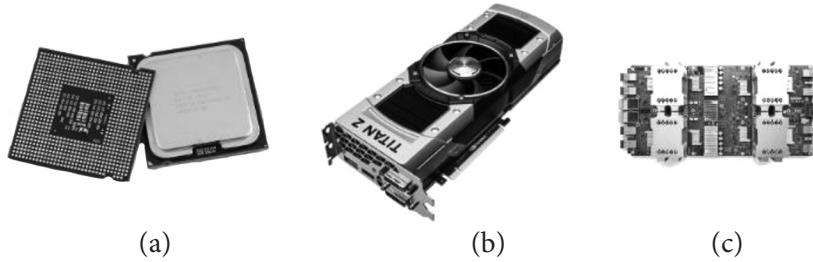


FIGURE 6.8 (a) CPU, (b) GPU and (c) TPU.

6.6.1 | Central Processing Unit

Central processing unit, commonly abbreviated as CPU, is the brain of a computer. It is an electronic circuit capable of performing basic arithmetic, logical, and control operations including input/output operations. Any work that is submitted to CPU is split into jobs which comprise a set of tasks. To improve the performance, multiple cores are introduced. Multi-core systems perform consecutive execution which achieves throughput as high as parallel processing systems. Depending on the number of threads that the kernel (Operating System) supports, the number of tasks that can be simultaneously executed can be determined. Another factor that determines the capability of the CPU is the clock speed, that is, the number of operations that can be executed per unit time. It is commonly measured in Gigahertz (GHz) or sometimes FLOPS (Floating Point Operations Per Second) since floating-point operations take more time than integer-based operations. However, at any time, only one task can be executed at the CPU.

Intel and AMD are the two most popular CPU manufacturers (Fig. 6.9) for PCs and laptops. The generations of Intel processors can be found from the name of the processors, for example, 'i3', 'i5', 'i7', or 'i9'. Intel i9-9900K is the ninth generation processor. The seventh generation AMD-A series is the latest APU (integrated CPU and GPU) of AMD processors.

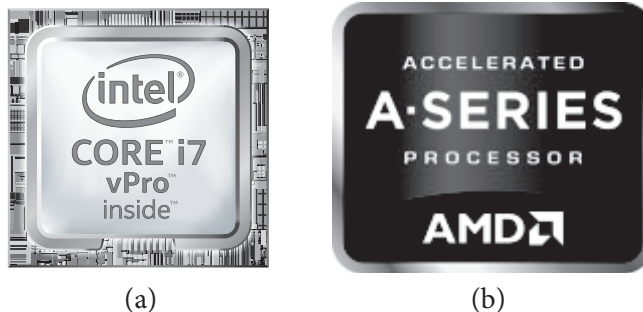


FIGURE 6.9 (a) Intel Core i7; (b) AMD A-series.

6.6.2 | Graphics Processing Unit

CPU is very fast when computations are small in number. However, when it comes to deep learning involving matrix multiplication of hundreds and thousands of numbers, the CPU struggles.

All gaming applications perform such massive matrix multiplications to render landscapes fair and square. They are designed to handle lots of parallel processing using thousands of cores. They have a high memory bandwidth to deal with the colossal amount of data. Because of this performance improvement, GPU is introduced in the field of deep learning to aid the CPU. The GPU is also called graphics card in gaming. The factors determining the speed of GPUs are DirectX support, manufacturer, video card bus type, and clock speed.

DirectX (Microsoft) is the collection of APIs for dealing with multimedia and game programming for Microsoft platforms. The minimum support for the graphics card is version 9 or greater for the best possible performance. The popular GPU manufacturers are AMD, Intel and NVIDIA. The choice of the manufacturer mostly depends on the architecture. For example, using AMD graphics card works better with AMD processor. NVIDIA is the dedicated GPU that outperforms the rest as it is a dedicated GPU and is not a co-processor like the previous example of AMD. Video card buses provide a way to connect the graphics card to the motherboard. PCIe (peripheral components interface – express) is the newer slot to install GPUs, being twice faster than the older Advanced Graphics Port (AGP) type slots. The GPU clock speed depends on the GPU architecture. So, higher clock speeds do not guarantee higher execution speed.

NVIDIA TITAN V and NVIDIA Quadron GV100 are desktop accelerators whereas NVIDIA DGX Systems and NVIDIA Tesla Solutions (Fig. 6.10) are datacenter solutions. Some other desktop NVIDIA GPUs are Vanta, GeForce 256 series, GeForce 2xx, GeForce 3xx and GeForce FX series. Many other workstation GPUs are also available such as Quadro CX, Quadro FX, Quadro Plex, Quadro2 and Quadro4. All GPUs are categorized and listed in <https://www.nvidia.com/page/legacy.html>.

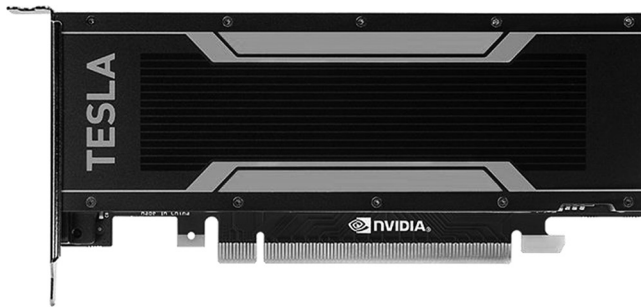


FIGURE 6.10 NVIDIA Tesla GPU.

6.6.3 | VPU (VISION PROCESSING UNIT)

Vision Processing Unit (VPU) is a dedicated highly parallel vector processor which produces high quality of images with ambient light and reduces the power consumption by freeing up the CPU and GPU space thereby improving the overall performance of the system. VPU is a tiny GPU which works with the power envelop of less than 1W. It accelerates the performance of deep learning models for machine vision tasks like object detection, 3D mapping.

VPU's global market can be segmented based on its usage in smartphones, digital cameras, automobiles, wearable devices, drones, robots and medical devices. The key players using VPU are Movidius Inc, Synopsys Inc and Media Tech.

Two major Vision Processing Unit developed by Movidius, an Intel company are:

1. Intel Movidius Myriad X VPU [Fig. 6.11(a)].
2. Intel Movidius Myriad 2 VPU [Fig. 6.11(b)].

6.6.3.1 | Intel Movidius Myriad X VPU

It is the third generation and most advanced VPU from Movidius, an Intel Company. It features Neural Compute Engine – a dedicated hardware accelerator for deep neural network inferences.

6.6.3.2 | Intel Movidius Myriad 2 VPU

It is the second generation VPU from Movidius, an Intel Company. It provides high performance with low power. Industry proven performance on computer vision and AI is achieved by this VPU.

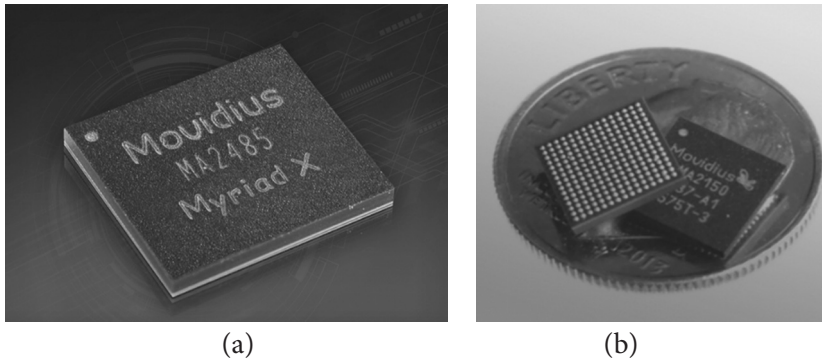


FIGURE 6.11 (a) Intel Movidius Myriad X VPU; (b) Intel Movidius Myriad 2 VPU.

6.6.4 | NEURAL COMPUTE STICK (NCS)

The NCS is tiny and used to learn AI programming on huge range of host devices (Fig. 6.12). NCS is a modular, deep learning accelerator in a standard USB3.0 stick. NCS supports TensorFlow and Caffe. The most popular NCS is the Intel Movidius Neural Compute Stick (NCS). It is the world's first USB-based Deep Learning Inference Kit. It allows anyone to test and deploy AI models at the edges locally. The models trained using these networks need to be converted into a proper format to run on the device. Using NCS SDK, we can test whole host of deep learning application on the device with a few lines of code. NCS SDK has three libraries that work for this:

1. Compiler – converts the model into an appropriate format.
2. Profiler – gives layer by layer statistics to evaluate performance.
3. Checker – compares the results by running on the device VC pure TensorFlow or Caffe.

The approximate cost of NCS is 79USD and NCS has been used in smart security cameras, gesture controlled drones and industrial machine vision equipment.

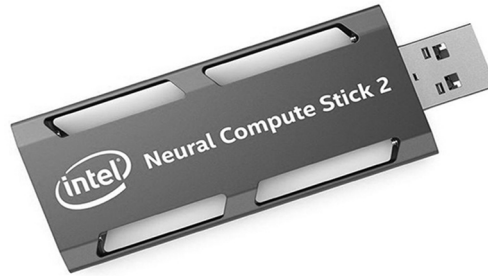


FIGURE 6.12 Neural compute stick.

6.6.5 | Tensor Processing Unit

Tensor processing unit (TPU), shown in Fig. 6.13, is an AI accelerator, a custom application-specific integrated chip (ASIC) designed by Google for machine learning applications. Major products of Google such as Translate, Photos, Gmail and Search assistant are powered by TPUs. It is specifically designed for supporting TensorFlow, which involves the flow of tensors. Google has recently announced its second generation TPUs that run on the cloud. Google builds “TPU pods” from the network available from the 64 second-generation TPUs providing up to 11.5 petaflops. It is a coprocessor so that it cannot execute codes on its own but support the CPU to do computations faster.

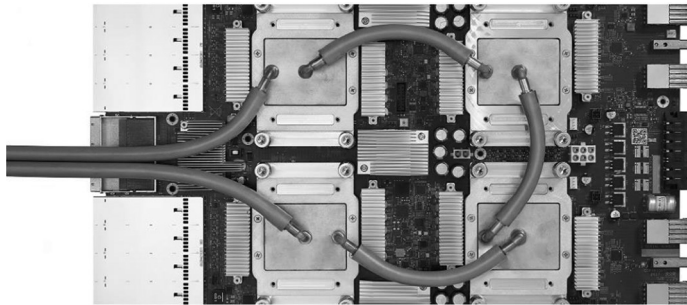


FIGURE 6.13 Google TPU.

SUMMARY

This chapter introduces a high-level programming language Python and the frameworks that are built for it. The chapter further explains rich machine learning and deep learning frameworks supported by tech giants. The notable ones include TensorFlow from Google, CNTK from Microsoft, Keras, and PyTorch. It concludes by explaining the new hardware advancements developed to support deep learning.

REVIEW QUESTIONS

1. Why is Python the most preferred language for working on deep learning applications?
2. List a few deep learning libraries, along with the features they provide.
3. How do you choose a deep learning framework for your application?
4. List the unique features provided by TensorFlow, Keras, and PyTorch.
5. How are CPU, GPU, TPU different?
6. Is GPU and TPU a must for deep learning applications? Explain.
7. Why is moving AI to the edge important in the current scenario?
8. Since the Neural Compute Stick is powered by Vision Processing Unit, will it be suitable for users who want to use text or audio data?

ASSIGNMENT PROBLEMS

1. Given two n -dimensional NumPy arrays, perform dot product and element-wise multiplication between them and find the difference between the two.
2. Develop a web-app-based face recognition application using TenorFlow.js.
3. Implement a multilayer perceptron using Keras and analyze the hyper-parameters for different non-linear activations.
4. Develop a simple application that detects pedestrian from video surveillance using PyTorch.
5. Find the difference in time taken for training a model using CPU and TPU.
6. Develop an application for image classification using the Neural Compute Stick.

REFERENCES

1. <https://blog.slavv.com/picking-a-gpu-for-deep-learning-3d4795c273b9>
2. <https://software.intel.com/en-us/movidius-ncs>
3. <https://www.geekboots.com/story/cpu-vs-gpu-vs-tpu>
4. <https://www.marutitech.com/top-8-deep-learning-frameworks/>
5. <https://www.movidius.com/myriad2>
6. <https://www.movidius.com/myriadx>
7. https://www.tensorflow.org/api_docs
8. <https://www.tutorialspoint.com/python/>

Applications of Deep Learning

LEARNING OBJECTIVES

After reading this chapter, you will be able to

- Understand how to approach a deep learning solution.
- Identify what kind of neural network can be used for a given problem.
- Get an exposure to practical applications of neural networks.

7.1 | INTRODUCTION

Deep learning algorithms are one of the ways of solving artificial intelligence problems (subfields) including Natural Language Processing (NLP), Automatic Speech Recognition (ASR), Human-Computer Interaction (HCI) and expert systems. Developing deep learning solutions for several problems require the following steps to be followed in sequence:

1. Data collection and cleaning
2. Pre-processing
3. Model building
4. Training and testing

Each phase is significant to obtain accurate results. This chapter explains some example problems in those subfields and how to implement them using deep learning frameworks discussed in Chapter 6.

7.2 | IMAGE CLASSIFICATION USING CNN

7.2.1 | Problem Description

Convolutional neural networks (CNNs) are very similar to ordinary neural networks. They are made up of neurons that have learnable weights and biases. Each neuron receives some inputs, performs a dot product and optionally follows it with a non-linear activations. CNN architectures make the

explicit assumption that the inputs are images, which allows us to encode certain properties into the architecture. These then make the forward function more efficient to implement and vastly reduce the number of parameters in the network.

We know that the machine's perception of an image is completely different from what we see. In fact, it is only numbers that machines see in an image. Each pixel in the image is given a value between 0 and 255. Thus, for the machine to classify any image, it requires some preprocessing for finding patterns or features that distinguish an image from another. The following is an image classification problem making use of the convolutional neural network in Keras. The dataset used here is CIFAR-10 which can be found at <https://www.cs.toronto.edu/~kriz/cifar.html>

7.2.2 | Language and Framework

Language used: Python

Framework used: Keras using TensorFlow backend and implemented on Jupyter Notebook, Anaconda IDE.

7.2.3 | Dataset

The CIFAR-10 dataset contains 60,000 32×32 color images in 10 classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images. The dataset is divided into five training batches and one test batch, each with 10,000 images. The test batch contains exactly 1,000 randomly selected images from each class. The training batches contain the remaining images in random order, but some training batches may contain more images from one class than another. Between them, the training batches contain exactly 5,000 images from each class.

The classes are completely mutually exclusive. There is no overlap between automobiles and trucks. 'Automobile' includes sedans, SUVs and so on. 'Truck' includes only big trucks. 'Neither' includes pickup trucks.

Figure 7.1 shows the classes in the dataset as well as 10 random images from each.

7.2.4 | Code Snippets

These code snippets are based on Keras CNN-CIFAR/keras-cnn-cifar10.ipynb.

```
from keras.datasets import cifar10
(train_images, train_labels), (test_images, test_labels) =
cifar10.load_data()
```

Initially, we need to import the data. The CIFAR-10 dataset comes bundled with Keras. We will use the `load_data()` function to load it into memory.

```
classes = np.unique(train_labels)
no_of_classes = len(classes)
```



FIGURE 7.1 Labels in CIFAR-10 dataset.

Then we explore the data and find the number of images in training and test data and the number of classes. Once loading and exploring of data is completed, the data is preprocessed. Preprocessing of data includes normalization of data and converting the labels to categorical variables.

```
no_of_rows,no_of_cols,no_of_dims = train_im.shape[1:]
training_Data = train_im.reshape(train_im.shape[0], no_of_rows,
no_of_cols, no_of_dims)
testing_Data = test_im.reshape(test_im.shape[0], no_of_rows,
no_of_cols, no_of_dims)
input_shape = (no_of_rows, no_of_cols, no_of_dims)

training_Data = training_Data.astype('float32')
testing_Data = testing_Data.astype('float32')
```

```

training_Data /= 255
testing_Data /= 255

train_labels = to_categorical(train_lb)
test_labels = to_categorical(test_lb)

```

We find the shape of input images and create the variable *input_shape*. The datatype is changed to float and the data is scaled to lie between 0 and 1. The labels are converted to categorical data. Then we need to define the model. We can create the model with a convolutional layer, max pooling layer, dropout layer and dense layer. The first two layers are processed with 32 filters of window size 3×3 .

```

def createModel():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), padding='same',
        activation='relu',
        input_shape=input_shape))
    model.add(Conv2D(32, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3, 3), padding='same',
        activation='relu'))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3, 3), padding='same',
        activation='relu'))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Flatten())
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(no_of_classes, activation='softmax'))

    return model

```

After defining the model, we need to train the model. Here the model is trained for 50 epochs with a batch size of 256.

```

model_one = createModel()
batch_size = 256
epochs = 50
model_one.compile(optimizer='rmsprop', loss=
'categorical_crossentropy',
metrics=['accuracy'])
model_one.summary()
history = model_one.fit(training_Data, train_labels,
batch_size=batch_size, epochs=epochs, verbose=1, validation_
data=(testing_Data, test_labels))
model_one.evaluate(test_data, test_labels)

```

The architecture of CNN used in this example is shown in Fig. 7.2.

Layer (type)	Output shape	Param #
conv2d_13 (Conv2D)	(None, 32, 32, 32)	896
conv2d_14 (Conv2D)	(None, 30, 30, 32)	9248
max_pooling2d_7 (MaxPooling2D)	(None, 15, 15, 32)	0
dropout_9 (Dropout)	(None, 15, 15, 32)	0
conv2d_15 (Conv2D)	(None, 15, 15, 64)	18496
conv2d_16 (Conv2D)	(None, 13, 13, 64)	36928
max_pooling2d_8 (MaxPooling2D)	(None, 6, 6, 64)	0
dropout_10 (Dropout)	(None, 6, 6, 64)	0
conv2d_17 (Conv2D)	(None, 6, 6, 64)	36928
conv2d_18 (Conv2D)	(None, 4, 4, 64)	36928
max_pooling2d_9 (MaxPooling2D)	(None, 2, 2, 64)	0
dropout_11 (Dropout)	(None, 2, 2, 64)	0
flatten_3 (Flatten)	(None, 256)	0
dense_4 (Dense)	(None, 512)	131584
dropout_12 (Dropout)	(None, 512)	0
dense_5 (Dense)	(None, 10)	5130
Total params: 276,138		
Trainable params: 276,138		
Non-trainable params: 0		

FIGURE 7.2 Architecture of CNN used in the example.

7.2.5 | Complete Code

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from __future__ import print_function
import keras
from keras.models import Sequential
from keras.layers import Dense, Conv2D, MaxPooling2D, Dropout, Flatten

from keras.datasets import cifar10
(train_images, train_labels), (test_images, test_labels) = cifar10.load_data()

from keras.utils import to_categorical

print('Shape of Training : ', train_images.shape, train_labels.shape)
print('Shape of Testing data : ', test_images.shape, test_labels.shape)

# Find the unique numbers from the train labels
classes = np.unique(train_labels)
no_of_Classes = len(classes)
print('Total number of outputs : ', no_of_Classes)
print('Output classes : ', classes)

"""Find the shape of input images and create the variable input_shape"""
no_of_rows, no_of_cols, no_of_dims = train_images.shape[1:]
training_Data = train_images.reshape(train_images.shape[0], no_of_rows, no_of_cols, no_of_dims)
testing_Data = test_images.reshape(test_images.shape[0], no_of_rows, no_of_cols, no_of_dims)
input_shape = (no_of_rows, no_of_cols, no_of_dims)

# Change the datatype to float
training_Data = training_Data.astype('float32')
testing_Data = testing_Data.astype('float32')

# Scale the data to lie between 0 to 1
training_Data /= 255
testing_Data /= 255

# Change the labels from integer to categorical data
train_Labels = to_categorical(train_labels)
test_Labels = to_categorical(test_labels)
```

```
# Display the change for category label using one-hot encoding
print('Original label 0 : ', train_labels[0])
print('On conversion to categorical ( one-hot ) : ', train_
Labels[0])

def createModel():
    model = Sequential()
    # The first two layers with 32 filters of window size 3x3
    model.add(Conv2D(32, (3, 3), padding='same',
activation='relu',
input_shape=input_shape))
    model.add(Conv2D(32, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3, 3), padding='same',
activation='relu'))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3, 3), padding='same',
activation='relu'))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.25))

    model.add(Flatten())
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(no_of_Classes, activation='softmax'))

    return model

model_one = createModel()
batch_size = 3
epochs = 3
model_one.compile(optimizer='rmsprop', loss=
'categorical_crossentropy',
metrics=['accuracy'])
model_one.summary()
history = model_one.fit(training_Data, train_Labels, batch_
size=batch_size, epochs=epochs, verbose=1, validation_
data=(testing_Data, test_Labels))
model_one.evaluate(testing_Data, test_Labels)
```

```

model_one.predict(testing_Data)

# Loss Curves
plt.figure(figsize=[8,6])
plt.plot(history.history['loss'],'r',linewidth=3.0)
plt.plot(history.history['val_loss'],'b',linewidth=3.0)
plt.legend(['Training loss', 'Validation Loss'],fontsize=18)
plt.xlabel('Epochs',fontsize=16)
plt.ylabel('Loss',fontsize=16)
plt.title('Loss Curves',fontsize=16)

# Accuracy Curves
plt.figure(figsize=[8,6])
plt.plot(history.history['acc'],'r',linewidth=3.0)
plt.plot(history.history['val_acc'],'b',linewidth=3.0)
plt.legend(['Training Accuracy', 'Validation Accuracy'],fontsize=18)
plt.xlabel('Epochs',fontsize=16)
plt.ylabel('Accuracy',fontsize=16)
plt.title('Accuracy Curves',fontsize=16)

```

7.2.6 | Result

Each value in the array represents the category of that particular image in the test dataset. Figure 7.3 represents the output results and Fig. 7.4 represents the loss and accuracy curves.

```

array([[3.4562217e-03, 9.2505850e-04, 1.8809618e-02, ..., 8.5772546e-03,
        7.6413718e-03, 3.8437799e-04],
       [2.4021919e-01, 3.6076340e-01, 3.5939459e-03, ..., 3.2659632e-04,
        3.5520118e-01, 3.5589766e-02],
       [2.6662815e-01, 5.5819660e-02, 1.6485509e-01, ..., 1.3124739e-02,
        2.5609285e-01, 3.0395191e-02],
       ...,
       [1.8850275e-03, 1.5383511e-04, 6.3279994e-02, ..., 5.5726457e-02,
        4.1047897e-04, 3.2096161e-04],
       [5.3246684e-02, 5.1997840e-02, 6.4273000e-02, ..., 7.0452951e-02,
        2.2547310e-03, 3.1353589e-03],
       [3.5260079e-04, 9.4345105e-06, 3.8702232e-03, ..., 8.7736481e-01,
        1.4745045e-06, 1.3149938e-05]], dtype=float 32)

```

FIGURE 7.3 Integer values of predicted classes.

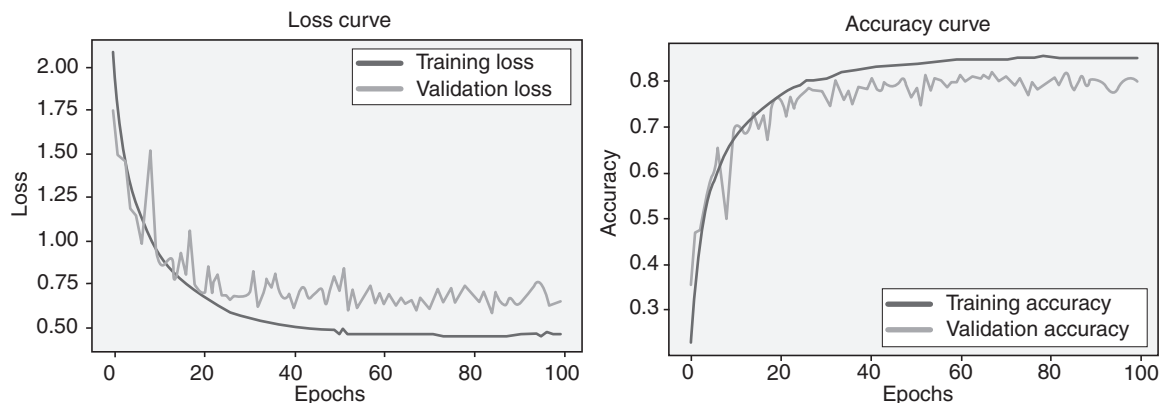


FIGURE 7.4 Loss and accuracy curves.

7.3 | VISUAL SPEECH RECOGNITION USING 3D-CNN

7.3.1 | Problem Description

Let us consider the following scenario: A person is mute and he wants to communicate. Just imagine if there is a machine that visualizes his lips movements and converts it into text! Visual speech recognition aims at transcribing lip movements into readable text. The system will convert an input video into transcribed text. The technique uses classification of visemes into texts; visemes are particular lip moments that can be converted into text. In recent years, there have been many advances in automatic speech reading system with the inclusion of audio and visual speech features to recognize words even under noisy conditions. The proposed system focuses mainly on the visual element as a support for the acoustic segment. It classifies a set of visemes to text. The system has two modules. The first one handles extraction of lip features from input while the second is a neural network system trained to process the lip visemes and classify into text.

7.3.2 | Language and Framework

Language used: Python

Framework used: Keras using TensorFlow backend and implemented on Spyder, Anaconda IDE.

7.3.3 | Dataset

MIRACL-VC1 is a lip-reading dataset that has both depth and color images. It can be used for diverse research fields like visual speech recognition, face detection and biometrics. Here, 15 speakers (5 men and 10 women) positioned in the frustum of an MS Kinect sensor have uttered a set of 10 words and 10 phrases, 10 times. Each instance of the dataset consists of a synchronized sequence of color and depth images (both of 640×480 pixels). The MIRACL-VC1 dataset contains a total number of 3,000 instances ($15 \times 10 \times 10$). In the proposed system, only the color images are used. The sample color and depth image frames from MIRACL-VC1 dataset is shown in Fig. 7.5. This example uses only the words from the dataset.



FIGURE 7.5 Sample color and depth image frames from MIRACL-VC1.

Note: Color image available at <https://www.wileyindia.com>

7.3.4 | Code Snippets

The video cannot directly be processed. It is first broken into individual frames and the lip region alone is extracted from all the frames. It is known that frame rate (number of frames per second) differs for each video. All videos are equalized to have the same frame rate, say, 30 fps, before processing to make further processing easier. A sample lip region extraction for the word ‘choose’ is shown in Fig. 7.6.



FIGURE 7.6 Seven visemes of the second utterance of choose.

Note: Color image available at <https://www.wileyindia.com>

To fix a standard number of visemes for each input, a dummy viseme image of closed mouth is padded before processing.

```
def append_frames(self, im_seq, max_frame, counter):
    """Pads a dummy frame to make fixed number of visemes
    for an input"""

    dummy_viseme = misc.imresize(misc.imread(self.in_path
+ '/../dummy.jpg', mode = 'F', flatten = True), size= (30,
48))/255
    for i in range(counter, max_frame):
        im_seq[i] = dummy_viseme
    return im_seq
```

The label is the one-hot encoded representation of words themselves that are encoded as integers.

```
def make_one_hot_encoding(self, Y):
    """Encodes string labels into one-hot version"""
    label_encoder = sk_LabelEncoder()
    integer_encoded = label_encoder.fit_transform(Y)
    onehot_encoder = sk_OneHotEncoder(sparse = False)
    integer_encoded = integer_encoded.reshape(len(integer_
encoded), 1)
    onehot_encoded = onehot_encoder.fit_transform(integer_
encoded)
    return onehot_encoded, label_encoder
```

Once the X and Y data are ready, the three-dimensional CNN (3D-CNN) can be built using the Keras library. A CNN is called *3D-CNN* when it processes 3D inputs. The input is of the dimension (15, 30, 48), that is, there are 15 visemes appended together, each of size 48×30 .

```
def build_net(self, load_name = None):
    """Loads the saved model or builds a new one if not built
already"""
    if load_name:
        self.model = load_model('{} .h5'.format(load_name))
    else:
        self.model.add(Conv3D(name = 'conv1',
filters = 16,
                                kernel_size = 3,
                                strides = 1,
                                input_shape = (15, 30, 48, 1),
                                data_format = 'channels_last',
                                activation = 'relu'))
        self.model.add(MaxPooling3D(name = 'pool1',
pool_size = (1, 3, 3),
strides = (1, 2, 2)))
        self.model.add(Conv3D(name = 'conv2',
filters = 32,
                                kernel_size = 3,
                                strides = 1,
                                data_format = 'channels_last',
                                activation = 'relu'))
        self.model.add(MaxPooling3D(name = 'pool2',
pool_size = (1, 3, 3),
strides = (1, 2, 2)))
        self.model.add(Conv3D(name = 'conv3',
filters = 64,
                                kernel_size = 3,
```

```

        strides = 1,
        data_format = 'channels_last',
        activation = 'relu'))
self.model.add(MaxPooling3D(name = 'pool3',
                             pool_size = (1, 3, 3),
                             strides = (1, 2, 2)))
self.model.add(Flatten(name = 'flatten'))
self.model.add(Dense(256, name = 'fc5'))
self.model.add(Dense(10, name = 'fc6'))
self.model.add(Dense(10, name = 'fc7'))
self.model.add(Activation('softmax', name = 'softmax'))

```

The built model is trained and tested using the data extracted and preprocessed from the MIRACL-VC1 dataset.

```

#Training CNN
cnn = CNN()
cnn.build_net()
print(cnn.summary())
cnn.train(train_X, train_Y, epochs = 20, validation_split = 0.2)

#Testing CNN
pred = cnn.test(test_X)
dec_pred = feat.decode_one_hot(pred, label_encoder).
reshape(len(test_X))
dec_truth = feat.decode_one_hot(test_Y, label_encoder).
reshape(len(test_X))

#Evaluating the performance of the result
cnn.evaluate(test_X, test_Y)
print('Predicted vs Ground truth labels')
for t, g in zip(dec_pred, dec_truth):
    print('%-10s %-5s %-10s'%(labels[t], '=>', labels[g]))

```

7.3.5 | Complete Code

7.3.5.1 *makefeatures.py*

This module contains a *FeatureGen* class providing all the required functions including feature extraction (*make_features*), appending dummy viseme to make the number of frames same for all features (*append_frames*), encode them in one-hot representation (*make_one_hot_encoding*), and decode back to labels (*decode_one_hot*).

```

from re import match
from scipy import misc
from sklearn.preprocessing import LabelEncoder as sk_LabelEncoder, OneHotEncoder as sk_OneHotEncoder
from numpy import array as np_array
import numpy as np
import os

class FeatureGen:
    def __init__(self, type_):
        if type_ not in ('words', 'phrases'):
            raise ValueError('unsupported type')
        self.type = type_
        self.X = np_array(eval('[[[0.0]*48]*30]*15']*150'))
        self.Y = np_array(eval('"01"*150'))

    def get_color_images(self, images):
        return list(filter(lambda file:
            bool(match('^color__\d{3}\.jpg$', file)), images))

    def make_features(self, in_path, verbose = True):
        i = 0
        cwd = os.getcwd()
        in_path += '/words' if self.type == 'words' else '/phrases'
        self.in_path = in_path
        os.chdir(in_path)
        contents = os.listdir(os.getcwd())
        print('Words' if self.type == 'words' else 'Phrases')
        for content in contents:
            j = 0
            if verbose == True:
                print('Word' if self.type == 'words' else
                    'Phrase', content)
            utterances = os.listdir('./' + content)
            for utterance in utterances:
                if verbose == True:
                    print('Reading utterance', utterance)
                counter = 0
                for image in self.get_color_images(os.
                    listdir('./' + content + '/' + utterance)):

```

```

        self.X[i][j] = misc.imresize(misc.
imread('./' + content + '/' + utterance + '/' + image, mode =
'F', flatten = True), size = (30, 48))/255
        counter += 1
        if counter < 15:
            self.X[i] = self.append_frames(self.X[i],
15, counter)
            j += 1
        self.Y[i] = content
        i += 1
    os.chdir(cwd)
    return (self.X, self.Y)

    def append_frames(self, im_seq, max_frame, counter):
        dummy_viseme = misc.imresize(misc.imread(self.in_path
+ '/../dummy.jpg', mode = 'F', flatten = True), size= (30,
48))/255
        for i in range(counter, max_frame):
            im_seq[i] = dummy_viseme
        return im_seq

    def make_one_hot_encoding(self, Y):
        label_encoder = sk_LabelEncoder()
        integer_encoded = label_encoder.fit_transform(Y)
        onehot_encoder = sk_OneHotEncoder(sparse = False)
        integer_encoded = integer_encoded.reshape(len(integer_
encoded), 1)
        onehot_encoded = onehot_encoder.fit_transform(integer_
encoded)
        return onehot_encoded, label_encoder

    def decode_one_hot(self, predictions, label_encoder):
        dec = []
        for i in range(len(predictions)):
            dec.append(label_encoder.inverse_transform([np.
argmax(predictions[i])]))
        return np.array(dec)

```

7.3.5.2 *cnn3d.py*

This module contains a *CNN* class that provides functions to build the model (*build_net*), train it (*train*), test (*test*) and evaluate the performance (*evaluate*), and save the classification report (*save_report*).

```

import keras
from keras.models import Sequential, load_model
from keras.layers import Dense, Activation, Flatten
from keras.layers import Conv3D, MaxPooling3D
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
from IPython import get_ipython
get_ipython().run_line_magic('matplotlib', 'inline')

class CNN:
    def __init__(self):
        print('---3D CNN for Visual Speech Recognition---')
        self.model = Sequential()
        self.history = None

    def build_net(self, load_name = None):
        if load_name:
            self.model = load_model('{} .h5'.format(load_name))
        else:
            self.model.add(Conv3D(name = 'conv1',
                                   filters = 16,
                                   kernel_size = 3,
                                   strides = 1,
                                   input_shape = (15, 30, 48, 1),
                                   data_format = 'channels_last',
                                   activation = 'relu'))
            self.model.add(MaxPooling3D(name = 'pool1',
                                         pool_size = (1, 3, 3),
                                         strides = (1, 2, 2)))
            self.model.add(Conv3D(name = 'conv2',
                                   filters = 32,
                                   kernel_size = 3,
                                   strides = 1,
                                   data_format = 'channels_last',
                                   activation = 'relu'))
            self.model.add(MaxPooling3D(name = 'pool2',
                                         pool_size = (1, 3, 3),
                                         strides = (1, 2, 2)))
            self.model.add(Conv3D(name = 'conv3',
                                   filters = 64,
                                   kernel_size = 3,
                                   strides=1,
                                   data_format = 'channels_last',
                                   activation = 'relu'))
            self.model.add(MaxPooling3D(name = 'pool3',

```

```

        pool_size = (1, 3, 3),
        strides = (1, 2, 2)))
self.model.add(Flatten(name = 'flatten'))
self.model.add(Dense(256, name = 'fc5'))
self.model.add(Dense(10, name = 'fc6'))
self.model.add(Dense(10, name = 'fc7'))
self.model.add(Activation('softmax', name = 'softmax'))

self.model.compile(loss = keras.losses.categorical_
crossentropy,
                    optimizer = keras.optimizers.Adam(),
                    metrics = ['accuracy'])

def train(self, X, Y, epochs, validation_split, save =
False, name = None):
    print('---Training CNN3D---')
    self.history = self.model.fit(x = X,
        y = Y,
        epochs = epochs,
        validation_split = validation_split).history
    if save:
        self.model.save('{} .h5'.format(name))

acc = self.history['acc']
val_acc = self.history['val_acc']
loss = self.history['loss']
val_loss = self.history['val_loss']
epochs = list(range(len(acc)))
plt.plot(epochs, acc, 'b', label = 'Training accuracy')
plt.plot(epochs, val_acc, 'g', label = 'Validation
accuracy')
plt.title('Training and Validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(loc = 'upper left')
plt.figure()
plt.plot(epochs, loss, 'b', label = 'Training loss')
plt.plot(epochs, val_loss, 'g', label = 'Validation loss')
plt.title('Training and Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')

```

```

plt.legend(loc = 'upper left')
plt.show()

def test(self, X, steps = None):
    print('---Testing CNN3D---')
    predictions = self.model.predict(x = X,
                                     steps = steps)
    print('Predictions: ', predictions.tolist())
    return predictions

def evaluate(self, X, Y):
    print('---Evaluating CNN3D---')
    eval_ = self.model.evaluate(x = X, y = Y)
    print('Test loss:', eval_[0])
    print('Test accuracy:', eval_[1])

def save_report(self, name, test_y, predictions,
               num_classes):
    target_names = ['Class {}:'.format(i) for i in range(1,
                                                       num_classes)]
    with open(name, 'w') as report:
        report_ = classification_report(test_y, predictions,
                                       target_names = target_names)
        report.write(report_)
        print('Report saved successfully')

def summary(self):
    return self.model.summary()

```

7.3.5.3 lip_reading.py (Driver code)

This is the driver code using up all the functionalities defined in the modules to classify visemes to text.

```

from models.cnn3d import CNN
from preprocess.makefeatures import FeatureGen

#Dictionary mapping Word IDs to words
labels = {'01': 'begin',
          '02': 'choose',
          '03': 'connection',
          '04': 'navigation',

```

```

        '05': 'next',
        '06': 'previous',
        '07': 'start',
        '08': 'stop',
        '09': 'hello',
        '10': 'web' }

#Inputting and extracting features
input_path = 'C:/Users/SandeepArockia/Desktop/Code/data3/F01'
feat = FeatureGen(type_ = 'words')
(train_X, train_Y) = feat.make_features(input_path)
train_X = train_X.reshape(train_X.shape + (1, ))
train_Y, label_encoder = feat.make_one_hot_encoding(train_Y)
test_X, test_Y = train_X[71:121], train_Y[71:121]

#Training CNN
cnn = CNN()
cnn.build_net()
print(cnn.summary())
cnn.train(train_X, train_Y, epochs = 20, validation_split = 0.2)

#Testing CNN
pred = cnn.test(test_X) dec_pred = feat.decode_one_hot(pred,
label_encoder).reshape(len(test_X))
dec_truth = feat.decode_one_hot(test_Y, label_encoder).
reshape(len(test_X))

#Evaluating the performance of the result
cnn.evaluate(test_X, test_Y)
print('Predicted vs Ground truth labels')
for t, g in zip(dec_pred, dec_truth):
    print('%-10s %-5s %-10s'%(labels[t], '=>', labels[g]))

```

7.3.6 | Result

Independent lip reading systems do not provide a higher accuracy. These systems are usually used as a support for speech-to-text systems. As can be seen from the graphs shown in Figs. 7.7 and 7.8, the performance is less compared to speech-to-text systems. But these models can be made perform better when complemented with next word predictors.

Figure 7.9 gives a sample output that shows some predicted labels against their true labels.

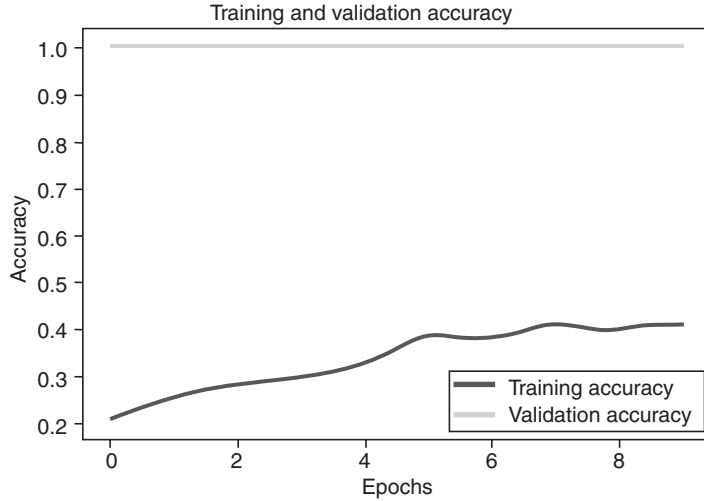


FIGURE 7.7 Accuracy graph.

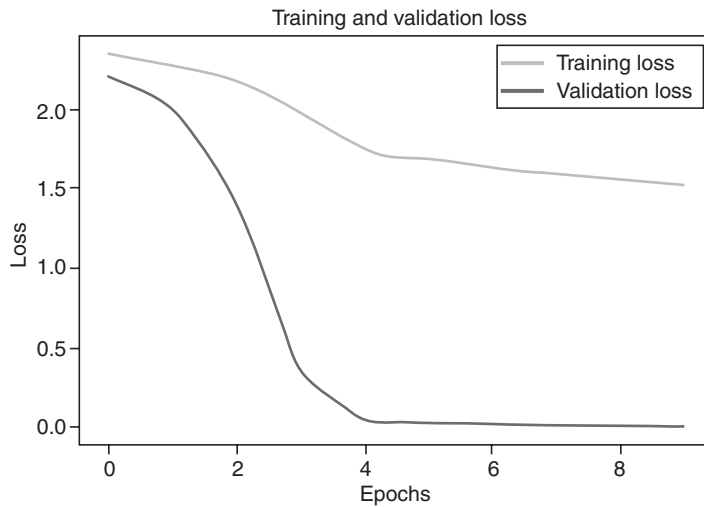


FIGURE 7.8 Loss graph.

7.4 | STOCK MARKET PREDICTION USING RECURRENT NEURAL NETWORK

7.4.1 | Problem Description

The economy of a country greatly depends upon the stock exchanges of that country. Every change in the economy reflects on the share prices and, inversely, the rise and fall in share prices determines the inflation or deflation for that economy. Henceforth, prediction of stock prices is an indispensable task. Analysis involves predicting accurately the future closing values of a stock by using the historical data

Predicted vs ground truth labels

hello	=>	stop
hello	=>	stop
next	=>	stop
hello	=>	stop
hello	=>	stop
next	=>	stop
hello	=>	stop
hello	=>	stop
web	=>	stop
web	=>	hello
hello	=>	hello
choose	=>	hello
next	=>	hello
hello	=>	hello
hello	=>	hello
next	=>	hello
hello	=>	hello

FIGURE 7.9 Sample output with predicted and ground truth labels.

available. The project aims at using neural networks for prediction. Here, we have used a long short-term memory (LSTM) network model (discussed in Chapter 3) capable of learning from time series data.

7.4.2 | Language and Framework

Language used: Python

Framework used: Keras using TensorFlow backend and implemented on Jupyter notebook, Anaconda IDE.

7.4.3 | Dataset

The data used for this project has been pulled from Google Finance Python API. It consists of data points which are indexed in order of time. This dataset has been chosen for the ease of its reusability and reproducibility. The sample dataset is of the following form:

Date	Open	High	Low	Close	Volume
30-Jun-17	943.99	945.00	929.61	929.68	2287662
29-Jun-17	951.35	951.66	929.60	937.82	3206674
28-Jun-17	950.66	963.24	936.16	961.01	2745568

The multiple variables in the dataset are Date, Open, High, Low, Close, Volume.

1. **Open:** Starting price at which stock is traded on a particular day.
2. **Close:** Final price at which stock is traded on a particular day.
3. **High:** Maximum price of the share for the day.
4. **Low:** Minimum price of the share for the day.
5. **Volume:** Total number of shares processed in the particular day.

7.4.4 | Code Snippets

Download the dataset and store as *google.csv*. We will use this dataset for further prediction. We will use the *read_csv()* function to read the data.

```
data = pd.read_csv('google.csv')
```

Once the data is loaded, we need to preprocess it. Preprocessing involves removal of less important features and thereby retaining only the important features. It is followed by normalization of data.

```
def remove_data(data):
    # Define columns of data to keep from historical stock data
    number = []
    openvalue = []
    closevalue = []
    volume = []
    """Loop through the stock data objects backwards and store
    factors we want to keep"""
    i = 0
    for i in range(len(data) - 1, -1, -1):
        number.append(i)

        openvalue.append(data['Open'][i])
        closevalue.append(data['Close'][i])
        volume.append(data['Volume'][i])
        i += 1
    # Create a data frame for stock data
    stockdata = pd.DataFrame()
    # Add factors to data frame
    stockdata['Number'] = number
    stockdata['Open'] = openvalue
    stockdata['Close'] = pd.to_numeric(closevalue)
    stockdata['Volume'] = pd.to_numeric(volume)
    return stockdata

def get_normalised_data(data):
    # Initialize a scaler, then apply it to the features
    scaler = MinMaxScaler()
    numerical = ['Open', 'Close', 'Volume']
    data[numerical] = scaler.fit_transform(data[numerical])
    return data
```

The dataset is split into the training (82.95%) and test (17.05%) datasets for LSTM model. This involves unrolling of recurrent neural network (RNN).

```
def train_test_split_lstm(stocks, prediction_time=1, test_data_size=450, unroll_length=50):
    # training data
    test_data_cut = test_data_size + unroll_length + 1
    x_train = stocks[0:-prediction_time - test_data_cut].as_matrix()
    y_train = stocks[prediction_time:-test_data_cut]['Close'].as_matrix()
    # test data
    x_test = stocks[0 - test_data_cut:-prediction_time].as_matrix()
    y_test = stocks[prediction_time - test_data_cut:]['Close'].as_matrix()
    return x_train, x_test, y_train, y_test

def unroll(data, sequence_length=24):
    result = []
    for index in range(len(data) - sequence_length):
        result.append(data[index: index + sequence_length])
    return np.asarray(result)
```

The LSTM model is built and implemented using Keras.

```
def build_model(input_dim, output_dim, return_sequences):
    model = Sequential()
    model.add(LSTM(
        input_shape=(None, input_dim),
        units=output_dim,
        return_sequences=return_sequences))
    model.add(Dropout(0.2))
    model.add(LSTM(128, return_sequences=False))
    model.add(Dropout(0.2))
    model.add(Dense(
        units=1))
    model.add(Activation('linear'))
    return model
```

We now need to train our model. Built-in library functions are used to train the model.

```
model.fit(X_train,
        y_train,
        batch_size=batch_size,
        epochs=epochs,
        verbose=2,
        validation_split=0.05
    )
```

The outcome of the model is predicted using built-in library functions.

```
predictions = model.predict(X_test, batch_size=batch_size)
```

Finally, we calculate the test score of the LSTM model.

```
trainScore = model.evaluate(X_train, y_train, verbose=0)
print('Train Score: %.5f MSE (%.5f RMSE)' % (trainScore,
math.sqrt(trainScore)))
testScore = model.evaluate(X_test, y_test, verbose=0)
print('Test Score: %.5f MSE (%.5f RMSE)' % (testScore,
math.sqrt(testScore)))
```

7.4.5 | Complete Code

```
import pandas as pd
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import math
from IPython.display import display
from keras.layers.core import Dense, Activation, Dropout
from keras.layers.recurrent import LSTM
from keras.models import Sequential
from keras.metrics import mean_squared_error
from sklearn.model_selection import StratifiedKFold
import time
from keras.layers.core import Dense, Activation, Dropout
from keras.layers.recurrent import LSTM
from keras.models import Sequential
import matplotlib.pyplot as plt
```

```

plt.rcParams['figure.figsize'] = (18, 12)
def remove_data(data):
    # Define columns of data to keep from historical stock data
    number = []
    openvalue = []
    closevalue = []
    volume = []
    """Loop through the stock data objects backwards and store
    factors we want to keep"""
    i= 0
    for i in range(len(data) - 1, -1, -1):
        number.append(i)
        openvalue.append(data['Open'][i])
        closevalue.append(data['Close'][i])
        volume.append(data['Volume'][i])
    i+= 1
    # Create a data frame for stock data
    stockdata = pd.DataFrame()
    # Add factors to data frame
    stockdata['Number'] = number
    stockdata['Open'] = openvalue
    stockdata['Close'] = pd.to_numeric(closevalue)
    stockdata['Volume'] = pd.to_numeric(volume)
    return stockdata

def get_normalised_data(data):
    # Initialize a scaler, then apply it to the features
    scaler = MinMaxScaler()
    numerical = ['Open', 'Close', 'Volume']
    data[numerical] = scaler.fit_transform(data[numerical])
    return data

def train_test_split_lstm(stocks, prediction_time=1, test_data_
size=450, unroll_length=50):
    # training data
    test_data_cut = test_data_size + unroll_length + 1
    x_train = stocks[0:-prediction_time - test_data_cut].as_
matrix()
    y_train = stocks[prediction_time:-test_data_cut]['Close'].
as_matrix()
    # test data
    x_test = stocks[0 - test_data_cut:-prediction_time].as_
matrix()

```

```

        y_test = stocks[prediction_time - test_data_cut:]
    ['Close'].as_matrix()
    return x_train, x_test, y_train, y_test

def unroll(data, sequence_length=24):
    result = []
    for index in range(len(data) - sequence_length):
        result.append(data[index: index + sequence_length])
    return np.asarray(result)

def build_model(input_dim, output_dim, return_sequences):
    model = Sequential()
    model.add(LSTM(
        input_shape=(None, input_dim),
        units=output_dim,
        return_sequences=return_sequences))
    model.add(Dropout(0.2))
    model.add(LSTM(
        128,
        return_sequences=False))
    model.add(Dropout(0.2))
    model.add(Dense(
        units=1))
    model.add(Activation('linear'))
    return model

#Step 1:load the data
data = pd.read_csv('google.csv')
print(data.head())
"""Step 2: As the main objective of project is to predict the
closing value of the stock,only important features such as
open,close and volume is retained and the others are discarded"""
stockdata = remove_data(data)
#Print the dataframe head and tail
print("dataframe with only important features:")
print(stockdata.head())
print("----")
print(stockdata.tail())
#Step 3:Normalise the data values using MinMaxScaler from
sklearn
stockdata = get_normalised_data(stockdata)
print(stockdata.head())
#Step 4:Store the preprocessed data for future use
stockdata.to_csv('preprocesseddata.csv',index= False)

```

```

#LSTM MODEL:
#Step 1: Load the data to build model
stockdata = pd.read_csv('preprocesseddata.csv')
stockdata_data = stockdata.drop(['Number'], axis =1)
print("data considered for lstm model construction:")
display(stockdata_data.head())
"""Step 2 : Split train and test data sets and Unroll train and
test data for lstm model."""
X_train, X_test, y_train, y_test = train_test_split_
lstm(stockdata_data, 5)
unroll_length = 50
X_train = unroll(X_train, unroll_length)
X_test = unroll(X_test, unroll_length)
y_train = y_train[-X_train.shape[0]:]
y_test = y_test[-X_test.shape[0]:]
print("x_train", X_train.shape)
print("y_train", y_train.shape)
print("x_test", X_test.shape)
print("y_test", y_test.shape)
#Step 3:building a LSTM Model
# Set up hyperparameters
batch_size =10
epochs = 1
# build improved lstm model
model = build_model( X_train.shape[-1],output_dim = unroll_
length, return_sequences=True)
#compile the model
model.compile(loss='mean_squared_error', optimizer='adam')
#Step 4:Train the model
model.fit(X_train,
        y_train,
        batch_size=batch_size,
        epochs=epochs,
        verbose=2,
        validation_split=0.05
        )

# Generate predictions
predictions = model.predict(X_test, batch_size=batch_size)
#Step 5:Evaluation of the model
trainScore = model.evaluate(X_train, y_train, verbose=0)
print('Train Score: %.5f MSE (%.5f RMSE)' % (trainScore,
math.sqrt(trainScore)))
testScore = model.evaluate(X_test, y_test, verbose=0)
print('Test Score: %.5f MSE (%.5f RMSE)' % (testScore,
math.sqrt(testScore)))

```

7.4.6 | Result

The graph in Fig. 7.10 depicts the training and validation loss for the considered dataset of around 5 epochs. The performance improves with more number of epochs.

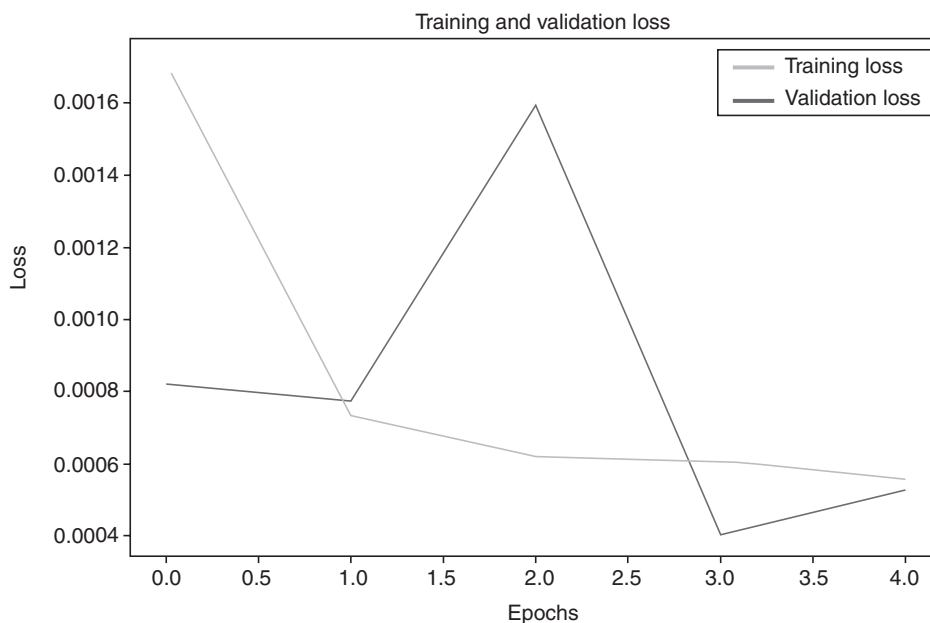


FIGURE 7.10 Loss graph.

The training score for this model is 0.01807 root mean square error (RMSE) and the test score is 0.04540 RMSE. The score can be improved by increasing the number of epochs. The presented model can be improved by incorporating a recommendation system feature into it as it will help an investor choose the best company to invest.

7.5 | NEXT-WORD PREDICTION USING RNN-LSTM

7.5.1 | Problem Description

The Next-Word Prediction is a word prediction problem. The word prediction problem comes in three levels: word completion, next-word prediction and multi-word prediction. This simple next-word prediction is the process of predicting the next word given a sequence of words, and is based on the previous history. This small application can easily be witnessed in smartphones and virtual keyboards in desktops. This next-word prediction problem is generally used to reduce the number of keystrokes. Since a sentence is a sequence of words, we use RNN to predict the next word.

7.5.2 | Language and Framework

Language used: Python

Framework used: Keras using TensorFlow backend and implemented on Spyder, Anaconda IDE.

7.5.3 | Dataset

Though there are many datasets available for NLP, a simple dataset is taken for this application. *The Hill of Dreams* written by Arthur Machen in 1907 (shown in Fig. 7.11) is taken as the dataset. For processing, the sentences are split into words and a sequence of these words are taken as the input and the next word is taken as the output (or label).

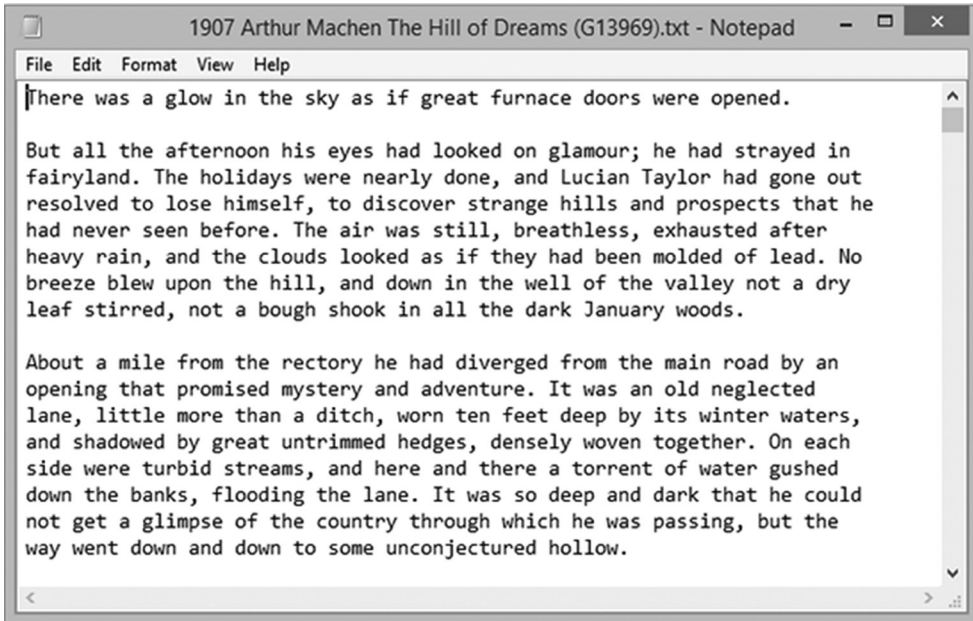


FIGURE 7.11 First few lines of the dataset.

7.5.4 | Code Snippets

The text cannot be processed directly by the neural network and so it needs to be converted into integers for processing and then converted back into words after processing. To accomplish this, two dictionaries (key:value pairs) are defined to map words to integers and vice versa.

```
word_to_int = dict()
int_to_word = dict()

def load_content(path_to_file):
    """Loads the file into memory and convert it for processing"""
    global word_to_int, int_to_word #Using global structures
    with open(path_to_file, 'r') as file:
        content = file.readlines() #Read file contents
```

```

content = ' '.join(content).lower()
#Extract only alphanumeric values
tokenizer = RegexpTokenizer(r'\w+')
#Split the sentences into words
content = tokenizer.tokenize(content)
#Populate dictionaries
word_to_int = {word:i for i, word in
enumerate(list(set(content)))}
int_to_word = {val:key for key, val in word_to_int.items()}
#Converting words to numbers
for i, word in enumerate(content):
    content[i] = word_to_int[word]
return content

```

Once the words are mapped into numbers, they need to be converted into $[X, Y]$ pairs so that the neural network model operates on them. A group of words is taken as X , where the length of the sequence is crucial. It is taken as 10 by default, but a length of 40 is considered in this application. It can also be taken as the average sentence length for better results.

```

def prepare_dataset(content, window = 10):
    """Converts the file data into a proper dataset as X and Y"""
    X = list() #Input
    Y = list() #Output
    content = content if len(content) > 3000 else
content[:3000] #Limiting the data for easier processing
    for i in range(0, len(content)-window, window):
        #One row contains a sequence of "window" words
        X.append(content[i: i + window])

        #Label is the next word to each of "window" words
        Y.append(content[i + window])
    #Converting into numpy arrays
    X = np.asarray(X, dtype = np.int32)
    Y = np.asarray(Y, dtype = np.int32)
    return X, Y

```

It is time to build the RNN-LSTM model. The input to the model is of dimension (1162, 40, 1), where 1162 is the number of instances, 40 is the number of words forming a row and 1 is the batch size. The final dense layer should consist of neurons equal to the number of classes, that is, 7555, the number of unique words. RNN has to predict one of these 7555 words.

```
def build_rnn_model(input_dim, n_output):
    """Create an RNN model with loss and optimizer"""
    #Create a sequential model to stack up layers
    model = Sequential()
    model.add(LSTM(128, input_shape = (input_dim, 1), name =
'input.lstm1')) #Add an LSTM layer with 128 units
    #Add a fully connected layer
    model.add(Dense(n_output, name = 'output.fc'))
    #Add a softmax activation on all outputs to get a final value
    model.add(Activation('softmax'))
    #Use RMSprop optimization return model
    model.compile(loss = 'categorical_crossentropy', optimizer
= RMSprop(lr = 0.01), metrics = ['acc'])
```

The model built should then be trained with the prepared dataset. The history object is saved for plotting of graphs afterwards.

```
def train_model(model, x_train, y_train, epochs = 5):
    """Train the RNN-LSTM with the training X and Y data"""
    #Save the history for plotting return hist
    hist = model.fit(x_train, y_train, epochs = epochs, valida-
tion_split = 0.25, verbose = 2).history
```

Visualization is the best tool for learning the response of the model. The Matplotlib library is used for plotting the accuracy and loss curves.

```
def visualize_result(hist):
    """Plotting the graph with accuracy and loss"""
    #Extracting the values
    acc = hist['acc']
    val_acc = hist['val_acc']
    loss = hist['loss']
    val_loss = hist['val_loss']
    epochs = list(range(len(acc)))
    #Plotting accuracies
    plt.plot(epochs, acc, 'b', label = 'Training accuracy')
    plt.plot(epochs, val_acc, 'g', label = 'Validation accuracy')
    plt.title('Training and Validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend(loc = 'upper left')
```

```
plt.figure()
#Plotting losses
plt.plot(epochs, loss, 'b', label = 'Training loss')
plt.plot(epochs, val_loss, 'g', label = 'Validation loss')
plt.title('Training and Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(loc = 'upper left')
plt.show()
```

The output data should be one-hot encoded to enable RNN to process them. One-hot encoding is a symmetric matrix whose order is equal to the number of classes, where only the position of the corresponding label is set (number 1) while the others remain unset (number 0). The following code is the sequence of the above function calls.

```
PATH_TO_FILE = '1907 Arthur Machen The Hill of Dreams (G13969).
txt' #Path to the dataset
data = load_content(PATH_TO_FILE) #Load the contents
num_classes = len(word_to_int) #Number of classes - unique
words

X, Y = prepare_dataset(data, window = 40) #Split into X and Y
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_
size = 0.3)
#Split into train and test sets
#Convert numerical values to one hot encoded vectors
y_train = to_categorical(y_train, num_classes = len(word_to_
int))
y_test = to_categorical(y_test, num_classes = len(word_to_int))

#Build the model
rnn = build_rnn_model(x_train.shape[1], num_classes)
print(rnn.summary())

#Reshape the inputs to (n_features, n_words, batch)
x_train = x_train.reshape(x_train.shape + (1,))
x_test = x_test.reshape(x_test.shape + (1,))

#Train the model and save the history object
hist = train_model(rnn, x_train, y_train, epochs = 20)
visualize_result(hist) #Plot graphs
```

```

y_pred = rnn.predict(x_test) #Predict test data
eval_ = rnn.evaluate(x_test, y_test) #Evaluate the score

#Print the loss and accuracy
print('Test loss:', eval_[0])
print('Test accuracy:', eval_[1])

```

7.5.5 | Complete Code

```

from keras.models import Sequential
from keras.layers import LSTM, Dense, Activation
from keras.optimizers import RMSprop
from keras.utils.np_utils import to_categorical
from sklearn.model_selection import train_test_split
from nltk.tokenize import RegexpTokenizer
import numpy as np
import matplotlib.pyplot as plt

word_to_int = dict()
int_to_word = dict()

def load_content(path_to_file):
    """Loads the file into memory and convert it for processing"""
    global word_to_int, int_to_word #Using global structures
    with open(path_to_file, 'r') as file:
        content = file.readlines() #Read file contents
        content = ' '.join(content).lower()
        #Extract only alphanumeric values
        tokenizer = RegexpTokenizer(r'\w+')
        #Split the sentences into words
        content = tokenizer.tokenize(content)
        #Populate dictionaries
        word_to_int = {word:i for i, word in enumerate(list(set(
            content))))
        int_to_word = {val:key for key, val in word_to_int.items()}
        #Converting words to numbers
        for i, word in enumerate(content):
            content[i] = word_to_int[word]
        return content

def prepare_dataset(content, window = 10):
    """Converts the file data into a proper dataset as X and Y"""

```

```
X = list() #Input
Y = list() #Output
content = content if len(content) > 3000 else content[:3000] #Limiting the data for easier processing
for i in range(0, len(content)-window, window):
    #One row contains a sequence of "window" words
    X.append(content[i: i + window])
    #Label is the next word to each of "window" words
    Y.append(content[i + window])
#Converting into numpy arrays
X = np.asarray(X, dtype = np.int32)
Y = np.asarray(Y, dtype = np.int32)
return X, Y

def build_rnn_model(input_dim, n_output):
    """Create an RNN model with loss and optimizer"""
    model = Sequential() #Create a sequential model to stack up
layers
    model.add(LSTM(128, input_shape = (input_dim, 1), name =
'input_lstm1')) #Add an LSTM layer with 128 units
    #Add a fully connected layer
    model.add(Dense(n_output, name = 'output.fc'))
    #Add a softmax activation on all outputs to get a final value
    model.add(Activation('softmax'))
    model.compile(loss = 'categorical_crossentropy', optimizer =
RMSprop(lr = 0.01), metrics = ['acc']) #Use RMSprop optimization
    return model

def train_model(model, x_train, y_train, epochs = 5):
    """Train the RNN-LSTM with the training X and Y data"""
    #Save the history for plotting
    hist = model.fit(x_train, y_train, epochs = epochs,
validation_split = 0.25, verbose = 2).history
    return hist

def visualize_result(hist):
    """Plotting the graph with accuracy and loss"""
    #Extracting the values
    acc = hist['acc']
```

```

val_acc = hist['val_acc']
loss = hist['loss']
val_loss = hist['val_loss']
epochs = list(range(len(acc)))
#Plotting accuracies
plt.plot(epochs, acc, 'b', label = 'Training accuracy')
plt.plot(epochs, val_acc, 'g', label = 'Validation accuracy')
plt.title('Training and Validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(loc = 'upper left')
plt.figure()
#Plotting losses
plt.plot(epochs, loss, 'b', label = 'Training loss')
plt.plot(epochs, val_loss, 'g', label = 'Validation loss')
plt.title('Training and Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(loc = 'upper left')
plt.show()

```

```

PATH_TO_FILE = '1907 Arthur Machen The Hill of Dreams (G13969).
txt' #Path to the dataset
data = load_content(PATH_TO_FILE) #Load the contents
#Number of classes - unique words
num_classes = len(word_to_int)

X, Y = prepare_dataset(data, window = 40) #Split into X and Y
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_
size = 0.3)
#Split into train and test sets
#Convert numerical values to one hot encoded vectors
y_train = to_categorical(y_train, num_classes = len(word_
to_int))
y_test = to_categorical(y_test, num_classes = len(word_to_int))

#Build the model
rnn = build_rnn_model(x_train.shape[1], num_classes)
print(rnn.summary())

#Reshape the inputs to (n_features, n_words, batch)
x_train = x_train.reshape(x_train.shape + (1,))
x_test = x_test.reshape(x_test.shape + (1,))

```

```
#Train the model and save the history object
hist = train_model(rnn, x_train, y_train, epochs = 20)
visualize_result(hist) #Plot graphs

y_pred = rnn.predict(x_test) #Predict test data
eval_ = rnn.evaluate(x_test, y_test) #Evaluate the score

#Print the loss and accuracy
print('Test loss:', eval_[0])
print('Test accuracy:', eval_[1])
```

7.5.6 | Result

Next word prediction systems, when trained for very long epochs, will produce very good accuracy. For this example, the model is trained for only 20 epochs.

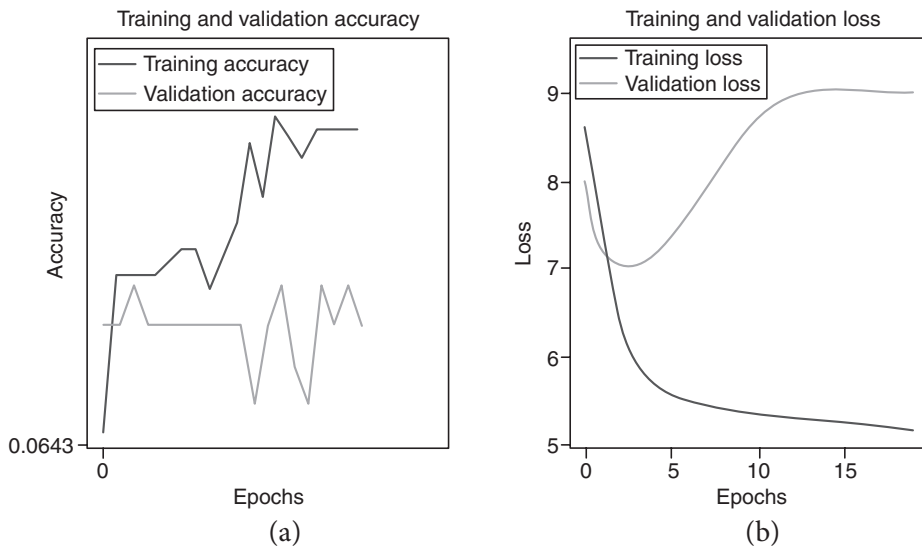


FIGURE 7.12 (a) Accuracy graph and (b) Loss graph.

From the graphs shown in Fig. 7.12, it can be shown that the model does not generalize well. The test accuracy is found to be around 7% as shown in Fig. 7.13, which is a decent score for 20 epochs. By changing the number of words forming an input to the average sentence length, tweaking parameters, and quality adding some more layers, the response can be improved further.

Test loss: 9.690076287970486

Test accuracy: 0.06827309238443892

FIGURE 7.13 Test scores: Loss and accuracy.

7.6 | TAMIL HANDWRITTEN CHARACTER OPTICAL RECOGNITION USING CRNN

7.6.1 | Problem Description

Handwritten text recognition is a challenging problem because of the large character set, diversity of handwriting styles and character touching problem. An optical character recognition (OCR) system for a language like Tamil may provide a solution to data entry problems in various domains like digitization of old government records, police files, accounts of small-scale industries, and digitization of other manuscripts. Though OCR for Tamil already exists for printed text, this application aims to extend that recognition to handwritten text with better accuracy by using convolutional recurrent neural network (CRNN) which consists of the convolutional layer, recurrent layers and transcription layer, for image-based sequence recognition. This application uses image processing to extract text from the input image and conglomerates deep learning concepts with image processing to identify Tamil characters, irrespective of the handwriting style. This project can also be extended to perform context recognition by identifying key phrases in the input text. Experiments show that by incorporating lexical constraints and prior knowledge about the language, the language model can further decrease its error rate by 2%–5% over segmentation–recognition based models.

7.6.2 | Language and Framework

Language used: Python

Framework used: PyTorch and implemented on Spyder, Anaconda IDE.

7.6.3 | Dataset

Isolated Handwritten Tamil Word Dataset and Isolated Handwritten Tamil Character collected by HP Labs India is used to train the OCR model. The dataset can be found in <http://lipitk.sourceforge.net/hpl-datasets.htm>. The character dataset consists of nearly 500 isolated samples, each of 156 Tamil characters, and the word dataset consists of nearly 100 handwritten Tamil words written by 131 authors.

7.6.4 | Code Snippets

These code snippets are based on Sergei Belousov's PyTorch implementation of OCR engine. The first step is to train the model with different characters before utilizing the model for an application. For training the model, we have to define the required parameters with appropriate values or they can be acquired at runtime. The neural net used here is ResNet-18. It is a pre-trained CRNN that has rich feature representation and can work with images of input size 224×224 .

```
#Initialize the required parameters
preTrained = None
sequence = "10x20"
testEpoch = 20
batch_size = 256
nNet = "resnet18"
inputSize = "320x32"
output_dir = "G:\\projects\\NLP\\Images\\Inputs"
cuda = False
```

Next, the data and model are loaded and the image is rotated, translated and scaled.

```
#Load data and model
data = TextDataset(data_path = data_path, mode = "train" ,
transform = transform)
net = load_model(data.get_abc(), sequence, nNet, preTrained, cuda)
loss_function = CTCLoss()

#Preprocessing
transform = Compose([
    Rotation(),
    Translation(),
    Scale(),
    Resize(size=(inputSize[0], inputSize[1]))
])
```

The training includes loading individual training data and converting it to a PyTorch variable. Wait calculation and backpropagation are performed based on this variable. The variable holds the value of the cost function. The *backward()* function performs the backpropagation and computes the gradients automatically.

```
#Training
pred_lens = Variable(Tensor([preds.size(0)] * batch_size).
int())      loss = loss_function(preds, labels, pred_lens,
label_lens) / batch_size
loss.backward()
nn.utils.clip_grad_norm(net.parameters(), 10.0)

#Testing
data.set_mode("test")
net = net.eval()
acc, avg_ed = test(net, data, data.get_abc(), cuda , visualize
= False)
```

7.6.5 | Complete Code

7.6.5.1 Train.py

```
import os
from models.model_loader import load_model
from torchvision.transforms import Compose
from dataset.data_transform import Resize, Rotation,
Translation, Scale
from dataset.text_data import TextDataset
from dataset.collate_fn import text_collate

import torch
from torch import nn
from torch.autograd import Variable
from torch import Tensor
from torch.utils.data import DataLoader
from warptc_pytorch import CTCLoss
from test import test

def main(data_path):
    #Initialize the required parameters
    preTrained = None
    sequence = "10x20"
    testEpoch = 20
    testing = True
    batch_size = 256
    nNet = "resnet18"
    inputSize = "320x32"
    output_dir = "G:\\projects\\NLP\\Images\\Inputs"
    cuda = False

    inputSize = [int(x) for x in inputSize.split('x')]

    #preprocessing
    transform = Compose([
    Rotation(),
    # Translation(),
    # Scale(),
    Resize(size=(inputSize[0], inputSize[1]))
    ])

    #Load data and model
    data = TextDataset(data_path=data_path, mode="train",
    transform=transform)
```

```

    net = load_model(data.get_abc(), sequence, nNet, pre-
Trained, cuda)
    loss_function = CTCLoss()

    epoch_count = 0
    while True:

        #Calling test - for testing
if (testEpoch is not None and epoch_count != 0 and epoch_count
% testEpoch == 0) or (testing and epoch_count == 0):
    print("Test phase")
    data.set_mode("test")
    net = net.eval()
acc, avg_ed = test(net, data, data.get_abc(), cuda, visualize =
False)
    net = net.train()
    data.set_mode("train")

    #training part
    data_loader = DataLoader(data, batch_size=batch_size, num_
workers=1, shuffle=True, collate_fn=text_collate)
    loss_mean = []
    iterator = len(data_loader)
    iter_count = 0
    for sample in iterator:
        imgs = Variable(sample["img"])
        labels = Variable(sample["seq"]).view(-1)
        label_lens = Variable(sample["seq_len"].int())
        if cuda:
            imgs = imgs.cuda()
            preds = net(imgs).cpu()

        pred_lens = Variable(Tensor([preds.size(0)] * batch_size).
int())
        loss = loss_function(preds, labels, pred_lens, label_lens)
/ batch_size
        loss.backward()
        nn.utils.clip_grad_norm(net.parameters(), 10.0)
        loss_mean.append(loss.data[0])
        iter_count += 1
        if output_dir is not None:
            torch.save(net.state_dict(), os.path.join(output_dir,
"crnn_" + nNet + "_" + str(data.get_abc()) + "_last"))
        epoch_count += 1
    return

```

7.6.5.2 Test.py

```

import cv2
from tqdm import tqdm
import numpy as np

import torch
from torch.autograd import Variable
from torch.utils.data import DataLoader
from dataset.test_data import TestDataset
from dataset.text_data import TextDataset
from dataset.collate_fn import text_collate
from dataset.data_transform import Resize, Rotation,
Translation, Scale
from models.model_loader import load_model
from torchvision.transforms import Compose
import editdistance

def test(net, data, abc, cuda, visualize, batch_size=256):
    data_loader = DataLoader(data, batch_size=batch_size,
num_workers=4, shuffle=False, collate_fn=text_collate)
    count = 0
    tp = 0
    avg_ed = 0
    iterator = tqdm(data_loader)
    for sample in iterator:
        imgs = Variable(sample["img"])
        if cuda:
            imgs = imgs.cuda()
        out = net(imgs, decode=True)
        gt = (sample["seq"].numpy() - 1).tolist()
        lens = sample["seq_len"].numpy().tolist()
        pos = 0
        key = ` `
        for i in range(len(out)):
            gts = ` `.join(abc[c] for c in gt[pos:pos+lens[i]])
            pos += lens[i]
            if gts == out[i]:
                tp += 1
            else:
                avg_ed += editdistance.eval(out[i], gts)
                count += 1
        if visualize:
            status = "pred: {}; gt: {}".format(out[i], gts)

```

```

        iterator.set_description(status)
        img = imgs[i].permute(1, 2, 0) .cpu().data.numpy().
astype(np.uint8)
        cv2.imshow("img", img)
        key = chr(cv2.waitKey() & 255)
        if key == 'q':
            break
        if key == 'q':
            break
        if not visualize:
            iterator.set_description("acc: {0:.4f}; avg_ed: {0:.4f}".
format(tp / count, avg_ed / count))
        acc = tp / count
        avg_ed = avg_ed / count
        return acc, avg_ed
def main(data_path):
    sequence = "10x20"
    nNet = "resnet18"
    inputSize = "320x32"
    output_dir = "G:\\projects\\NLP\\Images\\Inputs"
    visualize = True
    cuda = False
    inputSize = [int(x) for x in inputSize.split('x')]
    transform = Compose([
        Rotation(),
        Resize(size=(inputSize[0], inputSize[1]))
    ])

    if data_path is not None:
        data = TextDataset(data_path=data_path, mode="test",
transform=transform)
    else:
        data = TextDataset(transform=transform, abc=data.get_
abc())
    sequence = [int(x) for x in sequence.split('x')]
    net = load_model(data.get_abc(), sequence, nNet, output_
dir, cuda).eval()
    acc, avg_ed = test(net, data, data.get_abc(), cuda, visualize)
    print("Accuracy: {}".format(acc))
    print("Edit distance: {}".format(avg_ed))
if __name__ == '__main__':
    main()

```

7.6.6 | Result

The model was tested with several handwritten Tamil texts, and the average accuracy was calculated to be around 82%. The accuracy can be increased using a larger dataset and more epochs for training. Screenshot of a sample test data and the corresponding output is shown in Figs. 7.14(a) and 7.14(b).

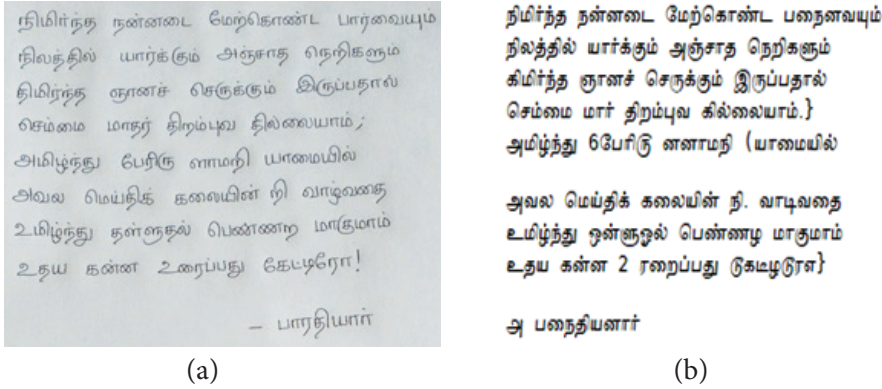


FIGURE 7.14 (a) Input image with handwritten text; (b) output image with predicted characters.

7.7 | FUTURE SCOPE

Data is a commercial commodity now, but a large reserve of unstructured data is no use unless it is trained and can be used. Just consider Facebook having millions of user data tagged, or Pinterest with data in each and every pin.

The amount of data grows exponentially so that the tasks to be carried out to extract useful information and the processes that can be done on that information need more efficacy. Neural networks, with their efficiency having proved are used in almost all the fields of AI. Hyperparameter tuning is the major concern when it comes to neural networks. With the parameters tuned to their best and powerful enough hardware and software support, neural networks can be made more efficient to achieve nearly 100% accuracy. Neural networks will definitely replace any traditional inefficient systems in all fields and will supplement AI to serve people.

SUMMARY

This chapter explains some of the real-world problems solved using deep learning. The problem that we are trying to solve has been defined and the program to solve it along with the dataset used is explained. This enables the reader to identify suitable problems that can be solved using deep learning and to code using appropriate open-source framework.

REVIEW QUESTIONS

1. Propose a sequence of steps to solve a problem using deep learning.
2. Why would RNN be a good choice to create a mobile keyboard software?

3. How does the convolution layer process any image matrix?
4. Would increasing the problem size have an impact on the CNN system? Justify your answer.
5. Change the kernel size of a sample application. How does it impact the output?

ASSIGNMENT PROBLEMS

1. Given an image, Google Assistant can recognize the image in it and also the text in English and other languages. Provide a solution for the above application.
2. Improve the accuracy of the Next Word Prediction problem by following the suggestions given in Section 7.5.6.
3. Predict whether the type of breast cancer is benign or malignant using neural networks and the Breast Cancer Wisconsin (Diagnostic) Data Set available in Kaggle (<https://www.kaggle.com/uciml/breast-cancer-wisconsin-data>).
4. Classify the SMS as either SPAM or HAM using recurrent networks and the SMS Spam Collection Data Set available in UCI Machine Learning Repository (<https://archive.ics.uci.edu/ml/datasets/sms+spam+collection>).

REFERENCES

1. Ahmed Rekik, Achraf Ben-Hamadou and Walid Mahdi. 2014. A new visual speech recognition approach for RGB-D cameras. *Proceedings of International Conference on Image Analysis and Recognition*
2. Amirsina Torfi, Seyed Mehdi Iranmanesh, Nasser Nasrabadi, and Jeremy Dawson. 2017. 3D convolutional neural networks for cross audio-visual matching recognition. *Proceedings of IEEE* [doi 0.1109/ACCESS.2017.2761539]
3. Amit Garg, Jonathan Noyola and Sameep Bagadia. 2016. Lip reading using CNN and LSTM. [http://cs231n.stanford.edu/reports/2016/pdfs/217_Report.pdf]
4. Eric Petajan, Bradford Bischoff, David Bodoff, and N. Michael. 1998. An improved automatic lipreading system to enhance speech recognition. [ACM SIGCHI, Washington D.C]
5. Gao Huang, Zhuang Liu, Laurens van der Maaten and Kilian Q. Weinberger. 2017. Densely connected convolutional networks. *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition (CVPR)*
6. Ibrahim Almajai, Stephen Cox, Richard Harvey, and Yuxuan Lan. 2016. Improved speaker independent lip reading using speaker adaptive training and deep neural networks. *International Conference on Acoustics, Speech, and Signal Processing*
7. Nazeeh Alothmany, Robert Boston, Ching Li, Susan Shaiman, and John Durrant. 2010. Classification of Visemes Using Visual Cues. *Proceedings of IEEE*
8. Prashant Borde, Amarsinh Varpe, Ramesh Manza, and Pravin Yannawar. 2015. Recognition of isolated words using Zernike and MFCC features for audio visual speech recognition. *International Journal of Speech Technology*. 18(2) [doi 10.1007/s10772-014-9257-1]
9. Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of IEEE*

Bibliography

1. Matthew D. Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. *European Conference on Computer Vision (ECCV)*, Part I, LNCS 8689, pp. 818–833.
2. Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Procedure of IEEE*.
3. <https://www.tutorialspoint.com/python/>
4. <https://www.marutitech.com/top-8-deep-learning-frameworks/>
5. <https://www.geekboots.com/story/cpu-vs-gpu-vs-tpu>
6. <https://blog.slavv.com/picking-a-gpu-for-deep-learning-3d4795c273b9>
7. https://www.tensorflow.org/api_docs

- A**
 - activation function, 8
 - AdaDelta, 33
 - AdaGrad, 32
 - Adam, 33–34
 - AlexNet, 56–57
 - AND function, 9–10, 15
 - artificial neural network (ANN), 7–37, 91
 - types, 8–19
 - autoencoders, 91–92
 - features of, 92
 - types of, 93–105
 - average pooling, 50–51
 - axon, 7
- B**
 - backpropagation algorithm, 25–26
 - backpropagation of error, 27–29
 - backpropagation through time (BPTT), 74–75
 - batch gradient, 24
 - Bernoulli–Bernoulli RBM, 114
 - bias, 6, 11
 - bidirectional RNNs, 79–80
 - binary classification problem, 12–13
 - binary step function, 8
 - biological neural network, 7
 - biological neuron, 7
 - Boltzmann machines, 109
- C**
 - central processing unit, 137
 - classification
 - definition, 2
 - classification model, 3
 - code, 94
 - conditional RBM, 114
 - contractive autoencoders, 102–103
 - convolutional autoencoder, 99–100
 - convolutional layer of CNN, 42
 - convolutional neural network (CNN), 41–42
 - applications of, 61
 - architectures of, 54–61
 - components of, 42–51
 - convolutional layer, 42
 - feature map, 47–48
 - flattening layer, 42, 51
 - fully connected layer, 42
 - image classification using, 143–151
 - layers, 41–42
 - main operation in, 41
 - pooling or downsampling layer, 42, 48–51
 - translation invariance, 54
 - visual speech recognition using 3D-, 151–161
 - weight sharing, 53
 - convolutional neural networks (CNNs)
 - vs recurrent neural networks (RNNs), 70
 - convolutional recurrent neural network (CRNN), 178
 - cross-entropy loss (CEL) function, 21
 - customer relationship management (CRM), 87
- D**
 - data flow programming, 122
 - decoder, 94
 - deep autoencoder, 97–98
 - deep belief networks (DBN), 114–115
 - deep learning, 1
 - definition, 36
 - functioning of, 37
 - deep recurrent neural network, 86–88
 - denoising autoencoder, 98–99
 - DenseNet, 61, 63
 - depth of feature map, 47
 - dimensionality reduction, 92
 - downsampling, 50
- E**
 - encoder, 93
 - epoch, 22
 - error function, 19–21
 - minimization of, 22
 - errors, calculation for, 26–27
 - Euclidean norm of a matrix, 102
 - exploding gradient problem, 34, 77
 - exponential linear units (ELUs), 52–53

F

face recognition, 54, 61, 64
 feature extraction, 37
 feature hierarchy, 87
 feedforward network, 18
 feedforward neural network, 70
 flattening layer of CNN, 42, 51
 Frobenius norm, 102–103
 fully connected layer of CNN, 42

G

gated recurrent unit (GRU), 85–86
 Gaussian–Bernoulli RBM, 114
 generalization, 3
 generative adversarial network (GAN), 51
 GoogLeNet, 56, 59
 gradient clipping, 34
 gradient descent, 21–24
 graphics processing unit, 138–139

H

handwritten digit recognition, 66
 handwritten text recognition using CRNN
 code snippets, 178–179
 complete code, 180–183
 dataset, 178
 language and framework, 178
 problem description, 178
 result, 184
 hidden layers
 calculation for, 26
 updating weights at, 29–30
 Hopfield networks, 110
 house price dataset, 3, 5
 hyperbolic tangent, 15

I

identity encoding, 99
 identity shortcut connection, 61
 image classification using CNN
 architecture of CNN used in, 147
 code snippets, 144–147
 complete code, 148–150
 dataset, 144
 language and framework, 144
 problem description, 143–144
 result, 150–151
 ImageNet Large Scale Visual Recognition
 Competition (ILSVRC) 2012, 56

Intel Movidius Myriad 2 VPU, 139
 Intel Movidius Myriad X VPU, 139
 iris flower dataset, 2–3
 iris setosa, 2
 iris versicolor, 2
 iris virginica, 2

J

Jacobian matrix, 102

K

Keras
 APIs, 129
 configuration, 128–129
 features of, 127–133, 128
 implementation of, 130–133
 setting up, 128

L

leaky ReLU, 52
 learning algorithm, 13–14
 learning rate, 23–24
 least absolute deviations (LAD), 21
 least square error (LSE), 21
 LeNet, 54–55
 linear classifiers, 15
 linear discriminant analysis (LDA), 91
 linear functions, 18
 linearly separable problems, 15–18
 linear regression, 3
 linear relationship, 4
 logistic function, 14–15
 long short term memory (LSTM), 88
 long short-term memory (LSTM), 80–85
 implementation steps of, 85
 long-term dependencies, 71
 loss function of sparse autoencoders, 102
 loss function of variational autoencoder, 103–105
 L1 regularization, 35
 L2 regularization, 35

M

machine learning, 1
 problems solved using, 2–5
 types of, 1–2
 many-to-many for sequence input and sequence
 output topology, 75
 many-to-many for synced sequence input and
 output topology, 75

- many-to-one topology, 75
- Matplotlib, 120
- MAX operation, 48
- Maxout neuron, 53
- max pooling, 50–51
- McCulloch–Pitts model
 - of network, 8–10
 - of neuron, 9
- memory forward, process of transmitting, 71
- mini-batch gradient descent, 24–25
- Modified National Institute of Standards and Technology (MNIST) dataset, 66
- momentum-based gradient descent, 31
- multilayer autoencoder, 95
- multilayer perceptron (MLP), 18–19, 51

N

- narrow convolution, 48
- nesterov accelerated gradient (NAG), 31–32
- Neural Compute Stick (NCS), 139–140
- neural network model, 124–126
- next-word prediction using RNN-LSTM
 - code snippets, 170–174
 - complete code, 174–177
 - dataset, 170
 - language and framework, 169
 - problem description, 169
 - result, 177
- non-linear functions, 18–19
- non-negative matrix factorization (NMF), 91
- numerical Python (NumPy), 120

O

- object detection, 54, 61
- one-to-many topology, 75
- one-to-one topology, 75
- OpenCV, 54
- optical character recognition (OCR), 54, 66
- optimal fit, 5–6
- optimization techniques, 19–31
- OR function, 15
- output layers, calculation for, 26
- overcomplete autoencoder, 99
- overfitting, 5–5

P

- Pandas, 120
- perceptron, 10–14

- pooling or downsampling layer of CNN, 42, 48–51
- principal component analysis (PCA), 91
- Python

- central processing unit, 137
- characteristics, 117
- deep learning with, 119
- environmental setup, 118–119
- features, 118
- frameworks, 120–136
- graphics processing unit, 138–139
- hardware support for deep learning, 136–140
- tensor processing unit (TPU), 140
- vision processing unit (VPU), 138–139

PyTorch, 133–135

- ecosystem of tools and libraries, 134–135
- features, 134
- implementation, 135–136

R

- randomized ReLU, 52
- raw media, 87
- receptive fields, 41
- Rectified Linear Units (ReLU) layers, 52
- recurrent neural networks (RNNs), 69–70, 133
 - bidirectional and stateful, 79–80
 - deep, 86–88
 - simple, 71–85
 - stock market prediction using, 161–169
 - three-layered, 77
 - topology, 75–76
 - training, 73
 - vs convolutional neural networks (CNNs), 70
 - vs feedforward neural network, 70
- regression line, 4–5
- regression problem, 3–5
- regularization in autoencoders, 100–105
- reinforcement learning, 2
- ReLU node, 52
- ResNet, 61–62
- restricted Boltzmann machines (RBMs), 87, 110
 - Bernoulli–Bernoulli, 114
 - conditional, 114
 - contrastive divergence, 112–113
 - deep belief networks (DBN), 114–115
 - energy-based, 110
 - example, 113
 - Gaussian–Bernoulli, 114

Gibbs distribution, 111
 Gibbs sampling, 111–112
 RMSProp, 32–33

S

Scaled ELU (SELU), 53
 scene labeling, 65
 Scientific Python (SciPy), 119
 sequential model, 130
 sigmoid function, 14–15
 sigmoid neuron, 14–15
 skip connections, 61
 sparse autoencoders, 100–102
 loss function of, 102
 regularization term in, 101
 spatial pooling, 50
 stacked autoencoders, 95–97
 stateful RNNs, 79–80
 stochastic gradient descent, 24
 stock market prediction using RNN, 161–169
 code snippets, 163–165
 complete code, 165–168
 dataset, 162
 language and framework, 162
 problem description, 161–162
 result, 169
 stride of feature map, 47–48
 subsampling, 50
 sum pooling, 51
 supervised learning, 1
 synapses, 7

T

TensorFlow, 121–126
 advantages, 127
 APIs, 122–123
 disadvantages, 127
 enactments in other languages, 126
 implementing, 123–126
 setting up, 123
 tensor processing unit (TPU), 140
 transition matrix, 71
 translation invariance in CNN, 54

two-dimensional convolutional neural networks
 (2D CNNs), 131–132

U

undercomplete autoencoder, 92
 underfitting, 5–5
 uniform initialization, 35–36
 unsupervised learning, 2

V

valid padding, 48
 vanilla autoencoder, 93
 loss function for, 95
 structure of, 93–95
 vanishing gradient problem, 34, 77
 variance, 6
 variational autoencoders, 51, 103
 VGGNet, 56, 60
 vision processing unit (VPU), 138–139
 visual speech recognition using 3D-CNN
 code snippets, 152–154
 complete code, 154–160
 dataset, 144
 language and framework, 144
 problem description, 151
 result, 160–161

W

weight decay, 100
 weight initialization, 35
 weight regularization, 35
 weight sharing in CNN, 53
 weight updation, 23, 24, 25, 31–34
 wide convolution, 48

X

Xavier initialization, 36

Z

zero error, 6
 zero initialization, 35
 zero-padding, 48
 ZFNet, 56, 58

About the Book

With the world moving towards artificial intelligence, it has become mandatory for anyone seeking to work in this field to have considerable knowledge in deep learning. *Deep Learning Using Python* aims to lay a strong foundation for students and other professionals in the area of deep learning. It gives a hands-on experience on common deep learning applications using popular frameworks of the most suitable language, Python. This book will help readers to solve artificial intelligence problems using neural networks and deep learning architectures.

Salient Features

- Strong foundation in the **Fundamentals of Deep Learning**.
- Detailed explanation with elaborate examples of **Deep Learning Architectures**.
- Detailed analysis of **Convolutional Neural Networks**.
- Study on **Recurrent Neural Networks**.
- Examination of **Autoencoders**.
- Familiarization with **Restricted Boltzmann Machines**.
- Elucidation of **Open-Source Frameworks for Deep Learning**.
- **Real-World Applications** of deep learning.

Visit www.wileyindia.com for

- Datasets for reader to practice.
- Code for all Python examples to provide quick start to the readers and help teachers.

follow us on



facebook.com/wileyindia



twitter.com/wileyindiapl



linkedin.com/in/wileyindia



google.com/+wileyindia

SHELVING CATEGORY
Engineering

Wiley India Pvt. Ltd.

Customer Care +91 120 6291100
csupport@wiley.com
www.wileyindia.com
www.wiley.com

WILEY

ISBN: 978-81-265-7991-4

